

Università degli Studi di Napoli
Federico II

A.A. 2024-2025

Programmazione

Estratto

Questa dispensa raccoglie, riorganizza e spiega il materiale del corso di Programmazione tenuto dal professor *Francesco Isgrò* per il corso di laurea triennale in Informatica della Federico II di Napoli (slide dell'A.A. 2024-2025). Il linguaggio del corso è il **C**.

È pensata per chi **parte da zero**: non si dà per scontato di aver mai scritto una riga di codice, né di aver mai usato un terminale. Il primo capitolo accompagna passo passo nell'installazione di tutto ciò che serve (l'editor **Visual Studio Code**, il compilatore e le estensioni) su Windows, macOS e Linux; i successivi seguono l'ordine delle lezioni.

L'obiettivo è **capire**, non memorizzare: ogni regola è accompagnata dal suo *perché*, ogni programma è stato compilato ed eseguito e ne viene mostrato l'output reale, e ciò che accade *dentro* il computer (le variabili in memoria, i puntatori, le chiamate di funzione, i nodi di una lista) è disegnato in uno schema. Dove le slide contengono una svista, il testo la corregge e lo segnala.

Sono inclusi **soltanto i contenuti tecnici** presentati a lezione; le informazioni organizzative (orari, modalità d'esame, appelli) restano sulle slide e sui canali del docente, dove vengono aggiornate. Questo documento **non sostituisce** le lezioni, le slide e i libri di testo: ne è un complemento.

Nel testo ricorrono alcuni elementi grafici: riquadri **verdi** con approfondimenti, **gialli** con avvertenze ed errori tipici, **rossi** con errori gravi da evitare, riquadri **Definizione** e riquadri **In sintesi** che riassumono il ragionamento. Il codice è disegnato come una finestra di VS Code e, **accanto**, compare ciò che produce: il **terminale**, dove i comandi digitati sono in bianco e ciò che si scrive mentre il programma è in esecuzione è in **giallo**, oppure uno schema.

Redazione a cura di **Fabio Rosiello**

Indice

1 L'ambiente di lavoro 7

Che cosa ci serve 7
Linux, Windows o macOS? 8
Passo 1: installare Visual Studio Code 9
Scaricare il programma 9
Installazione su Windows 10
Installazione su macOS 10
Installazione su Linux 11
Passo 2: installare il compilatore 11
Windows: WSL e Ubuntu 11
Windows: creare l'utente di Ubuntu 12
Windows: installare il compilatore in Ubuntu 12
macOS: gli strumenti da riga di comando 13
Linux 14
Passo 3: le estensioni di VS Code 14
Windows: collegare VS Code a Ubuntu 15
Passo 4: la cartella del corso 15
Com'è fatta la finestra di VS Code 16
Creare un file 17
Il terminale 17
Il prompt 18
Cartelle e percorsi 18
I comandi essenziali 19
Trucchi che fanno risparmiare tempo 20
Il primo programma 21
Compilare ed eseguire 21
Perché `./`? 22
Gli errori di compilazione 22
E il pulsante "Run"? 23
Quando qualcosa non funziona 24
Riferimenti 24

2 Il calcolatore 25

Che cos'è la programmazione 25
Lo schema di un calcolatore 26
La CPU e il linguaggio macchina 26
Che cosa sa fare un processore 26
Il linguaggio macchina 27
L'assembly 27
Un esempio: contare i bit a 1 28
I limiti dell'assembly 28
I livelli di astrazione 29
Algoritmo e programma 30
La memoria 30
Celle e indirizzi 30
La notazione binaria 31
Quanti numeri in un byte 31
I numeri negativi 32
I tipi di dato 32
Gli stessi bit, significati diversi 32

Il tipo come astrazione 33
La gerarchia di memoria 33
Il software 34
Gli strati del software 34
L'avvio del computer 34
Che cosa fa il sistema operativo 35
I servizi per i programmi 35
Installare e scrivere un sistema operativo 36
Riferimenti 36

3 Dal problema al programma 37
Programmare 37
I linguaggi di alto livello 37
Interprete e compilatore 38
Una via di mezzo: il bytecode 38
Le librerie 39
Come nasce un eseguibile 39
Compilazione e linking 40
La catena completa 41
Librerie statiche e dinamiche 42
Gli algoritmi 43
Che cos'è un algoritmo 43
Quando le risorse non bastano 44
Come si descrive un algoritmo 44
Esempio 1: l'area di un cerchio 45
Esempio 2: da Fahrenheit a Celsius 46
Esempio 3: il segno di un numero 47
Riferimenti 48

4 La programmazione strutturata 49
Il linguaggio C 49
Da dove viene 49
Lo standard 50
La programmazione strutturata 50
Il problema: gli spaghetti 50
Il teorema di Böhm e Jacopini 51
Le tre strutture di controllo 51
La sequenza 52
La selezione a una via 52
La selezione a due vie 53
La selezione a n vie 54
L'iterazione 55
Il ciclo su contatore 56
Il ciclo su condizione 56
Condizione in coda 57
Condizione in testa 58
Combinare le strutture 59
Lo sviluppo top-down 59
Dividere il problema 59
Come si invoca un sotto-programma 60
I tipi di istruzione 61
Riferimenti 62

5 Primi programmi in C 63
Il primo programma, riga per riga 63
Scrivere, compilare, eseguire 63
Il preprocessore e `#include` 64
La funzione `main` 64
`printf`, le stringhe e il punto e virgola 64
`return` e il valore restituito al sistema 65
L'output con `printf` 65
Il cursore 65
Disegnare con i caratteri 66

Variabili, espressioni e assegnazione 66
Il problema della maratona 66
I commenti 66
Le dichiarazioni 67
L'assegnazione 67
Come si calcola un'espressione 67
La trappola della divisione intera 68
Parole chiave e nomi 68
Il preprocessore: `#define` e `#include` 68
Le costanti simboliche 68
I file di intestazione 68
Header e libreria: la prova 69
Un esempio con le costanti: Pacific Sea 70
Il programma 70
Gli `#include` a catena 71
Le costanti dell'header 71
`const` 71
I tipi reali e la notazione scientifica 71
`printf` e `scanf` 72
Funzioni di input e output formattato 72
Le conversioni di `printf` 72
`scanf` e l'operatore `&` 73
Un esempio: echo 74
Il controllo del flusso 75
Vero e falso in C 75
`if` 75
`if-else` 76
`while` 77
Assegnazioni abbreviate e incremento 78
`for` 78
Un esempio completo: `running_sum` 79
Il programma 79
Riga per riga 79
Tre modi di dare l'input 80
Una versione senza la lettura iniziale 81
Quando la compilazione fallisce 82
Le funzioni 83
Dichiarare prima di usare 83
L'esempio maxmin 84
Come funziona 85
Due `x` diverse 85
Che cosa succede senza prototipi 86
Un punto e virgola di troppo 86
Il passaggio dei parametri per valore 87
Riferimenti 88

6 Elementi lessicali e operatori 89

Dai caratteri ai token 90
I token 90
Gli spazi 91
Identificatori della libreria 91
I commenti 92
Le parole chiave 92
Gli identificatori 93
La regola 93
Maiuscole, lunghezza, scelta dei nomi 93
Gli operatori 94
Precedenza e associatività 94
La tabella degli operatori 95
Incremento e decremento 96
Prefisso e postfisso 96
Un esempio da simulare 96
Quando `++` diventa pericoloso 97
La libreria standard e i nomi riservati 98
Riferimenti 99

7 Programmi su più file e make 100

Perché più file 100
- Un piccolo progetto 101
- Compilare separatamente, collegare insieme 102

Il progetto delle lezioni 103
- I file 103
- Il problema 103

make 104
- L'idea: dipendenze e date 104
- Il makefile 104
- I tipi di riga 105
- Che cosa fa `make`, passo per passo 105
- Le dipendenze dall'header 106
- Il nome del target 107
- Le macro 107

Un makefile completo 108

Riferimenti 109

L'AMBIENTE DI LAVORO

Prima di scrivere il primo programma bisogna preparare il proprio *laboratorio*: qualche programma da installare e un paio di abitudini da prendere. Questo capitolo accompagna passo passo dall'installazione alla prima esecuzione, su **Windows**, **macOS** e **Linux**. Ogni passo è spiegato: non serve sapere nulla di computer oltre ad accenderli e a scaricare un file da Internet.

Leggi solo la parte del tuo sistema operativo

Le istruzioni di installazione sono divise per sistema operativo. Segui quelle del tuo computer e salta le altre: dalla sezione *Il terminale* in poi le istruzioni tornano a essere le stesse per tutti.

CHE COSA CI SERVE

Un computer non capisce il C. Capisce soltanto il proprio **linguaggio macchina**: lunghe sequenze di numeri che il processore sa eseguire, una dopo l'altra. Noi invece scriveremo i programmi in C, un linguaggio fatto di parole inglesi e simboli, comprensibile a una persona. Fra noi e il processore servono quindi tre strumenti, che compaiono nella figura 1.1.

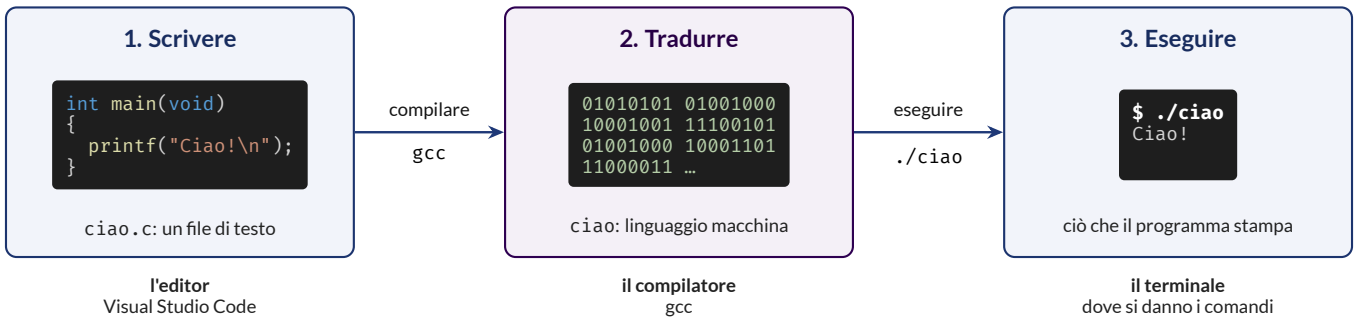


Figura 1.1: *Dal testo scritto da noi al programma che gira: i tre strumenti da installare.*

- L'**editor** è il quaderno su cui scriviamo il programma. Il programma, per ora, è un normale **file di testo** con estensione `.c` (per esempio `ciao.c`). Useremo **Visual Studio Code** (o semplicemente *VS Code*), un editor gratuito che colora il codice, segnala gli errori mentre si scrive e contiene al suo interno anche un terminale.
- Il **compilatore** è il traduttore: legge il file di testo e produce un nuovo file, l'**eseguibile**, che contiene lo stesso programma tradotto in linguaggio macchina. Useremo **gcc**, il compilatore C del progetto GNU, lo stesso usato a lezione.
- Il **terminale** è una finestra in cui si *scrivono* i comandi al computer invece di cliccare su icone. Da lì chiederemo a gcc di compilare e poi lanceremo il programma, che stamperà i suoi risultati nella stessa finestra.

Il capitolo 3 spiegherà in dettaglio che cosa succede durante la traduzione. Per ora basta l'idea: **si scrive, si compila, si esegue**.

Linux, Windows o macOS?

Il corso **assume Linux** come sistema operativo: i comandi visti a lezione (e all'esame) sono quelli di Linux. Non serve però cambiare computer o cancellare Windows: in ogni sistema c'è un modo semplice per avere gli stessi strumenti.

Sistema	Che cosa si installa	Perché
Windows	**WSL** (*Windows Subsystem for Linux*) con **Ubuntu**, e dentro Ubuntu gcc	WSL fa girare un vero Linux dentro Windows, senza riavvii né partizioni
macOS	gli **strumenti da riga di comando** di Apple (*Command Line Tools*)	macOS è un parente di Linux: ha già un terminale molto simile, manca solo il compilatore
Linux	il pacchetto `build-essential` (Ubuntu, Debian, Mint)	è il sistema del corso: basta un comando

VS Code si installa allo stesso modo ovunque (passo 1). Su Windows c'è una particolarità che conviene capire subito, perché spiega diversi passaggi successivi: i file del corso e il compilatore stanno *dentro* Ubuntu, mentre la finestra di VS Code è un normale programma di Windows. VS Code allora si divide in due parti che collaborano (figura 1.2): l'interfaccia, su Windows, e un piccolo *server*, dentro Ubuntu, che legge e salva i file e lancia il compilatore.

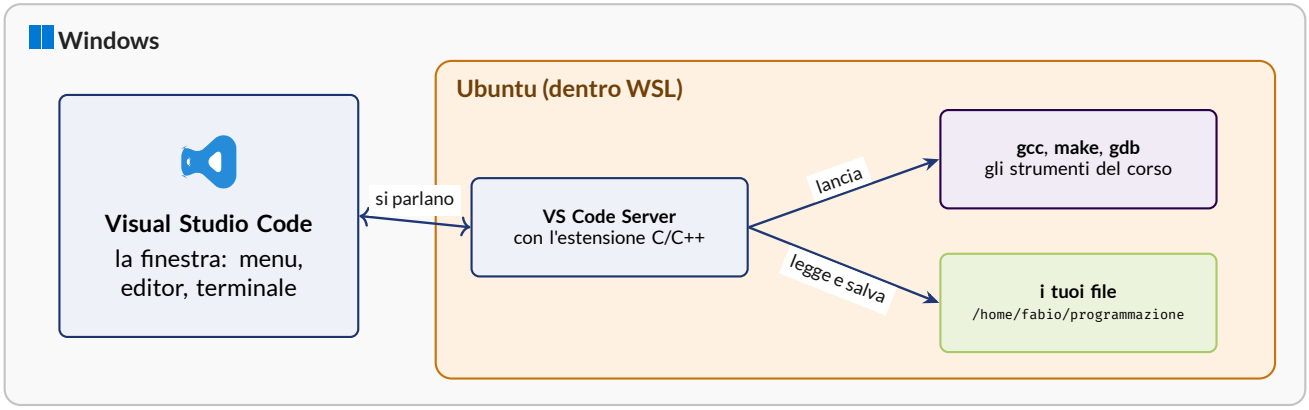


Figura 1.2: *Su Windows, con WSL: l'interfaccia di VS Code resta su Windows, mentre file e compilatore vivono in Ubuntu. Su macOS e su Linux tutto sta nello stesso sistema e questa divisione non c'è.*

Perché non un compilatore "nativo" per Windows?

Esistono versioni di gcc per Windows (MinGW, per esempio) e funzionano. Ma con esse alcuni dettagli cambiano: l'eseguibile si chiama `a.exe` invece di `a.out`, la fine dell'input si segnala con Ctrl + Z invece di Ctrl + D, i comandi del terminale sono diversi. Con WSL si usano **esattamente** i comandi delle lezioni, ed è ciò che conta.

Passo 1: installare Visual Studio Code

Scaricare il programma

Visual Studio Code si scarica gratuitamente dal sito ufficiale, `https://code.visualstudio.com`. La pagina *Download* propone un pulsante per ogni sistema operativo (figura 1.3): si sceglie il proprio.

Visual Studio Code, non Visual Studio

Microsoft produce anche **Visual Studio** (senza "Code"): è un altro programma, molto più grande, pensato per altri linguaggi. Il nome giusto è **Visual Studio Code**, con l'icona blu della figura 1.3.

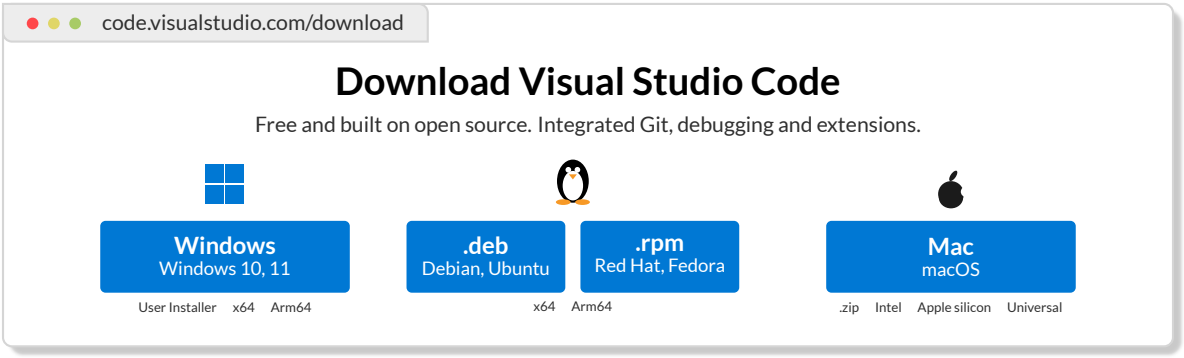


Figura 1.3: *La pagina di download di VS Code. L'aspetto del sito cambia spesso; i tre gruppi di pulsanti, invece, restano.*

Installazione su Windows

Il pulsante *Windows* scarica un file chiamato più o meno `VSCodeUserSetup-x64-1.xx.x.exe` (i numeri sono la versione). È un normale programma di installazione:

1. Apri il file scaricato (doppio clic, oppure clic sulla notifica di download del browser).
2. Nella prima schermata scegli *I accept the agreement* (accetti la licenza) e fai clic su *Next*.
3. Nelle due schermate successive (cartella di installazione e menu Start) lascia tutto com'è e fai clic su *Next*.
4. Nella schermata *Select Additional Tasks* (figura 1.4) spunta le due voci *Add "Open with Code" action…*: aggiungono la voce "Apri con Code" al menu del tasto destro sui file e sulle cartelle, comodissima. La voce **Add to PATH** deve restare spuntata: permette di aprire VS Code scrivendo `code` nel terminale, e la useremo.
5. Fai clic su *Next*, poi su *Install*. Al termine, lascia spuntato *Launch Visual Studio Code* e fai clic su *Finish*.

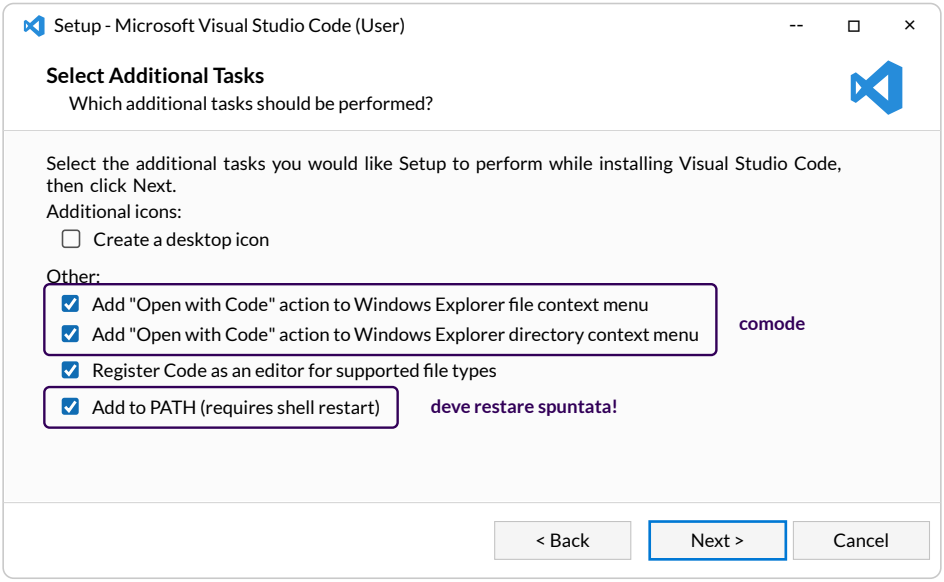


Figura 1.4: *La schermata più importante dell'installazione su Windows.*

Installazione su macOS

Il pulsante *Mac* scarica un archivio `.zip`. Safari di solito lo estrae da solo: nella cartella *Download* compare l'applicazione *Visual Studio Code* (altrimenti basta un doppio clic sul file `.zip`).

1. Trascina *Visual Studio Code* dalla cartella *Download* alla cartella *Applicazioni* (figura 1.5): così è installata davvero e la si trova nel Launchpad.
2. Aprila. Al primo avvio macOS avvisa che è un'applicazione scaricata da Internet: conferma con *Apri*.
3. Per poter aprire VS Code dal terminale con il comando `code`: premi Cmd + Shift + P, scrivi *shell command* e scegli *Shell Command: Install 'code' command in PATH*.

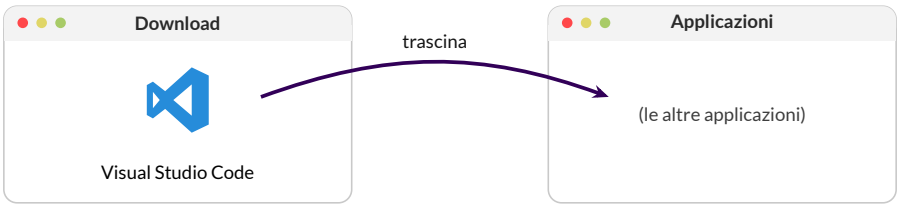


Figura 1.5: *Su macOS si installa un'applicazione trascinandola nella cartella Applicazioni.*

Installazione su Linux

Su Ubuntu, Debian e derivate si scarica il pacchetto `.deb` e lo si installa dal terminale (il terminale si apre con Ctrl + Alt + T; il suo uso è spiegato più avanti in questo capitolo):

Terminale

```
$ cd ~/Scaricati
$ sudo apt install ./code_*.deb
```

Il comando `cd` entra nella cartella dei download (su un sistema in inglese si chiama `Downloads`); `sudo apt install` installa il pacchetto, dopo aver chiesto la password dell'utente. In alternativa, VS Code si trova anche nel negozio di applicazioni della distribuzione (su Ubuntu, *App Center*).

Passo 2: installare il compilatore

Windows: WSL e Ubuntu

Su Windows 10 (aggiornato) e Windows 11 WSL si installa con un unico comando, da scrivere in **PowerShell** aperta **come amministratore**, cioè con il permesso di modificare il sistema.

1. Premi il tasto Windows, scrivi *powershell* e, nel riquadro a destra, scegli *Esegui come amministratore* (figura 1.6). Windows chiede conferma: rispondi *Sì*.
2. Nella finestra blu (o nera) che si apre, scrivi il comando `wsl --install` e premi Invio.
3. Attendi il download (qualche minuto: Ubuntu pesa circa un gigabyte) e, quando richiesto, **riavvia il computer**.

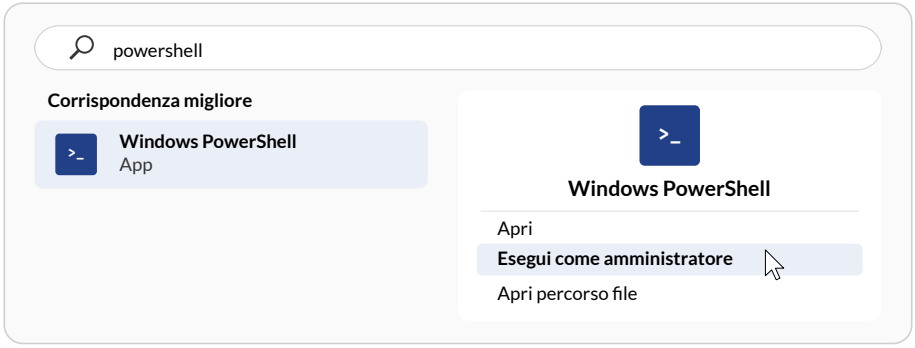


Figura 1.6: *La ricerca nel menu Start: PowerShell va aperta come amministratore.*

Amministratore: Windows PowerShell

```
PS C:\WINDOWS\system32> wsl --install
Installazione: Piattaforma macchina virtuale
Piattaforma macchina virtuale è stato installato.
Installazione: Sottosistema Windows per Linux
Sottosistema Windows per Linux è stato installato.
Installazione: Ubuntu
Ubuntu è stato installato.
L'operazione richiesta è riuscita. Le modifiche saranno effettive al riavvio del sistema.
```

Windows: creare l'utente di Ubuntu

I messaggi possono essere un po' diversi a seconda della versione di Windows: quello che conta è che non compaiano errori. Dopo il riavvio si apre da sola una finestra di Ubuntu (se non si apre, cerca *Ubuntu* nel menu Start) che completa l'installazione e chiede di creare un **utente Linux**: un nome (in minuscolo, senza spazi; non deve essere per forza uguale a quello di Windows) e una password.

Ubuntu

```
Installing, this may take a few minutes...
Please create a default UNIX user account. The username does not need to match your Windows username.
For more information visit: https://aka.ms/wslusers
Enter new UNIX username: fabio
New password:
Retype new password:
passwd: password updated successfully
Installation successful!
fabio@PC-FABIO:~$
```

Mentre scrivi la password non compare nulla

Nel terminale, mentre si digita una password, **non compaiono nemmeno gli asterischi**: sembra che la tastiera non funzioni, invece i caratteri vengono letti. Scrivi la password e premi Invio. Annotala: servirà ogni volta che installerai qualcosa.

Windows: installare il compilatore in Ubuntu

L'ultima riga stampata da Ubuntu, `fabio@PC-FABIO:~$`, è il **prompt**: Ubuntu è pronta a ricevere comandi (la spiegheremo pezzo per pezzo più avanti). Restano da installare il compilatore e gli altri strumenti del corso. Scrivi questi due comandi, uno alla volta, premendo Invio dopo ciascuno:

Ubuntu

```
$ sudo apt update
[sudo] password for fabio:
...
$ sudo apt install build-essential gdb
...
Do you want to continue? [Y/n] y
...
```

- `sudo` (*superuser do*) esegue il comando che segue con i permessi di amministratore, e per questo chiede la password appena creata.
- `apt` è il **gestore di pacchetti** di Ubuntu: una specie di negozio di applicazioni che si usa dal terminale. Con `update` scarica l'elenco aggiornato dei programmi, con `install` installa quelli indicati (e chiede conferma: si risponde `y`, *yes*).
- `build-essential` è un pacchetto che ne raccoglie altri: il compilatore **gcc**, il programma **make** (che useremo dal capitolo 7) e le *librerie* standard del C. **gdb** è il *debugger*, utile per eseguire un programma un'istruzione alla volta.

Per verificare che tutto sia andato a buon fine, si chiede a gcc la sua versione:

Ubuntu

```
$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions.  There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

I numeri di versione possono essere diversi: va bene qualunque versione. Se invece compare `gcc: command not found`, l'installazione non è riuscita: ripeti i due comandi `apt` e controlla che non stampino errori.

Se `wsl --install` non funziona

L'errore più comune all'avvio di Ubuntu è un messaggio che parla di *virtualizzazione* (per esempio il codice `0x80370102`): WSL usa una funzione del processore che su alcuni computer è disattivata. Si attiva dal BIOS/UEFI del computer (la voce si chiama *Virtualization Technology*, *Intel VT-x* o *AMD-V*); la procedura cambia da modello a modello, quindi conviene cercare "attivare virtualizzazione" seguito dal modello del proprio PC. Le istruzioni ufficiali di Microsoft sono indicate nei riferimenti a fine capitolo.

macOS: gli strumenti da riga di comando

Su macOS il compilatore fa parte degli *strumenti da riga di comando* di Apple, gratuiti. Non serve installare Xcode (l'ambiente di sviluppo completo, molto più grande): bastano gli strumenti.

1. Apri il **Terminale**: premi Cmd + Spazio (si apre Spotlight), scrivi *terminale* e premi Invio.
2. Scrivi il comando `xcode-select --install` e premi Invio.
3. Compare una finestra come quella della figura 1.7: fai clic su *Installa* (non su *Ottieni Xcode*) e accetta la licenza. Il download richiede qualche minuto.

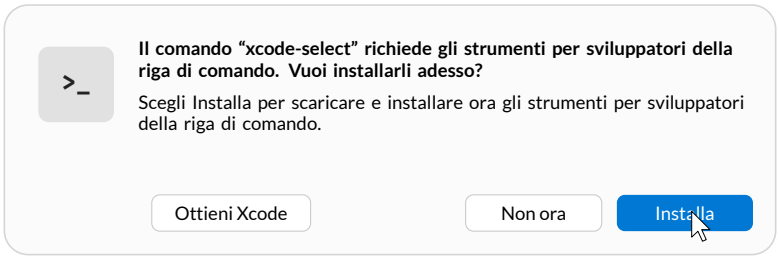


Figura 1.7: *La richiesta di installazione degli strumenti da riga di comando (il testo esatto dipende dalla versione di macOS).*

Al termine, la verifica:

Terminale di macOS

```
$ gcc --version
Apple clang version 16.0.0 (clang-1600.0.26.6)
Target: arm64-apple-darwin24.1.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
```

Sorpresa: il comando `gcc` risponde *Apple clang*. Su macOS il compilatore è **clang**, un altro compilatore C, che per comodità risponde anche al nome `gcc` e accetta le stesse opzioni. Per tutto ciò che si fa nel corso il comportamento è lo stesso.

Il prompt di macOS

Nel Terminale di macOS il prompt finisce con il simbolo `%` invece che con `$` (per esempio `fabio@MacBook-Air ~ %`). In questa dispensa i comandi sono sempre preceduti da `$`: su Mac si scrivono allo stesso modo.

Linux

Su Ubuntu, Debian e Linux Mint bastano gli stessi due comandi visti per Windows (in Ubuntu, il terminale si apre con Ctrl + Alt + T):

Terminale

```
$ sudo apt update
$ sudo apt install build-essential gdb
```

Su Fedora il comando equivalente è `sudo dnf install gcc make gdb`. La verifica, anche qui, è `gcc --version`.

Passo 3: le estensioni di VS Code

VS Code, appena installato, sa già colorare il codice C. Le **estensioni** sono componenti aggiuntivi gratuiti che gli insegnano a fare di più. Si installano dalla vista *Extensions*: si apre con l'icona dei quattro quadratini nella barra a sinistra (oppure con Ctrl + Shift + X, su Mac Cmd + Shift + X); si scrive il nome nella casella di ricerca e si fa clic su Install (figura 1.8).

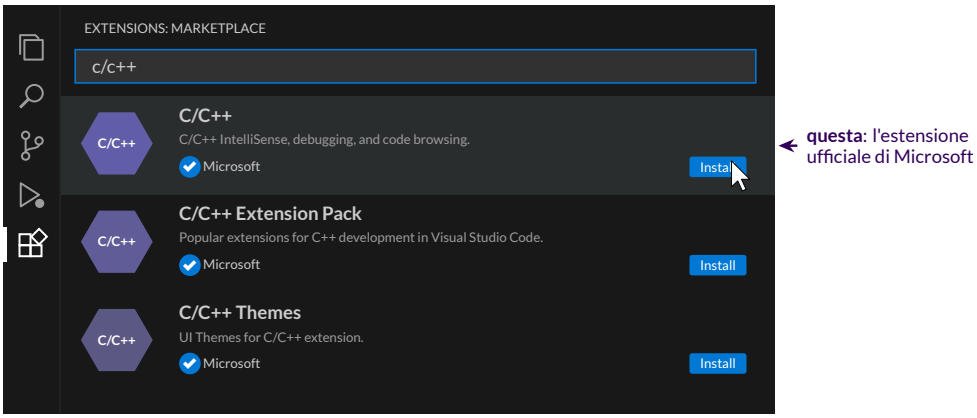


Figura 1.8: *La vista Extensions con la ricerca "c/c++". Basta la prima: C/C++ di Microsoft (si riconosce dal segno blu di "editore verificato").*

Le estensioni da installare sono poche:

Estensione	Per chi	A che cosa serve
C/C++ (Microsoft)	tutti	*IntelliSense*: suggerimenti mentre si scrive (nomi di funzione, parametri), errori sottolineati in rosso ancora prima di compilare, navigazione nel codice (clic destro, *Go to Definition*), debugger.
WSL (Microsoft)	solo Windows	collega VS Code a Ubuntu, come nella figura 1.2.

Spesso non serve nemmeno cercarle: aprendo per la prima volta un file `.c`, VS Code propone da solo, con una notifica in basso a destra, di installare l'estensione consigliata per il linguaggio C.

E la lingua italiana?

VS Code può essere tradotto in italiano con l'estensione *Italian Language Pack*. In questa dispensa però i menu sono citati **in inglese**, come compaiono dopo l'installazione: se scegli l'italiano dovrai tradurre i nomi delle voci (*File* resta *File*, *Terminal* diventa *Terminale*, e così via).

Windows: collegare VS Code a Ubuntu

Su Windows l'ordine conta: prima si installa l'estensione **WSL**, poi si apre VS Code *dentro* Ubuntu, e solo allora si installa **C/C++**. Il modo più semplice per aprire VS Code dentro Ubuntu è farlo dal terminale di Ubuntu, con il comando `code` seguito da un punto (il punto significa "la cartella in cui mi trovo"):

Ubuntu

```
$ code .
Installing VS Code Server for Linux x64 (...)
Downloading: 100%
Unpacking: 100%
```

La prima volta VS Code installa il suo *server* dentro Ubuntu (lo fa da solo, in pochi secondi), poi si apre la finestra. Come si capisce che è collegata a Ubuntu? Dall'**indicatore remoto**, il riquadro blu nell'angolo in basso a sinistra (figura 1.9).



Figura 1.9: *La barra di stato di VS Code collegato a Ubuntu. Se il riquadro in basso a sinistra non dice WSL: Ubuntu, VS Code sta lavorando su Windows e non troverà il compilatore.*

Ora, nella vista *Extensions*, l'estensione C/C++ mostra il pulsante Install in WSL: Ubuntu: è quello giusto, perché l'estensione deve lavorare dove stanno i file e il compilatore. Le volte successive, VS Code ricorda le cartelle aperte: dal menu **File › Open Recent** si riapre direttamente una cartella di Ubuntu, già collegata.

Dove sono i file di Ubuntu?

La cartella personale di Ubuntu (`/home/fabio`) è visibile anche da Esplora file di Windows, alla voce *Linux* nel riquadro di sinistra. Viceversa, da Ubuntu il disco `C:` di Windows si trova in `/mnt/c`. Conviene però tenere i file del corso **dentro Ubuntu**, nella cartella personale: lì compilare è più veloce e non ci sono problemi di permessi.

Passo 4: la cartella del corso

Conviene tenere tutti i programmi del corso in una cartella dedicata. La creiamo dal terminale (su Windows, quello di **Ubuntu**) e la apriamo in VS Code:

Terminale

```
$ mkdir programmazione
$ cd programmazione
$ code .
```

`mkdir` (*make directory*) crea la cartella, `cd` (*change directory*) ci entra, `code .` apre VS Code sulla cartella corrente. In alternativa si può fare tutto con il mouse: si crea la cartella come al solito e la si apre in VS Code da **File › Open Folder...**.

La prima volta che si apre una cartella, VS Code chiede se ci si fida degli autori dei file che contiene (*Do you trust the authors of the files in this folder?*): sono i nostri file, quindi si risponde Yes, I trust the authors. Senza questa conferma VS Code apre la cartella in *modalità con restrizioni* e disattiva le estensioni.

Com'è fatta la finestra di VS Code

La figura 1.10 mostra VS Code con la cartella del corso aperta, un file C nell'editor e il terminale in basso. Vale la pena guardarla con calma: sono le zone che useremo sempre.

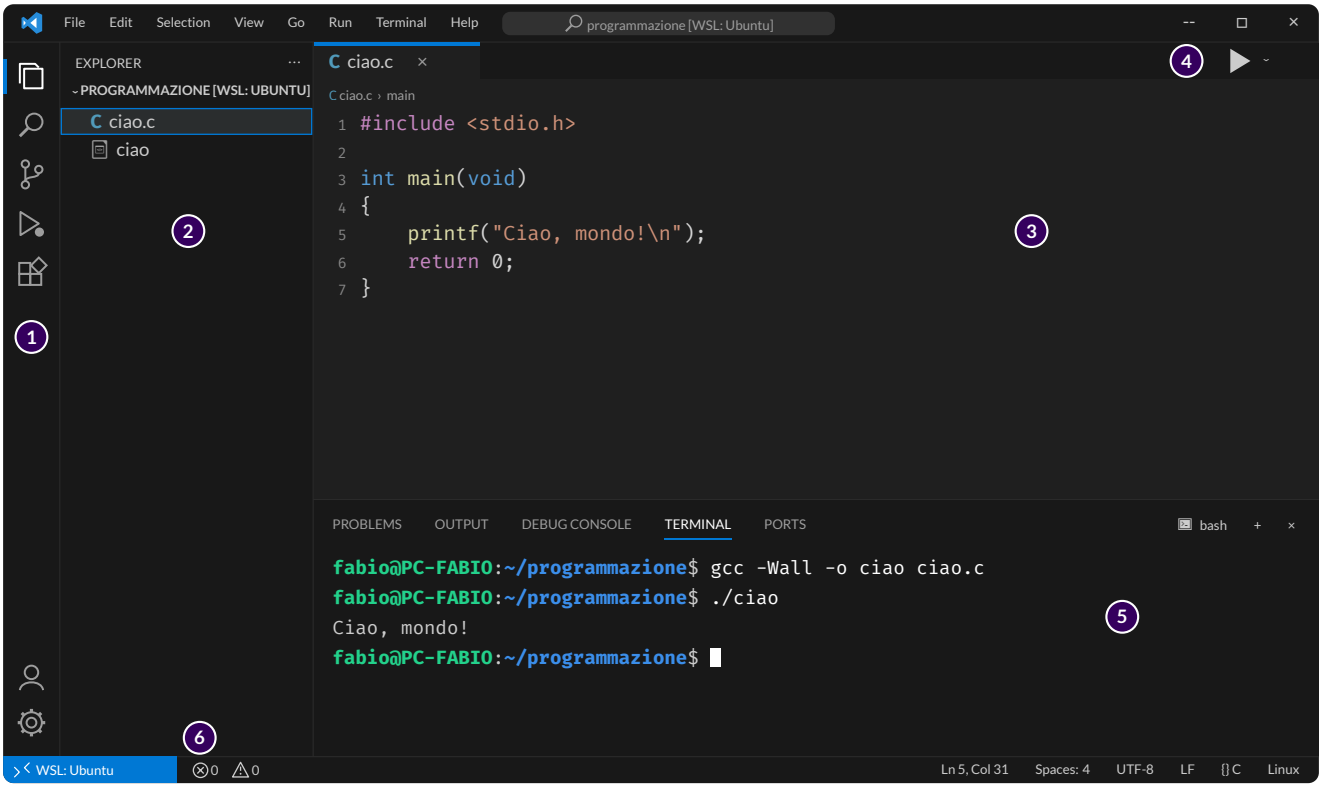


Figura 1.10: *Visual Studio Code con la cartella* `programmazione` *aperta (su Windows, collegato a Ubuntu).*

1. La **barra delle attività**: cambia ciò che compare nella barra laterale. Le icone, dall'alto: *Explorer* (i file), *Search* (cerca nei file), *Source Control* (Git), *Run and Debug* (il debugger), *Extensions*.

2. La **barra laterale**, qui con l'*Explorer*: i file della cartella aperta. Un clic apre un file nell'editor; il pulsante con il foglio e il segno "+", che compare passando con il mouse sul nome della cartella, crea un nuovo file.

3. L'**editor**: il file aperto, con i numeri di riga a sinistra. In alto ci sono le schede dei file aperti e il "percorso" (*ciao.c* › *main*: il cursore è dentro la funzione `main`).

4. Il pulsante *Run*, aggiunto dall'estensione C/C++ (ne parleremo alla fine del capitolo).

5. Il **pannello** in basso, qui con il *terminale* integrato: un vero terminale (su Windows, di Ubuntu) già posizionato nella cartella aperta. Si apre dal menu **Terminal › New Terminal**; accanto alla voce del menu è indicata anche la scorciatoia da tastiera, che dipende dalla tastiera in uso.

6. La **barra di stato**: a sinistra l'indicatore remoto e il numero di errori e avvisi; a destra la posizione del cursore (*Ln* riga, *Col* colonna) e il linguaggio riconosciuto (*C*).

Creare un file

Un nuovo file si crea dall'*Explorer* (pulsante "nuovo file" accanto al nome della cartella) oppure da **File** › **New File...**. Il nome deve finire con `.c`: è l'**estensione** del nome a dire a VS Code (e a gcc) che il file contiene codice C. Con un nome come `ciao.txt`, o senza estensione, niente colori, niente suggerimenti e gcc si rifiuterebbe di compilarlo.

Nomi di file semplici

Usa nomi brevi, in minuscolo, **senza spazi** e senza lettere accentate: `esercizio1.c`, `media_voti.c`. Gli spazi nei nomi costringono a usare le virgolette nei comandi del terminale e sono una fonte continua di errori.

IL TERMINALE

Il terminale è un dialogo scritto con il computer: noi scriviamo un **comando** e premiamo Invio, il computer lo esegue e risponde con del testo. Il programma che legge i comandi si chiama **shell**; in Ubuntu è *bash*, su macOS *zsh*. Per il corso servono pochi comandi, ma vanno capiti bene: sono il modo in cui compileremo ed eseguiremo ogni programma.

Il prompt

Quando la shell è pronta mostra il **prompt**, una riga che finisce con `$` (su Mac `%`) e dopo la quale si scrive. Il prompt di Ubuntu non è decorativo: dice *chi* siamo, *dove* siamo e che la shell *aspetta* un comando (figura 1.11).



Figura 1.11: *Le parti del prompt di Ubuntu.*

La parte più importante è la **cartella corrente** (o *directory di lavoro*): ogni comando lavora *lì*, a meno di indicare diversamente. Se scriviamo `gcc ciao.c` e il file `ciao.c` non sta nella cartella corrente, gcc non lo trova. Moltissimi problemi dei principianti si risolvono guardando il prompt.

Il prompt, nella dispensa

D'ora in poi i comandi sono mostrati con il solo `$` come prompt e in **grassetto bianco**; le righe normali sono ciò che il computer risponde, e il testo in **giallo** è ciò che si scrive mentre un programma è in esecuzione. Il `$` **non va digitato**: si scrive solo ciò che lo segue.

Cartelle e percorsi

I file sono organizzati in cartelle, che contengono altre cartelle, che contengono altri file: un **albero**, che in Linux parte da un'unica radice chiamata `/` (figura 1.12).

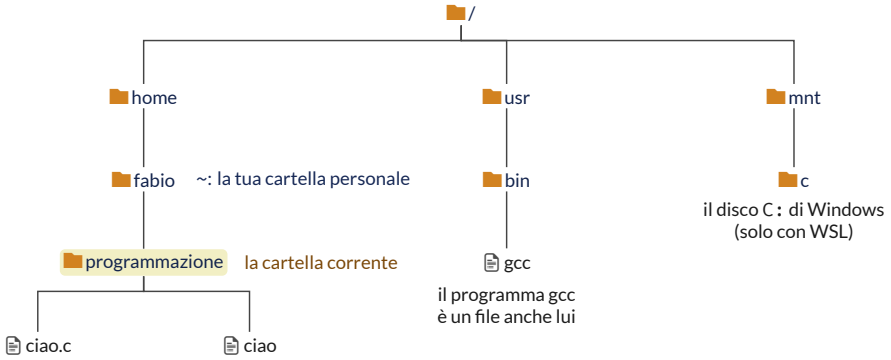


Figura 1.12: *Una parte dell'albero delle cartelle di Ubuntu.*

Un file si indica con il suo **percorso**, cioè la strada per raggiungerlo, con i nomi delle cartelle separati da `/`:

- un percorso **assoluto** parte dalla radice e vale ovunque ci si trovi: `/home/fabio/programmazione/ciao.c`;
- un percorso **relativo** parte dalla cartella corrente: se si è in `/home/fabio`, lo stesso file è `programmazione/ciao.c`; se si è già in `programmazione`, è semplicemente `ciao.c`.

Tre nomi speciali completano il quadro: `.` è la cartella corrente, `..` è la cartella che la contiene ("sali di un livello") e `~` è la cartella personale. Dalla cartella `programmazione`, quindi, `..` è `/home/fabio`.

Comando	Che cosa fa	Da dove viene il nome
`pwd`	stampa il percorso della cartella corrente	*print working directory*
`ls`	elenca il contenuto della cartella corrente	*list*
`ls -l`	come sopra, con dimensione e data dei file	*long*
`cd nome`	entra nella cartella `nome`	*change directory*
`cd ..`	sale nella cartella superiore	
`cd`	torna nella cartella personale	
`mkdir nome`	crea la cartella `nome`	*make directory*
`cp da a`	copia il file `da` nel file `a`	*copy*
`mv da a`	sposta o rinomina un file	*move*
`rm file`	cancella un file (**senza cestino!**)	*remove*
`cat file`	mostra il contenuto di un file di testo	*concatenate*
`clear`	pulisce lo schermo del terminale	

Ecco una breve sessione di lavoro; la spiegazione segue subito dopo.

Terminale

```
$ pwd
/home/fabio
$ ls
programmazione
$ cd programmazione
$ ls
ciao.c
$ mkdir lezione2
$ ls
ciao.c  lezione2
$ cp ciao.c lezione2/prova.c
$ cd lezione2
$ ls
prova.c
$ cd ..
$ pwd
/home/fabio/programmazione
```

Si parte dalla cartella personale (`pwd` lo conferma), si entra in `programmazione`, si crea la sottocartella `lezione2` e ci si copia il file `ciao.c` con il nuovo nome `prova.c` (si noti il percorso relativo `lezione2/prova.c`). Poi si entra in `lezione2`, si controlla che la copia ci sia, e con `cd ..` si torna su di un livello. Molti comandi, quando riescono, **non stampano niente**: è la convenzione di Linux, "nessuna notizia, buona notizia".

`rm` non perdona

Il comando `rm` cancella definitivamente: il file non passa per il cestino e non si recupera. Controlla sempre due volte il nome prima di premere Invio.

Trucchi che fanno risparmiare tempo

- Tab **completa** i nomi: scrivendo `cd prog` e premendo Tab, la shell completa da sola in `cd programmazione/`. Oltre a risparmiare tempo evita gli errori di battitura: se il completamento non funziona, il file non si trova dove pensiamo.
- ↑ e ↓ scorrono i **comandi già dati**: per ricompilare dopo una modifica basta premere ↑ fino a ritrovare il comando `gcc` e poi Invio.
- Ctrl + C **interrompe** il programma in esecuzione. Se un programma non termina mai (capiterà, con i cicli infiniti del capitolo **??**), è il modo per fermarlo.
- Ctrl + D, scritto come input a un programma, segnala la **fine dell'input**; lo useremo dal capitolo 5.
- Nel terminale Ctrl + C non copia: per copiare e incollare, nel terminale di VS Code si usano il tasto destro del mouse o Ctrl + Shift + C e Ctrl + Shift + V.

Il primo programma

È il momento di mettere insieme i pezzi. Nella cartella `programmazione` crea il file `ciao.c` e scrivici, esattamente così, il programma seguente.

```c
#include <stdio.h>

int main(void)
{
    printf("Ciao, mondo!\n");
    return 0;
}
```

Il significato di ogni riga sarà spiegato nel capitolo 5; per ora basta sapere che `printf` stampa il testo fra virgolette, e che `\n` significa "vai a capo". Prima di proseguire, **salva** il file con Ctrl + S (su Mac Cmd + S).



Figura 1.13: *La scheda del file dice se ci sono modifiche non salvate.*

gcc legge il file **dal disco**: se le modifiche non sono salvate, compila la versione vecchia. È l'errore più frequente all'inizio ("ho corretto ma non cambia niente!"). Per evitarlo una volta per tutte, si può attivare il salvataggio automatico dal menu **File › Auto Save**.

Compilare ed eseguire

Ora, nel terminale, compiliamo ed eseguiamo:

```
Terminale

$ gcc ciao.c
$ ls
a.out  ciao.c
$ ./a.out
Ciao, mondo!
```

- `gcc ciao.c` compila il file. Non stampa nulla: nessun messaggio significa **nessun errore**. Il risultato è un nuovo file, l'eseguibile, che per tradizione si chiama `a.out`.
- `./a.out` esegue il programma, che stampa la sua riga.

Il nome `a.out` va bene per una prova, ma con più programmi nella stessa cartella ogni compilazione sovrascriverebbe la precedente. Con l'opzione `-o` (*output*) si sceglie il nome dell'eseguibile; e con `-Wall` (*warnings: all*) si chiede a gcc di segnalare anche le cose sospette, che non impediscono la compilazione ma spesso nascondono un errore. È il modo in cui conviene compilare sempre:

```
Terminale

$ gcc -Wall -o ciao ciao.c
$ ./ciao
Ciao, mondo!
```

Ogni volta che si cambia il programma bisogna ripetere **tutto il giro** (figura 1.14): l'eseguibile è una traduzione fatta in un certo momento, e non si aggiorna da solo quando cambia il testo.

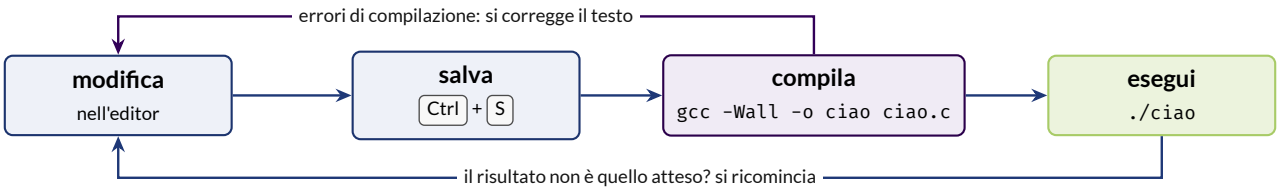


Figura 1.14: *Il ciclo di lavoro: modifica, salva, compila, esegui.*

Perché `./`?

Perché per eseguire il programma si scrive `./ciao` e non solo `ciao`, mentre per gcc basta `gcc`? Quando si scrive il nome di un comando, la shell non lo cerca dappertutto: lo cerca solo in un elenco di cartelle prestabilite, quelle dove stanno i programmi installati (come `/usr/bin`, dove si trova `gcc`). L'elenco è una *variabile d'ambiente* chiamata `PATH`, e per sicurezza non contiene la cartella corrente. Scrivendo `./ciao` si dà il **percorso** del file invece del nome di un comando: "il programma `ciao` che sta *qui*" (`.` è la cartella corrente).

Terminale

```
$ ciao
ciao: command not found
$ ./ciao
Ciao, mondo!
```

Gli errori di compilazione

Che cosa succede se il programma contiene un errore? Togliamo il punto e virgola alla fine della riga di `printf` e ricompiliamo:

errore.c

```c
#include <stdio.h>

int main(void)
{
    printf("Ciao, mondo!\n")
    return 0;
}
```

Terminale

```
$ gcc -Wall -o errore errore.c
errore.c: In function ‘main’:
errore.c:5:29: error: expected ‘;’ before ‘return’
    5 |     printf("Ciao, mondo!\n")
      |                             ^
      |                             ;
    6 |     return 0;
      |     ~~~~~~
```

Il messaggio va letto con calma, perché dice quasi tutto (figura 1.15): **dove** è l'errore (file, riga e colonna), **di che tipo** è e **che cosa** si aspettava il compilatore. Le righe successive ricopiano il codice e indicano con `^` il punto esatto; gcc arriva perfino a suggerire il `;` mancante. Quando c'è un `error` l'eseguibile **non viene prodotto**: bisogna correggere e ricompilare.



Figura 1.15: *Anatomia di un messaggio di errore di gcc.*

Si noti che l'errore è segnalato alla riga **5**, alla fine della riga di `printf`, anche se il compilatore se ne accorge solo leggendo `return` alla riga 6: il punto e virgola poteva mancare solo lì. In generale l'errore sta nella riga indicata **o poco prima**.

Esistono due tipi di messaggi:

- un `error` è un errore vero: il testo non è C valido e l'eseguibile non viene creato;
- un `warning` (avviso) segnala qualcosa di lecito ma sospetto: l'eseguibile viene creato, ma quasi sempre il warning indica un errore di ragionamento. **I warning vanno trattati come errori**: un programma compilato con `-Wall` non deve produrne nessuno.

Leggere solo il primo errore

Un solo errore può confondere il compilatore e generarne a cascata molti altri, alcuni dei quali senza senso. La regola pratica: si corregge **il primo** messaggio dell'elenco, si ricompila, e si ripete.

Con l'estensione C/C++ installata, VS Code si accorge di molti errori **prima** della compilazione e li sottolinea con un'ondina rossa; passando con il mouse sopra la sottolineatura compare la spiegazione (figura 1.16). Il numero di errori compare anche nella barra di stato, accanto a ⊗.



Figura 1.16: *L'estensione C/C++ sottolinea l'errore mentre si scrive.*

E il pulsante "Run"?

L'estensione C/C++ aggiunge in alto a destra nell'editor un pulsante ▶ (*Run C/C++ File*) che compila ed esegue il file aperto con un clic; la prima volta chiede quale compilatore usare (si sceglie *gcc*). È comodo, ma in questa dispensa compileremo **sempre dal terminale**: è ciò che si fa a lezione e all'esame, permette di scegliere le opzioni di gcc, e soprattutto fa vedere che cosa succede davvero, passo per passo. Dal capitolo 7 useremo `make`, che si lancia anch'esso dal terminale.

Quando qualcosa non funziona

Sintomo	Causa probabile e rimedio
`gcc: command not found`	Il compilatore non è installato (ripeti il passo 2). Su Windows, controlla di essere nel terminale di **Ubuntu** (prompt con `$`) e non in PowerShell (prompt `PS C:\...`).
`code: command not found`	Su Windows chiudi e riapri il terminale di Ubuntu dopo aver installato VS Code (il `PATH` si aggiorna solo all'apertura); su Mac esegui *Shell Command: Install 'code' command in PATH*.
`ciao.c: No such file or directory`	Il file non è nella cartella corrente: controlla con `pwd` e `ls` dove ti trovi e che cosa c'è, oppure il file non è stato salvato con quel nome.
`./ciao: No such file or directory`	Il programma non è stato compilato, oppure la compilazione è fallita (c'erano degli `error`), oppure l'eseguibile ha un altro nome (`a.out`?).
Ho corretto ma il programma fa ancora la cosa sbagliata	Hai salvato? Hai ricompilato? Stai eseguendo l'eseguibile giusto?
Il programma non termina	Probabilmente un ciclo infinito, oppure il programma aspetta un input da tastiera. Ctrl + C lo interrompe.
In VS Code `#include <stdio.h>` è sottolineato in rosso	L'estensione C/C++ non trova il compilatore. Su Windows: VS Code non è collegato a Ubuntu (guarda l'indicatore remoto) o l'estensione non è installata *in WSL*. Altrove: verifica che `gcc --version` funzioni.

In sintesi

Per scrivere ed eseguire un programma C servono tre strumenti: un **editor** (VS Code) per scrivere il testo del programma in un file `.c`, un **compilatore** (gcc) che lo traduce in linguaggio macchina, e un **terminale** da cui lanciare il compilatore e il programma. Su Windows tutto ciò vive in Ubuntu, grazie a WSL. Il ciclo di lavoro è sempre lo stesso: *modifica, salva, compila, esegui*; e i messaggi di gcc indicano file, riga e colonna del problema.

Riferimenti

- **Installare Linux in Windows con WSL** — Microsoft Learn.
  `https://learn.microsoft.com/it-it/windows/wsl/install`
- **Using C++ and WSL in VS Code** — documentazione di VS Code (vale anche per il C).
  `https://code.visualstudio.com/docs/cpp/config-wsl`
- **Using Clang in Visual Studio Code** (macOS) e **Using C++ on Linux in VS Code** — documentazione di VS Code.
  `https://code.visualstudio.com/docs/cpp/config-clang-mac`
  `https://code.visualstudio.com/docs/cpp/config-linux`
- **User Interface** — la guida alle zone della finestra di VS Code.
  `https://code.visualstudio.com/docs/getstarted/userinterface`

Il calcolatore

Per imparare a guidare non serve saper costruire un motore, ma aiuta molto sapere che cosa fanno il volante, i pedali e il cambio. Allo stesso modo, per programmare non serve conoscere l'elettronica di un computer, ma bisogna avere un'idea chiara di come è fatto e di come esegue un programma: molte regole del C, che altrimenti sembrerebbero capricci, diventano ovvie. Questo capitolo costruisce quell'idea, partendo da che cosa significa *programmare*.

Che cos'è la programmazione

Il corso si apre con la definizione del dizionario Treccani:

> *"In informatica, la stesura di un programma in una forma interpretabile dal calcolatore elettronico (o direttamente o dopo una rielaborazione opportuna), compiuta dal programmatore sulla base delle specifiche di programma fornite dall'analista; la programmazione comprende la definizione del problema da risolvere, la preparazione del diagramma di flusso del programma, la sua codifica (cioè la trascrizione, in un linguaggio opportuno, delle varie istruzioni), la messa a punto e la preparazione di una documentazione a uso dell'operatore."*

La definizione contiene già tutto il percorso del corso. Scrivere codice (la *codifica*) è solo una delle fasi, e non la prima: prima bisogna capire il problema e progettare la soluzione, dopo bisogna verificare che funzioni (la *messa a punto*) e spiegarla (figura 2.1). La parentesi "o direttamente o dopo una rielaborazione opportuna" è importante: il programma che scriviamo non è ancora ciò che il computer esegue, va *rielaborato*, cioè tradotto. È il compito del compilatore incontrato nel capitolo precedente.

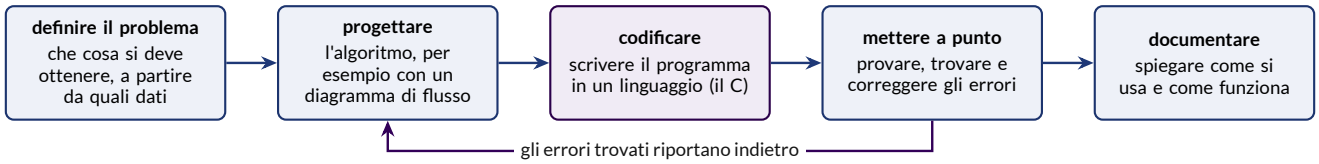


Figura 2.1: *Le fasi della programmazione secondo la definizione: scrivere il codice è solo una di esse.*

Come si impara a programmare

A lezione il docente avverte che il corso parte da zero e all'inizio va piano, ma che velocità e difficoltà crescono presto, e che entro il primo mese quasi tutti incontrano difficoltà serie, soprattutto negli esercizi. È normale e si supera, con alcune abitudini:

- non dare per scontato di saper già programmare, e non saltare le lezioni "facili";
- studiare con costanza: la programmazione si accumula, ogni argomento usa i precedenti;
- **scrivere ed eseguire** ogni esempio, poi modificarlo per vedere che cosa succede: leggere un programma non basta, come non basta guardare qualcuno nuotare;
- chiedere appena qualcosa non è chiaro ("non siete gli unici");
- prendere appunti e fare sintesi: le slide sono, per loro natura, sintetiche; questa dispensa prova a riempire gli spazi, ma non sostituisce le lezioni.

Lo schema di un calcolatore

Un computer moderno è un oggetto complicatissimo, ma il suo funzionamento si capisce con uno schema di tre soli blocchi (figura 2.2):

- la **CPU** (*Central Processing Unit*, il *processore*), che esegue le istruzioni;
- la **memoria centrale** (o *principale*, la RAM), che contiene le istruzioni da eseguire e i dati su cui lavorano;
- i **dispositivi di ingresso e uscita** (*I/O*, *Input/Output*): tastiera, mouse, schermo, disco, rete…, collegati attraverso delle *interfacce*.

I blocchi comunicano attraverso il **bus**, un insieme di collegamenti condiviso su cui viaggiano dati, indirizzi e comandi. Fisicamente, è la *scheda madre* (*motherboard*) a collegare fra loro processore, memoria e periferiche.

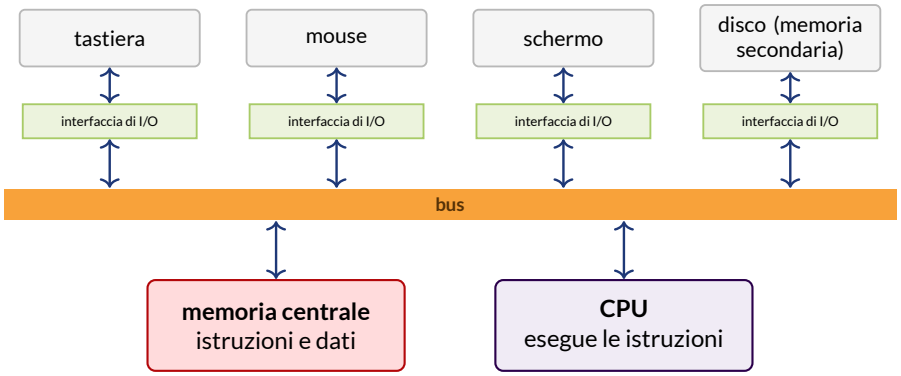


Figura 2.2: *Schema semplificato di un calcolatore (architettura di von Neumann): CPU, memoria e dispositivi di I/O collegati dal bus.*

Questo schema porta il nome di *John von Neumann*, che nel 1945 ne descrisse l'idea centrale: le **istruzioni** del programma stanno nella **stessa memoria** dei **dati**. Un programma non è cablato nei circuiti: è una sequenza di numeri in memoria, che si può caricare, sostituire, copiare. È questo che rende un computer una macchina *generale*, capace di fare cose diversissime a seconda del programma caricato.

La CPU e il linguaggio macchina

Che cosa sa fare un processore

Tutte le elaborazioni di un calcolatore sono svolte dalla CPU, oppure da altri componenti su suo comando. Sorprendentemente, un processore sa compiere solo operazioni **molto semplici**:

- leggere, scrivere o copiare una o più celle di memoria;
- sommare, sottrarre, moltiplicare, dividere il contenuto di celle di memoria;
- leggere e scrivere in zone di memoria "speciali" per pilotare i dispositivi di I/O (per esempio la scheda video);
- poche altre operazioni elementari, come confrontare due numeri e **saltare** a un'altra istruzione a seconda del risultato.

Il segreto è la velocità: un processore moderno esegue **miliardi** di queste operazioni al secondo. Ogni cosa che un computer fa, dal mostrare una pagina web al riprodurre un video, è scomposta in una lunghissima sequenza di passi elementari.

Ogni processore ha un proprio insieme di istruzioni, e ogni istruzione è identificata da una certa configurazione di bit (di zeri e uni). L'insieme delle istruzioni di un processore si chiama **linguaggio macchina** di quel processore. Processori di famiglie diverse hanno linguaggi macchina diversi: un programma tradotto per i processori Intel e AMD dei PC (famiglia *x86-64*) non gira su un Mac con processore Apple (famiglia *ARM*), e viceversa.

Per eseguire un programma, allora, bisogna fare due cose: mettere in memoria la sequenza di configurazioni di bit che rappresentano le sue istruzioni, e dire al processore a quale indirizzo si trova la prima. Da lì in poi il processore ripete sempre lo stesso ciclo (figura 2.3): **preleva** l'istruzione all'indirizzo indicato da un suo registro speciale, il **program counter**; **decodifica** l'istruzione, cioè capisce che cosa chiede; la **esegue**; e passa all'istruzione successiva, a meno che l'istruzione appena eseguita non fosse un *salto*, che cambia il program counter.

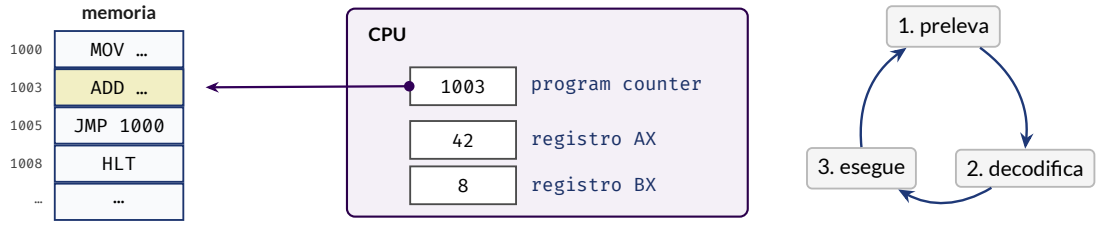


Figura 2.3: *Il program counter indica l'indirizzo della prossima istruzione; la CPU la preleva, la decodifica e la esegue, poi ricomincia. I registri sono piccole celle di memoria interne alla CPU.*

L'ASSEMBLY

Il linguaggio macchina è una sequenza di bit, praticamente illeggibile per una persona. Il primo passo per renderlo comprensibile è l'**assembly**: un linguaggio quasi identico al linguaggio macchina, in cui però ogni codice operativo è sostituito da un'abbreviazione (un *codice mnemonico*): `MOV` per *move* (copia), `ADD` per *add* (somma), `JMP` per *jump* (salta). Un programma, l'**assembler**, traduce l'assembly in linguaggio macchina, un'istruzione alla volta.

Un esempio: contare i bit a 1

Il programma seguente, visto a lezione, conta quanti bit valgono 1 in un dato di 16 bit. A sinistra di ogni istruzione c'è il suo indirizzo in memoria (in esadecimale, cioè in base 16: per ora basta considerarlo un "numero civico"), a destra la spiegazione.

indirizzo	istruzione		significato
`3F0D0100`	`00000000 00000000`		il dato da elaborare (16 bit)
`3F0D0102`	`00000000`		qui andrà il risultato (8 bit)
`40B1A200`	`MOV`	`$0, %AL`	azzera il contatore, il registro AL
`40B1A203`	`MOV`	`(3F0D0100), %DX`	copia nel registro DX il dato da elaborare
`40B1A207`	`CMP`	`%DX, $0`	confronta DX con 0...
`40B1A211`	`JE`	`40B1A232`	...e se sono uguali salta alla fine
`40B1A215`	`SHL`	`%DX`	sposta i bit di DX di un posto a sinistra: il bit che "esce" finisce nel *carry flag* CF
`40B1A217`	`JC`	`40B1A225`	se CF vale 1 (è uscito un 1) salta all'incremento
`40B1A221`	`JMP`	`40B1A207`	altrimenti torna al confronto
`40B1A225`	`ADD`	`$1, %AL`	incrementa il contatore AL
`40B1A228`	`JMP`	`40B1A207`	torna al confronto
`40B1A232`	`MOV`	`%AL, (3F0D0102)`	scrive il risultato in memoria
`40B1A236`	`HLT`		fine (*halt*)

L'idea è questa: a ogni giro si fa "scivolare" il dato di un posto a sinistra, e si guarda il bit che cade fuori dal bordo; se è un 1, si incrementa il contatore. Quando nel dato non resta nessun 1 (cioè vale 0), si è finito. Seguiamo l'esecuzione con un dato che contiene due bit a 1, `1010000000000000`:

giro	DX prima	bit uscito (CF)	DX dopo	AL	che cosa succede
1	`1010000000000000`	`1`	`0100000000000000`	`1`	è uscito un 1: `JC` salta all'`ADD`
2	`0100000000000000`	`0`	`1000000000000000`	`1`	è uscito uno 0: `JMP` torna al confronto
3	`1000000000000000`	`1`	`0000000000000000`	`2`	è uscito un 1: si incrementa
4	`0000000000000000`			`2`	DX vale 0: `JE` salta alla fine

Il risultato, 2, viene scritto in memoria e il programma si ferma. Si noti che in assembly non esiste un'istruzione "ripeti finché": la ripetizione si ottiene **saltando indietro** (`JMP 40B1A207`) e la scelta **saltando avanti** se una condizione è vera (`JE`, `JC`).

I limiti dell'assembly

Con l'assembly si può scrivere qualunque programma, anche un sistema operativo o un'interfaccia grafica, ma è un lavoro enorme: le istruzioni sono di **basso livello**, cioè vicine alla macchina e lontane dal modo in cui ragioniamo. Per "contare gli uni" abbiamo dovuto pensare in termini di registri, flag e indirizzi di salto; e il programma funziona solo sul processore per cui è scritto.

I LIVELLI DI ASTRAZIONE

Servono istruzioni più vicine al modo di ragionare umano. La soluzione è l'**astrazione**: costruire un linguaggio che "nasconde" la macchina sottostante e permette di esprimere gli algoritmi in modo simbolico. Sono i **linguaggi di alto livello**, come il C, che hanno due proprietà fondamentali:

- sono più vicini al linguaggio umano (e matematico) che al linguaggio macchina;
- sono **generali**: non dipendono dal processore. Lo stesso programma C si traduce per un PC, per un Mac o per un telefono.

La figura 2.4 mostra la stessa istruzione ai tre livelli. È la traduzione reale prodotta da gcc su un PC (processore x86-64) per l'istruzione C `c = a + b;`, in cui `a`, `b` e `c` sono tre variabili intere.

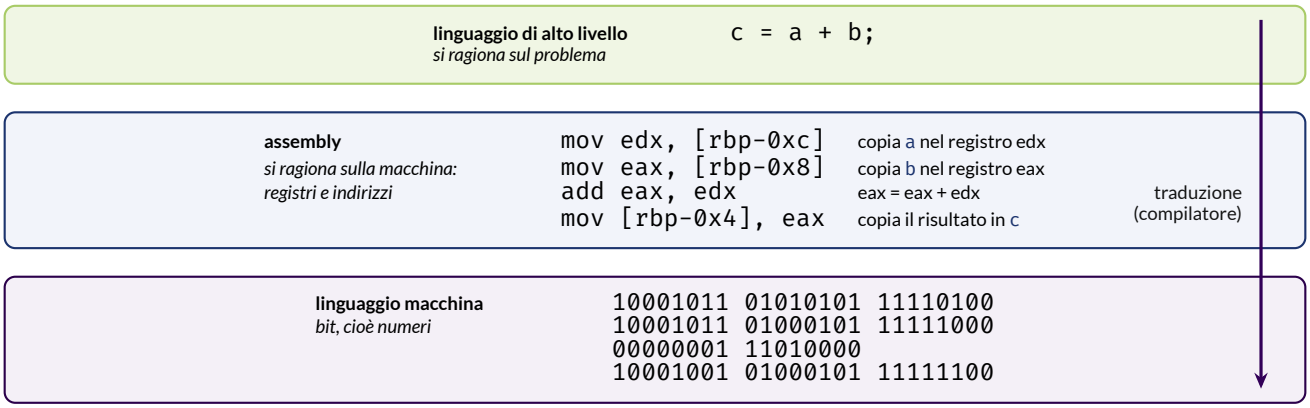


Figura 2.4: *La stessa istruzione a tre livelli di astrazione. Un'istruzione di alto livello diventa più istruzioni macchina; ogni istruzione assembly corrisponde a un'istruzione macchina.*

Ogni livello richiede di conoscere cose diverse:

- per il **linguaggio macchina**, come sono rappresentate le informazioni in bit;
- per l'**assembly**, le caratteristiche della macchina: registri, insieme di istruzioni, dimensione dei dati;
- per un **linguaggio di alto livello**, solo il linguaggio: ci si astrae dai dettagli della macchina.

Il prezzo da pagare è che un programma di alto livello non può essere eseguito direttamente: va **tradotto** in linguaggio macchina. È il compito del **compilatore**, l'argomento del capitolo 3.

Algoritmo e programma

Due parole che useremo continuamente vanno distinte con cura.

Algoritmo

Una sequenza di passi elementari che risolve un problema. La sequenza deve essere **finita**, **non ambigua**, **deterministica** e **generale**.

Programma

La formalizzazione di un algoritmo in un linguaggio di programmazione.

Le quattro proprietà dell'algoritmo non sono pignolerie: ciascuna esclude un modo concreto in cui un procedimento può fallire. Prendiamo una ricetta di cucina, che è quasi un algoritmo:

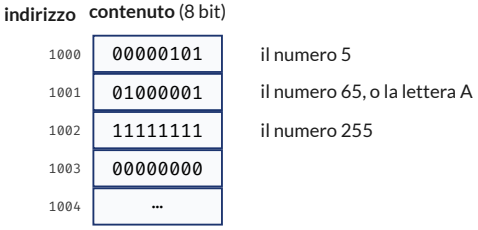
- **finita**: deve terminare. "Mescola finché non è pronto" è accettabile solo se prima o poi "è pronto";
- **non ambigua**: ogni passo deve avere un solo significato. "Sale quanto basta" va bene per un cuoco, non per un computer, che non ha buon senso: bisogna dire quanti grammi;
- **deterministica**: in ogni momento il passo successivo è determinato; con gli stessi dati si ottiene sempre lo stesso risultato;
- **generale**: risolve una *classe* di problemi, non un solo caso. Una ricetta "per 4 persone" è meno generale di una "per n persone"; un algoritmo che calcola l'area di un cerchio di raggio 2 è inutile, ne serve uno che funzioni per qualunque raggio.

La memoria

Celle e indirizzi

La memoria centrale è il contenitore dei dati su cui lavora il processore. Si può immaginare come una lunghissima fila di caselle consecutive, le **locazioni** (o *celle*) di memoria. Ogni cella:

- è l'unità minima di memorizzazione: si legge e si scrive una cella intera, non un pezzo;
- contiene tipicamente **8 bit**, cioè un **byte**;
- è identificata da un numero, il suo **indirizzo**, come le case di una strada dal numero civico.



Un **bit** (*binary digit*, cifra binaria) è la più piccola unità di informazione: vale 0 oppure 1. Elettronicamente corrisponde a qualcosa che ha due stati, come un interruttore acceso o spento. Con un bit solo si distinguono due situazioni; mettendone insieme 8 se ne distinguono molte di più: vediamo quante, e come si usano per rappresentare i numeri.

La notazione binaria

Siamo abituati alla notazione **decimale**: dieci cifre, da 0 a 9, e un sistema **posizionale**, in cui il valore di una cifra dipende dalla posizione. In `352` il 3 vale trecento perché sta nella posizione delle centinaia:

$$352 = 3 \cdot 100 + 5 \cdot 10 + 2 \cdot 1 = 3 \cdot 10^2 + 5 \cdot 10^1 + 2 \cdot 10^0.$$

La notazione **binaria** funziona allo stesso modo, ma con due sole cifre, 0 e 1, e con le potenze di **2** al posto delle potenze di 10 (figura 2.5).

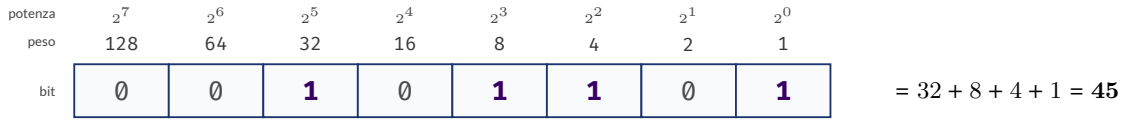


Figura 2.5: *Il byte `00101101`: ogni bit a 1 aggiunge il peso della sua posizione.*

Per passare dal binario al decimale, quindi, si sommano i pesi delle posizioni in cui c'è un 1. Il passaggio inverso si fa con le **divisioni successive per 2**: si divide il numero per 2, si annota il resto (0 o 1), si ripete sul quoziente fino ad arrivare a 0; i resti, letti **dal basso verso l'alto**, sono le cifre binarie.

```
13 : 2 = 6   resto 1
 6 : 2 = 3   resto 0
 3 : 2 = 1   resto 1
 1 : 2 = 0   resto 1
```



Per esempio, 13 in binario è `1101`: lo si verifica sommando i pesi, 8 + 4 + 1 = 13. Perché funziona? Ogni divisione per 2 "stacca" l'ultima cifra binaria del numero, esattamente come dividere per 10 un numero decimale ne stacca l'ultima cifra (il resto di 352 : 10 è 2, l'ultima cifra). Per questo il primo resto trovato è la cifra più a destra, e i resti vanno letti al contrario.

Quanti numeri in un byte

Con n posizioni e 2 simboli possibili per ciascuna si formano 2^n combinazioni diverse: ogni bit in più **raddoppia** le possibilità. Un byte ne ha $2^8 = 256$, quindi rappresenta i numeri naturali da 0 a 255:

bit	valore	bit	valore
`00000000`	0	`00000100`	4
`00000001`	1	`00000101`	5
`00000010`	2	…	…
`00000011`	3	`11111111`	255

I numeri negativi

E i numeri negativi? Bisogna usare una parte dell'informazione per il **segno**. L'idea più semplice, vista a lezione, è la **rappresentazione in modulo e segno**: il primo bit indica il segno (0 positivo, 1 negativo) e gli altri 7 il valore assoluto. Con 7 bit si rappresentano $2^7 = 128$ valori assoluti, da 0 a 127, quindi l'intervallo è $[-127,\ 127]$.

Una svista nelle slide

Le slide indicano l'intervallo $[-127..128]$, ma in modulo e segno il massimo è +127 (`01111111`), non 128: con 7 bit il valore assoluto più grande è $2^7 - 1 = 127$. L'intervallo è simmetrico, $[-127,\ 127]$, e lo zero compare due volte: `00000000` (+0) e `10000000` (−0).

Proprio quel "doppio zero", e il fatto che le somme con numeri negativi richiederebbero circuiti speciali, fanno sì che i computer reali usino un'altra rappresentazione, il **complemento a due**, in cui con un byte si rappresentano i numeri da −128 a +127. L'idea si vede bene con 3 bit (figura 2.6): i numeri sono disposti su una ruota, come le cifre del contachilometri di un'auto; partendo da `000` e andando **indietro** di un passo si arriva a `111`, che quindi vale −1.

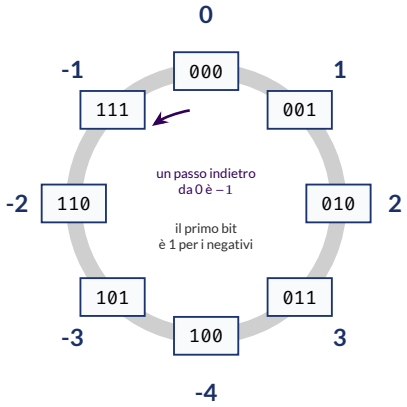


Figura 2.6: *Il complemento a due su 3 bit: 8 combinazioni, da −4 a +3, un solo zero. Su 8 bit si va da −128 a +127.*

Il complemento a due ha conseguenze molto concrete in C (per esempio su che cosa succede quando un numero "trabocca" oltre il massimo): le vedremo nel capitolo **??**.

I TIPI DI DATO

Gli stessi bit, significati diversi

Il byte `01000001` contiene semplicemente otto bit. È il numero 65? È la lettera *A*? Dipende da come lo si **interpreta**: la stessa quantità di memoria può rappresentare un numero naturale, un intero con segno, un numero reale o un carattere. La memoria non sa che cosa contiene; lo sa il programma.

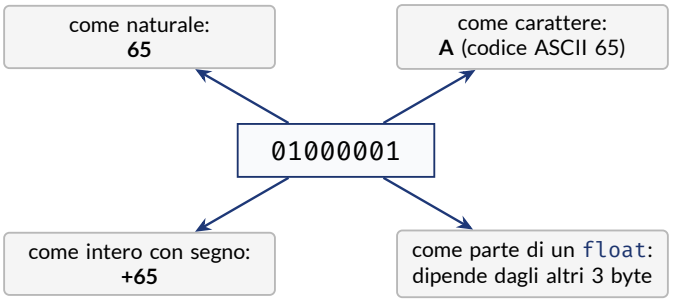


Figura 2.7: *Lo stesso byte, letto con occhiali diversi.*

Il tipo come astrazione

I linguaggi di alto livello risolvono il problema con i **tipi di dato**: si scrive il programma in termini di *che cosa* sono i dati, senza preoccuparsi di come e dove saranno davvero memorizzati. In C si può scrivere

```
1 int variabile = 5;
```

e non ci si chiede in quali celle finirà il 5, né come sarà scritto in bit: si usa `int` nel suo significato astratto di "contenitore di numeri interi". Eppure un `int` non sta in una cella sola: ne occupa **quattro consecutive** (32 bit, quindi valori ben oltre 255). È il tipo a stabilire che quelle quattro celle vanno lette insieme, come un unico numero (figura 2.8).

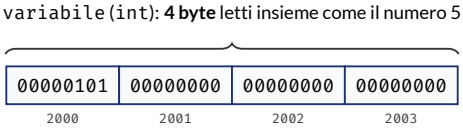


Figura 2.8: *Un `int` occupa 4 celle consecutive (sui PC il byte meno significativo sta all'indirizzo più basso).*

Ogni linguaggio definisce i propri tipi. In C, i principali sono:

tipo	che cosa rappresenta	dimensione tipica
`char`	un carattere (codice ASCII)	8 bit (1 byte)
`int`	un numero intero	32 bit (4 byte)
`float`	un numero reale	32 bit (4 byte)
`double`	un numero reale, più preciso	64 bit (8 byte)

Le dimensioni sono quelle dei computer di oggi; lo standard del C fissa solo dei minimi, come vedremo nel capitolo **??**.

LA GERARCHIA DI MEMORIA

La memoria centrale può contenere molti programmi e dati, ma ha due difetti:

- è **lenta** rispetto al processore: leggere una cella richiede il tempo di un centinaio di operazioni della CPU;
- è **volatile**: quando si spegne il computer, il contenuto si perde.

Contro la lentezza si costruisce una **gerarchia**: piccole memorie velocissime vicino al processore, memorie via via più grandi e lente man mano che ci si allontana. Le memorie veloci costano molto per ogni bit, quindi sono piccole. Contro la volatilità ci sono le **memorie di massa** (dischi, SSD, chiavette), che conservano i dati anche a computer spento (figura 2.9).

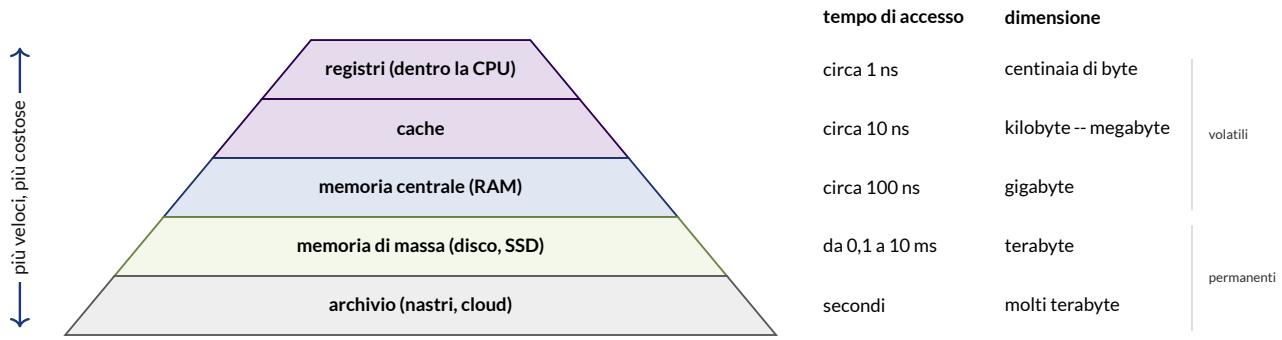


Figura 2.9: *La gerarchia di memoria (1 ns è un miliardesimo di secondo). In alto memorie piccole, veloci, costose e volatili; in basso grandi, lente, economiche e permanenti.*

Un'immagine aiuta: la CPU è un impiegato alla scrivania. I registri sono ciò che tiene in mano; la cache, i fogli sulla scrivania; la RAM, lo scaffale nella stanza; il disco, l'archivio in cantina. L'impiegato lavora solo su ciò che ha in mano, e se un foglio non è sulla scrivania deve alzarsi a prenderlo: per questo i dati usati spesso vengono tenuti vicini.

Gli strati del software

Fra l'hardware e i programmi che usiamo c'è un sistema software a strati (figura 2.10). Ogni strato usa i servizi di quello sotto e ne offre di più comodi a quello sopra: è ancora l'idea di astrazione.



Figura 2.10: *Fra l'hardware e l'ambiente in cui si sviluppano i programmi c'è un sistema software stratificato.*

L'avvio del computer

Il primo programma eseguito all'accensione è il **sistema operativo**: è lui che, quando chiediamo di eseguire un programma, lo carica in memoria e dice al processore dove si trova la sua prima istruzione. Ma chi dice al processore dove si trova la prima istruzione del sistema operativo? È un problema dell'uovo e della gallina, risolto così (figura 2.11): all'accensione il computer esegue un piccolo programma che sta in una memoria permanente della scheda madre (il *firmware*); questo carica in memoria il contenuto di un blocco predefinito di un disco, il **Master Boot Record** (MBR). Il codice del MBR, chiamato **bootloader**, è una sequenza di byte che si assume essere un programma in linguaggio macchina: il suo compito è caricare il sistema operativo. Il MBR è configurabile: se sul disco ci sono più sistemi operativi, si può scegliere quale avviare.



Figura 2.11: *L'avvio: ogni programma carica il successivo, più grande e più capace.*

MBR e UEFI

Il Master Boot Record è il meccanismo storico dei PC. I computer recenti usano il firmware *UEFI*, che invece di un singolo blocco legge il bootloader da una piccola partizione dedicata del disco. L'idea non cambia: un programma minuscolo, che sta in un posto noto in anticipo, avvia un programma più grande.

Che cosa fa il sistema operativo

Il sistema operativo **alza il livello di astrazione** della macchina: è più facile da programmare dell'hardware sottostante e semplifica la scrittura di ogni altro programma. Le sue funzioni principali:

- **gestione dei processi**, cioè dei programmi in esecuzione: li crea e li termina, li sospende e li riattiva, li fa comunicare. Il processore esegue un'istruzione alla volta, ma il sistema operativo lo assegna a turno ai vari programmi per pochi millesimi di secondo ciascuno (la *schedulazione*): così sembrano procedere tutti insieme;
- **gestione della memoria centrale**: assegna la memoria ai processi, li carica e li rimuove, e la **protegge**, cioè impedisce a un programma di scrivere nella memoria di un altro;
- **gestione delle periferiche**: le configura e le pilota, con il *buffering* (tiene i dati in memoria finché non vengono elaborati) e il *caching* (tiene vicini i dati usati spesso);
- **gestione del file system**: nasconde all'utente come sono organizzati i dati sul disco, offrendo il concetto di *file* e di *cartella* (l'albero di cartelle del capitolo 1), con le operazioni di creazione, lettura, scrittura, copia, ricerca e i diritti di accesso;
- **interfaccia utente**: a riga di comando (l'interprete dei comandi, la shell che usiamo nel terminale) o grafica;
- **rilevamento degli errori**: divisione per zero, carta finita, rete caduta…; per ogni errore serve un'azione di recupero appropriata.

L'ultima voce ci riguarderà da vicino. Un programma C che divide un intero per zero chiede al processore un'operazione impossibile; il processore avvisa il sistema operativo, che **termina il programma**:

div0.c

```c
#include <stdio.h>

int main(void)
{
    int a = 7, b = 0;
    printf("%d\n", a / b);
    return 0;
}
```

Terminale

```
$ gcc -o div0 div0.c
$ ./div0
Floating point exception
```

Il messaggio parla di "virgola mobile" anche se la divisione è fra interi: è un nome storico.

I servizi per i programmi

Il sistema operativo offre servizi ai programmi: caricarli in memoria ed eseguirli, compiere le operazioni di I/O (un programma normale non può pilotare direttamente i dispositivi), leggere e scrivere i file, far comunicare processi sullo stesso computer o su computer diversi (con memoria condivisa o scambio di messaggi), rilevare gli errori. Ci sono poi servizi per gestire il sistema nel suo insieme: allocazione delle risorse fra utenti e processi, contabilità del loro uso, protezione e sicurezza.

Anche il più semplice dei nostri programmi passa da questi servizi. Quando il programma del capitolo precedente esegue `printf`, il testo non va direttamente sullo schermo (figura 2.12).

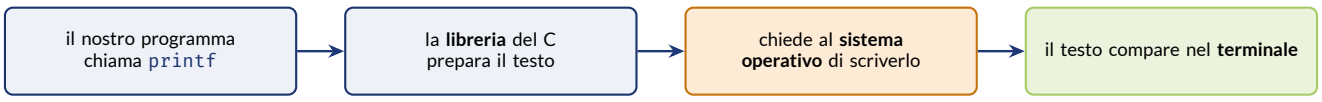


Figura 2.12: *Il percorso di una* `printf`*: dal programma alla libreria, dalla libreria al sistema operativo, e infine al dispositivo.*

Lo stesso sistema operativo gira su macchine diverse, quindi va **configurato** in fase di installazione: processore, memoria, suddivisione del disco (partizioni), dispositivi presenti, opzioni come l'algoritmo di schedulazione o il numero massimo di processi.

I primi sistemi operativi erano scritti in assembly; oggi si scrivono in linguaggi di alto livello, perché il codice si scrive più in fretta, è più compatto, più facile da capire e da correggere. Un sistema operativo scritto in un linguaggio di alto livello è molto più flessibile e manutenibile. Non a caso, il C nacque proprio per scrivere un sistema operativo, Unix, come racconta il capitolo 4.

In sintesi

Un calcolatore è fatto di CPU, memoria e dispositivi di I/O, collegati dal bus. La memoria è una fila di celle di un byte, ognuna con un indirizzo, che contengono sia i dati sia le istruzioni dei programmi. La CPU sa eseguire solo istruzioni elementari del proprio **linguaggio macchina**, una alla volta, guidata dal program counter; ripetizioni e scelte si ottengono con i salti. I linguaggi di alto livello come il C nascondono questi dettagli (astrazione), ma vanno **tradotti**. I bit di per sé non hanno significato: sono i **tipi** a stabilire come interpretarli. Il sistema operativo, il primo programma caricato all'avvio, gestisce processi, memoria, periferiche e file, e offre i suoi servizi a tutti gli altri programmi.

RIFERIMENTI



- **Deitel & Deitel**, *Il linguaggio C — Fondamenti e tecniche di programmazione*, Pearson: capitolo 1 (computer, hardware e software, linguaggi macchina, assembly e di alto livello).
- **Treccani**, voce *programmazione*.
  https://www.treccani.it/vocabolario/programmazione/

Dal problema al programma

Il capitolo precedente ha mostrato che cosa sa fare la macchina: eseguire, velocissima, istruzioni elementari scritte in linguaggio macchina. Questo capitolo percorre la strada nell'altro senso, partendo da un **problema** da risolvere: come si descrive la soluzione (l'algoritmo), come la si scrive in un linguaggio di alto livello (il programma) e come il programma diventa qualcosa che la CPU può eseguire.

Programmare

La programmazione è l'attività che porta a sviluppare un software. Non ogni procedimento, però, può diventare un programma: deve poter essere **descritto in modo non ambiguo**, deve **raggiungere l'obiettivo in un tempo finito** e deve essere **deterministico**. Un procedimento con queste caratteristiche si chiama **algoritmo**: una ricetta di cucina ben scritta, o il metodo per risolvere un'equazione di secondo grado.

Programmare significa descrivere al calcolatore *come* risolvere un problema. Il processore mette a disposizione molte operazioni di base (somma, sottrazione, operazioni logiche, spostamenti di valori, letture e scritture verso le periferiche), ma, come abbiamo visto, conviene esprimersi in un linguaggio di alto livello e lasciare a un programma il compito di tradurre.

I linguaggi di alto livello

Un linguaggio di alto livello (*HLL*, *High Level Language*) ha tre caratteristiche:

- un **alto livello di astrazione**: le istruzioni hanno un significato più complesso di quelle della macchina, e non serve sapere come il processore le realizza (`A + B` somma due valori, punto);
- viene **tradotto** in linguaggio macchina (o in assembly) da un programma traduttore, ed è quindi indipendente dal processore e dal sistema operativo;
- è più **simile al linguaggio umano**, quindi più facile da scrivere e da ricordare (in molti linguaggi `print X` mostra il valore di `X`).

Comunque avvenga la traduzione, il programmatore scrive le istruzioni in un file di testo: il **codice sorgente** (o *programma sorgente*, o semplicemente *programma*). Per tradurlo ci sono due strategie, l'**interprete** e il **compilatore**.

Interprete e compilatore

Un **interprete** traduce le istruzioni del codice sorgente **una alla volta** in linguaggio macchina, e ciascuna viene eseguita subito dalla CPU, prima di passare alla successiva. Un **compilatore**, invece, traduce **tutto** il codice sorgente in linguaggio macchina e lo salva in un file, il **programma eseguibile**; l'esecuzione avviene dopo, ed è un'operazione separata (figura 3.1).

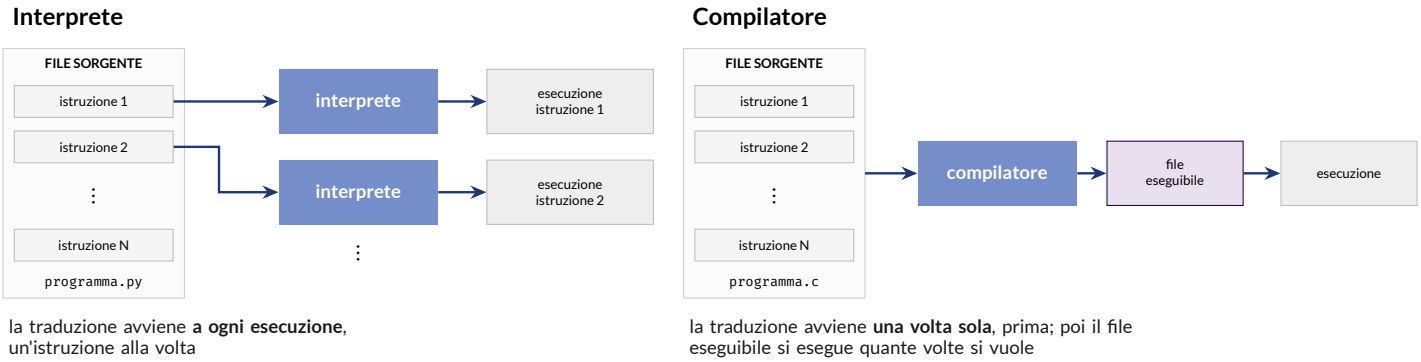


Figura 3.1: *Interprete (a sinistra) e compilatore (a destra).*

Il C è un linguaggio **compilato**; Python e JavaScript, per esempio, sono di solito interpretati. Le due strategie hanno pregi e difetti opposti:

	Programma interpretato	**Programma compilato**
velocità	più lento: a ogni esecuzione l'interprete deve ritradurre le istruzioni	più veloce: è già tradotto; e il compilatore sa *ottimizzare* il codice, per renderlo più veloce o più piccolo
chi deve avere che cosa	l'utente deve avere l'interprete e saperlo usare (si può nascondere, ma c'è)	all'utente basta l'eseguibile (un doppio clic); solo il programmatore ha bisogno del compilatore
codice sorgente	serve all'utente, quindi è visibile e modificabile da chiunque	non serve per eseguire: si può non distribuirlo, proteggendo il lavoro (risalire al sorgente è possibile, ma difficile)

Una via di mezzo: il bytecode

Alcuni linguaggi, come Java e Python, compilano il sorgente in un file intermedio, non eseguibile direttamente, scritto in una sorta di assembly generico chiamato **bytecode**, che si traduce molto rapidamente nel codice macchina di un processore reale. Su ogni computer si installa un interprete del bytecode (per Java, la *Java Virtual Machine*): così lo stesso file gira su macchine con linguaggi macchina diversi (figura 3.2).

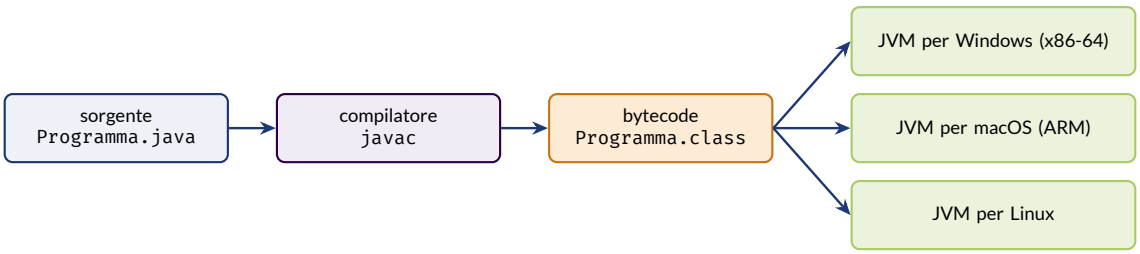


Figura 3.2: *Un solo file di bytecode, un interprete diverso per ogni tipo di macchina.*

Le librerie

In un linguaggio di alto livello il programmatore non deve programmare da sé le funzioni di base: calcolare una radice quadrata o una tangente, stampare un numero sullo schermo, leggere un valore dalla tastiera. Queste operazioni sono state programmate e compilate una volta per tutte da chi produce il compilatore, e sono a disposizione sotto forma di **funzioni** (per esempio `tan(x)` calcola la tangente di `x`, `printf` stampa).

Il loro codice, già tradotto in linguaggio macchina, è raccolto in file chiamati **librerie**: collezioni di funzioni pronte all'uso. Un programma C "vero" è quindi fatto di due parti: il codice scritto da noi e il codice delle librerie che usa.

Come nasce un eseguibile

Torniamo al compilatore. Finora lo abbiamo trattato come una scatola chiusa, che riceve il sorgente e restituisce l'eseguibile; in realtà la traduzione di un programma C avviene in più passaggi, ciascuno svolto da uno strumento diverso. Conoscerli aiuta a capire i messaggi di errore, che cambiano a seconda del passaggio in cui qualcosa è andato storto.

Compilazione e linking

La creazione di un eseguibile (in inglese *build*) avviene in due fasi (figura 3.3):

1. **compilazione**: il sorgente viene tradotto in codice macchina, ma le funzioni di libreria che usa sono ancora **mancanti**. Il risultato è un file intermedio, il **file oggetto** (estensione `.o`), che non si può eseguire;
2. **linking** (collegamento): uno o più file oggetto e una o più librerie (già in codice macchina) vengono **uniti** dal **linker**, che aggiunge le parti mancanti e produce un unico file eseguibile.

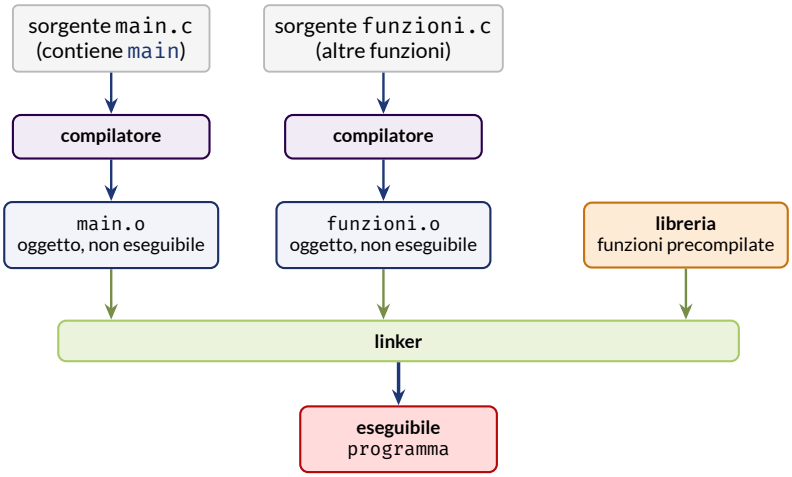


Figura 3.3: *La compilazione produce i file oggetto, il linking li unisce fra loro e con le librerie.*

Quando nel capitolo 1 abbiamo scritto `gcc ciao.c`, gcc ha eseguito entrambe le fasi in un colpo solo. Le si può però separare, ed è istruttivo farlo con il primo programma di esempio del corso, `area_cerchio.c` (lo vedremo per intero fra poco). L'opzione `-c` (*compile*) chiede a gcc di fermarsi dopo la compilazione:

Terminale

```
$ gcc -c area_cerchio.c
$ ls -l area_cerchio.c area_cerchio.o
-rw-r--r-- 1 fabio fabio  255 ott 31 10:09 area_cerchio.c
-rw-r--r-- 1 fabio fabio 1960 ott 31 10:09 area_cerchio.o
$ nm area_cerchio.o
                 U __isoc99_scanf
                 U __stack_chk_fail
0000000000000000 T main
                 U printf
```

Il comando `nm` elenca i nomi che compaiono in un file oggetto. La `T` accanto a `main` significa che il suo codice è **presente** nel file (*text*, cioè codice); la `U` accanto a `printf` e `scanf` (che compare con il nome interno `__isoc99_scanf`) significa **undefined**: il programma le usa, ma il loro codice non c'è. Sono proprio le "funzioni di libreria ancora mancanti". Il linking le aggiunge:

Terminale

```
$ gcc -o area_cerchio area_cerchio.o
$ ./area_cerchio
Inserisci il raggio del cerchio: 5
L'area del cerchio è: 78.54
```

Dato un file `.o` invece di un `.c`, gcc salta la compilazione e fa solo il linking. Separare le due fasi diventa utile quando un programma è diviso in più file: se ne cambia uno, basta ricompilare quello e rifare il linking. Lo vedremo nel capitolo 7.

La catena completa

La figura 3.4 mostra tutto il percorso di un programma C, dall'editor fino alla CPU, con gli strumenti coinvolti.

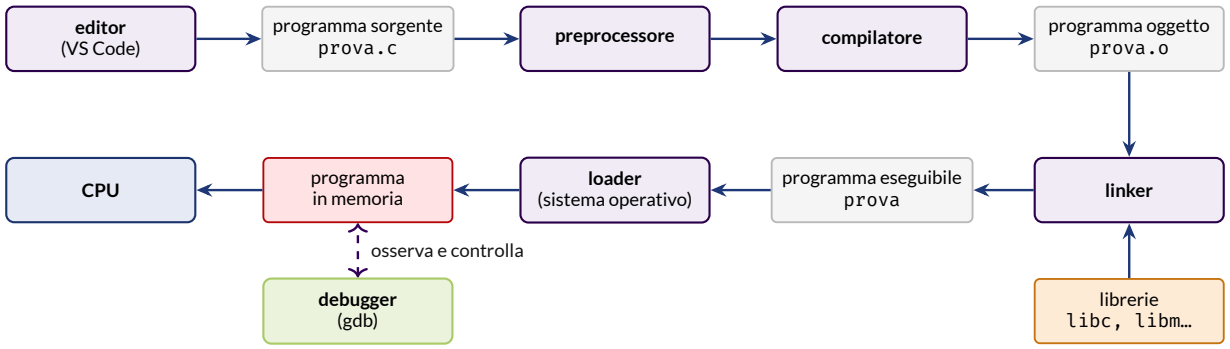


Figura 3.4: *Dal sorgente al processore: gli strumenti (in viola) e i file che producono.*

- L'**editor** serve a scrivere il sorgente.
- Il **preprocessore** lavora sul testo prima della compilazione vera e propria: esegue le righe che cominciano con `#`. `#include <stdio.h>` viene sostituita dall'intero contenuto del file `stdio.h`; `#define PI 3.14159` fa sì che ogni `PI` del programma diventi `3.14159`.
- Il **compilatore** traduce in codice macchina e produce il file oggetto; il **linker** lo unisce alle librerie.
- Il **loader** è la parte del sistema operativo che, quando scriviamo `./prova`, copia l'eseguibile in memoria e fa partire la CPU dalla sua prima istruzione (capitolo 2).
- Il **debugger** permette di eseguire il programma un passo alla volta, fermarlo e guardare il contenuto delle variabili: è lo strumento per la *messa a punto*.

Il lavoro del preprocessore si può vedere con l'opzione `-E`, che ferma gcc subito dopo questa fase e stampa il testo risultante. Il nostro programma di 15 righe diventa di oltre 800, quasi tutte provenienti da `stdio.h`; in fondo c'è il nostro codice, con `PI` già sostituito:

Terminale

```
$ gcc -E area_cerchio.c | wc -l
828
$ gcc -E area_cerchio.c | tail -6
    scanf("%f", &raggio);

    area = 3.14159 * raggio * raggio;

    printf("L'area del cerchio è: %.2f\n", area);
```

(Il simbolo `|` è una *pipe*: passa l'output di un comando a un altro. `wc -l` conta le righe, `tail -6` mostra le ultime 6.)

Librerie statiche e dinamiche

Il linking può collegare una libreria in due modi.

Con una **libreria statica**, il codice delle funzioni usate viene **copiato** dentro il file eseguibile. Le stesse funzioni possono essere usate da più programmi, ma ognuno ne contiene una copia (figura 3.5, a sinistra).

Con una **libreria dinamica**, nell'eseguibile **non** viene inserito il codice delle funzioni, ma solo un **riferimento**: il nome del file della libreria e il nome della funzione. Su Windows sono le *DLL* (*Dynamic Link Libraries*), su Linux e macOS le *shared libraries* (file `.so` e `.dylib`). Quando parte il primo programma che usa una libreria dinamica, il sistema operativo la carica in memoria; se poi parte un altro programma che usa la stessa libreria, la trova già in memoria e la **condivide** (figura 3.5, a destra).

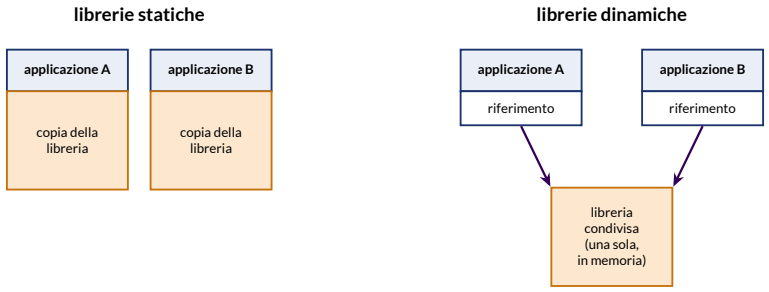


Figura 3.5: *A sinistra ogni eseguibile contiene la sua copia della libreria; a destra gli eseguibili contengono solo un riferimento a un'unica copia, condivisa.*

I vantaggi delle librerie dinamiche sono evidenti: gli eseguibili sono più piccoli, un programma parte più in fretta se la libreria è già in memoria e, se una funzione va aggiornata (una nuova versione, la correzione di un errore), basta sostituire la libreria, senza ritoccare i programmi. Il rovescio della medaglia: se la libreria manca o è danneggiata, il programma non funziona.

gcc usa le librerie dinamiche, se non si chiede diversamente. Il comando `ldd` mostra da quali dipende un eseguibile: il nostro dipende da `libc.so.6`, la libreria standard del C. Con l'opzione `-static` si ottiene invece un collegamento statico, e la differenza di dimensione parla da sola:

Terminale

```
$ ldd area_cerchio
        linux-vdso.so.1 (0x00007f1b90918000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f1b90600000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f1b9091a000)
$ gcc -static -o area_statico area_cerchio.o
$ ls -l area_cerchio area_statico
-rwxr-xr-x 1 fabio fabio  16072 ott 31 10:09 area_cerchio
-rwxr-xr-x 1 fabio fabio 915784 ott 31 10:09 area_statico
```

Lo stesso programma pesa 16 kilobyte con la libreria dinamica e quasi un megabyte con quella statica, che contiene dentro di sé tutto il codice di `printf`, `scanf` e delle funzioni che queste usano. (Su macOS `-static` non è disponibile per i programmi normali, e al posto di `ldd` si usa `otool -L`.)

GLI ALGORITMI

Che cos'è un algoritmo

Possiamo ora precisare la definizione del capitolo precedente. Un algoritmo è la **descrizione operativa** di come risolvere un problema: un **elenco finito di istruzioni elementari**, da eseguire in successione. Le singole istruzioni devono:

- richiedere un **tempo finito**;
- **non essere ambigue**;
- avere un **effetto osservabile**.

Inoltre l'algoritmo deve produrre un **risultato**, sempre lo stesso a partire dalle stesse condizioni iniziali, e le risorse che richiede devono essere **ragionevoli** per la tecnologia attuale: il tempo complessivo di esecuzione, la memoria massima necessaria, eventuali dispositivi periferici particolari.

Il termine nasce in informatica, ma si applica a qualunque procedimento. Cambiano le istruzioni elementari, non le caratteristiche:

Algoritmo per...	**un'istruzione elementare**
risolvere un'equazione di secondo grado	calcola $b^2 - 4ac$
preparare una ricetta	sbatti 4 albumi
montare un armadio IKEA	unisci i pannelli 12 e 13 con una vite C
avviare il motore di un aereo	inserisci i magneti
installare un'app sul cellulare	tocca il pulsante Installa

Sono invece istruzioni **non adeguate**:

- "calcola tutte le cifre di π": le cifre sono infinite, quindi l'istruzione non termina mai (tempo non finito);
- "aggiungi alcuni degli ingredienti": quali? quanti? (ambigua).

E un algoritmo che con gli stessi dati iniziali produce risultati diversi **non è corretto**, perché non è deterministico: un'equazione di secondo grado deve dare sempre le stesse soluzioni per gli stessi valori di a, b e c.

Un algoritmo che richiede un tempo irragionevole per la tecnologia attuale, considerati costi e benefici, **non risolve il problema**, anche se è corretto. L'esempio delle lezioni: aprire un file zip protetto da password senza conoscerla. Un algoritmo c'è, ed è semplicissimo: provare tutte le combinazioni di caratteri finché non si trova quella giusta (il cosiddetto attacco a *forza bruta*).

Supponiamo che la password sia di 8 caratteri, scelti fra i 96 caratteri stampabili del codice ASCII standard. Per il primo carattere ci sono 96 possibilità; per ciascuna, 96 per il secondo, e così via: in tutto $96^8 \approx 7{,}2 \cdot 10^{15}$ password possibili, cioè circa 7,2 milioni di miliardi. Con un computer che ne prova 34 milioni al secondo, senza mai fermarsi:

$$\frac{7{,}2 \cdot 10^{15}}{3{,}4 \cdot 10^{7} \text{ al secondo}} \approx 2{,}1 \cdot 10^{8} \text{ secondi} \approx 6{,}7 \text{ anni.}$$

Un conto da rifare

Le slide parlano di "quasi 5 anni" per provare tutte le password; il calcolo esatto dà circa **6,7 anni**. In media, però, la password giusta si trova dopo averne provate la metà, cioè in circa 3,4 anni. La conclusione non cambia: per chi ha bisogno del file oggi, l'algoritmo è inutile.

La cosa più istruttiva è che cosa succede allungando la password: ogni carattere in più **moltiplica per 96** il numero di tentativi.

lunghezza	password possibili	tempo per provarle tutte
8	$7{,}2 \cdot 10^{15}$	circa 6,7 anni
9	$6{,}9 \cdot 10^{17}$	circa 645 anni
10	$6{,}6 \cdot 10^{19}$	circa 62 000 anni

Una crescita di questo tipo si chiama *esponenziale*, ed è il motivo per cui le password lunghe sono sicure: non serve un algoritmo più furbo per violarle, servirebbe un tempo che nessuno ha.

Come si descrive un algoritmo

Prima di scrivere un programma conviene descrivere l'algoritmo in una forma intermedia, più libera di un linguaggio di programmazione. Le forme più usate sono:

- la **descrizione testuale**, in italiano;
- lo **pseudo-codice**: una descrizione testuale con un certo livello di formalismo, con parole chiave fisse (`LEGGI`, `STAMPA`, `SE`…) e una struttura simile a quella di un programma, ma senza le regole rigide di un linguaggio vero;
- il **diagramma di flusso** (*flow-chart*): una rappresentazione grafica, in cui ogni forma ha un significato (figura 3.6);
- il **linguaggio di alto livello**, cioè il programma vero e proprio.

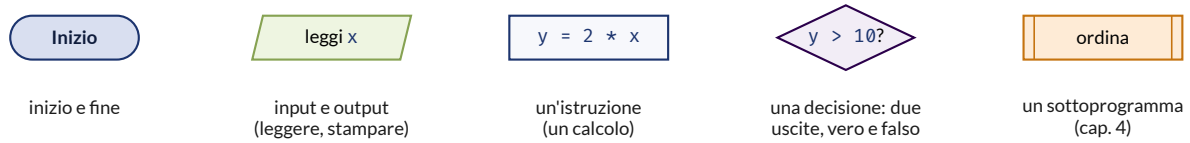


Figura 3.6: *Le forme dei diagrammi di flusso.*

Conviene anche organizzare la descrizione in tre parti: l'**input** (i dati di partenza), l'**elaborazione** (i calcoli) e l'**output** (i risultati). Vediamo i tre esempi delle lezioni, in tutte le forme. Il codice C è commentato solo a grandi linee: ogni dettaglio sarà spiegato nel capitolo 5. Qui conta vedere come ogni riga di pseudo-codice diventa una riga di programma.

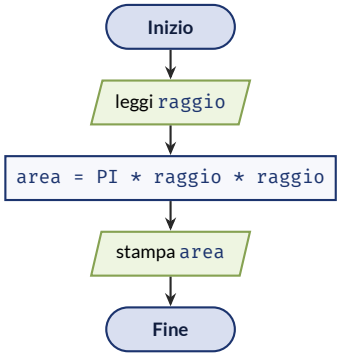
Esempio 1: l'area di un cerchio

- **Input**: il raggio del cerchio r, un numero reale positivo.
- **Elaborazione**: l'area si calcola con la formula $A = \pi \cdot r^2$, dove π è la costante che rappresenta il rapporto fra la circonferenza di un cerchio e il suo diametro.
- **Output**: il valore numerico dell'area.

In forma testuale: *acquisisci dall'utente il valore del raggio; calcola l'area con la formula $\pi \cdot r \cdot r$; visualizza il valore ottenuto.* In pseudo-codice e come diagramma di flusso:

pseudocodice

```
BEGIN
  LEGGI raggio
  AREA = PI * raggio * raggio
  STAMPA AREA
END
```



Ed ecco il programma C, con l'esecuzione accanto:

area_cerchio.c

```c
#include <stdio.h>
#define PI 3.14159

int main() {
    float raggio, area;

    printf("Inserisci il raggio del cerchio: ");
    scanf("%f", &raggio);

    area = PI * raggio * raggio;

    printf("L'area del cerchio è: %.2f\n", area);

    return 0;
}
```

Terminale

```
$ gcc -Wall -o area_cerchio area_cerchio.c
$ ./area_cerchio
Inserisci il raggio del cerchio: 5
L'area del cerchio è: 78.54
```

La corrispondenza con lo pseudo-codice è diretta:

pseudo-codice	**C**	**nota**
	`float raggio, area;`	in C bisogna dichiarare le variabili prima di usarle, dicendone il tipo (`float`: numero reale)
`LEGGI raggio`	`scanf("%f", &raggio);`	prima si stampa un messaggio che dice all'utente che cosa scrivere
`AREA = PI * raggio * raggio`	`area = PI * raggio * raggio;`	identico; `PI` è definito dalla riga `#define`
`STAMPA AREA`	`printf("…%.2f\n", area);`	`%.2f` significa: un numero reale con 2 decimali

Esempio 2: da Fahrenheit a Celsius

- **Input**: una temperatura in gradi Fahrenheit (un numero reale).
- **Elaborazione**: la formula di conversione è $Celsius = (Fahrenheit - 32) \cdot 5/9$.
- **Output**: la temperatura in gradi Celsius.

Lo pseudo-codice ha la stessa forma del primo esempio, e così il programma:

pseudocodice

```
BEGIN
  LEGGI fahrenheit
  CELSIUS = (fahrenheit - 32) * 5 / 9
  STAMPA CELSIUS
END
```

LEGGI diventa `scanf`, il calcolo resta identico, STAMPA diventa `printf`.

conversione.c

```c
#include <stdio.h>

int main() {
    float fahrenheit, celsius;

    printf("Inserisci la temperatura in Fahrenheit: ");
    scanf("%f", &fahrenheit);

    celsius = (fahrenheit - 32) * 5 / 9;

    printf("La temperatura in Celsius è: %.2f\n", celsius);

    return 0;
}
```

Terminale

```
$ gcc -Wall -o conversione conversione.c
$ ./conversione
Inserisci la temperatura in Fahrenheit: 98.6
La temperatura in Celsius è: 37.00
```

L'ordine delle operazioni conta

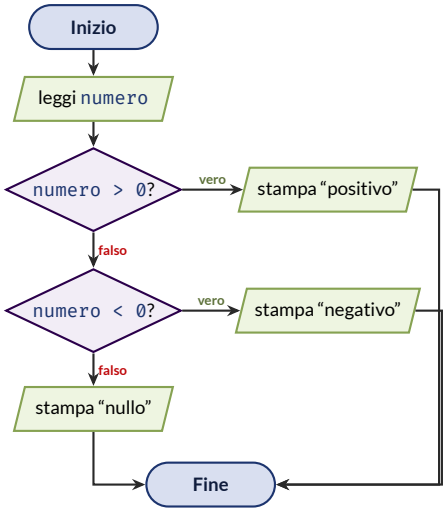
Matematicamente $(F - 32) \cdot 5/9$ e $5/9 \cdot (F - 32)$ sono la stessa cosa. In C no: scrivendo `celsius = 5 / 9 * (fahrenheit - 32);` il risultato è sempre **0**. Il motivo è che `5` e `9` sono numeri interi, e in C la divisione fra interi dà un intero, troncando i decimali: `5 / 9` vale 0, e 0 per qualunque cosa fa 0. Nella versione del programma, invece, `(fahrenheit - 32)` è un `float`, quindi anche le operazioni successive avvengono fra reali. È una delle trappole più comuni del C, e la ritroveremo nel capitolo 5.

Esempio 3: il segno di un numero

Il terzo esempio introduce una **scelta**: il programma deve fare cose diverse a seconda del dato.

- **Input**: un numero intero.
- **Elaborazione**: se il numero è maggiore di zero è positivo; se è minore di zero è negativo; altrimenti è nullo.
- **Output**: un messaggio che indica se il numero è positivo, negativo o nullo.

```pseudocodice
BEGIN
  LEGGI numero
  SE numero > 0 ALLORA
    STAMPA "Il numero è positivo"
  ALTRIMENTI SE numero < 0 ALLORA
    STAMPA "Il numero è negativo"
  ALTRIMENTI
    STAMPA "Il numero è nullo"
  END SE
END
```



Nel diagramma di flusso si vede bene che, qualunque sia il numero, viene eseguita **una sola** delle tre stampe: le strade si separano ai rombi e si ricongiungono prima della fine. In C la scelta si scrive con `if` ("se") ed `else` ("altrimenti"):

```c
#include <stdio.h>

int main() {
    int numero;

    printf("Inserisci un numero: ");
    scanf("%d", &numero);

    if (numero > 0) {
        printf("Il numero è positivo.\n");
    } else if (numero < 0) {
        printf("Il numero è negativo.\n");
    } else {
        printf("Il numero è zero.\n");
    }

    return 0;
}
```

```
$ ./positivo
Inserisci un numero: 7
Il numero è positivo.
$ ./positivo
Inserisci un numero: -3
Il numero è negativo.
$ ./positivo
Inserisci un numero: 0
Il numero è zero.
```

Per verificare un programma con una scelta, lo si prova con un dato per ogni strada possibile.

Si noti l'ultima nota: un programma con tre strade va provato **tre volte**, una per strada. Un errore nascosto in un ramo che non si prova mai resta lì, in attesa del giorno sbagliato.

In sintesi

Un algoritmo è un elenco finito di istruzioni elementari, non ambigue, ciascuna eseguibile in un tempo finito, che produce sempre lo stesso risultato dagli stessi dati con risorse ragionevoli. Lo si descrive in italiano, in pseudo-codice, con un diagramma di flusso e infine in un linguaggio di alto livello. Il programma sorgente va tradotto: un **interprete** lo fa un'istruzione alla volta a ogni esecuzione, un **compilatore** una volta sola. Per il C la traduzione passa per il preprocessore, il compilatore (che produce i file oggetto) e il linker (che li unisce alle librerie, statiche o dinamiche); al momento dell'esecuzione il loader carica il programma in memoria.

Riferimenti

- **Deitel & Deitel**, *Il linguaggio C*: capitoli 1 e 3 (ambiente di sviluppo, algoritmi, pseudo-codice).
- **GCC — Overall Options**: le opzioni `-E`, `-S`, `-c` e `-o`.
  `https://gcc.gnu.org/onlinedocs/gcc/Overall-Options.html`

La programmazione strutturata

Prima di scrivere programmi in C conviene sapere da dove viene il linguaggio e, soprattutto, quale **disciplina** segue: la programmazione strutturata. È un'idea semplice e potentissima: *ogni* programma si può costruire combinando tre soli schemi, la sequenza, la selezione e l'iterazione. Questo capitolo li presenta con i diagrammi di flusso, prima con esempi della vita di tutti i giorni e poi con problemi di informatica; nei capitoli successivi ciascuno diventerà un'istruzione del C.

Il linguaggio C

Da dove viene

Il C è stato creato da **Dennis Ritchie**, ai Bell Labs, nel 1972, come linguaggio per scrivere il sistema operativo **Unix**. La prima versione di Unix, opera di **Ken Thompson**, era scritta in assembly e in un linguaggio chiamato **B**. Il B aveva dei limiti seri:

- non era **tipizzato**: conosceva un solo tipo di dato, la *parola macchina* (il gruppo di bit su cui lavora il processore), e non distingueva fra numeri interi, caratteri o indirizzi;
- faceva un uso pesante di puntatori e indirizzi di memoria.

Il C nasce per superare questi limiti: è un'evoluzione del B che incorpora i **tipi** (ricordate gli "occhiali" del capitolo 2: è il tipo a dire come interpretare i bit), e rispetta i canoni della programmazione strutturata. Nel tempo il linguaggio si è evoluto (figura 4.1):

- il **C tradizionale** (*Traditional C*), descritto nel 1978 nel libro di Kernighan e Ritchie e diffuso all'inizio degli anni '80, introduce fra l'altro i tipi `void` ed `enum`;
- l'**ANSI C**, della fine degli anni '80, aggiunge il tipo `void *`, i **prototipi** delle funzioni e una definizione del linguaggio molto più precisa.

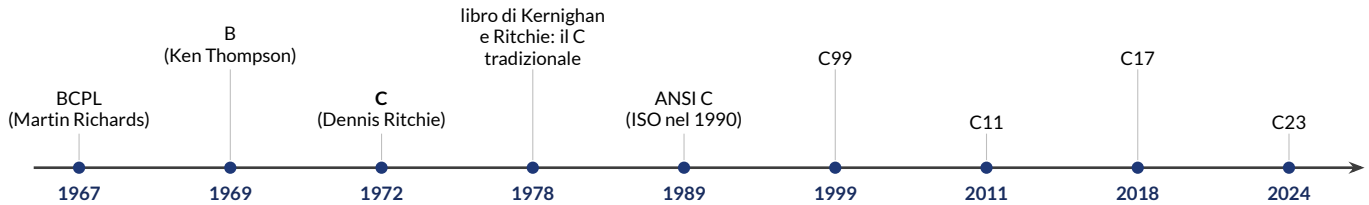


Figura 4.1: *Le tappe del C, dai suoi antenati alle versioni dello standard.*

Una precisazione storica

Le slide datano il B al 1967. Il 1967 è l'anno del suo antenato, il *BCPL* di Martin Richards; il B fu sviluppato da Ken Thompson intorno al 1969, partendo proprio dal BCPL.

Lo standard

L'**ANSI** (*American National Standards Institute*) è l'ente statunitense che definisce gli standard per molti tipi di sistemi, compresi i linguaggi di programmazione. Lo standard ANSI del C (1989) fu approvato nel 1990 dall'**ISO** (*International Organization for Standardization*), e l'ANSI C divenne così lo standard internazionale.

Uno standard specifica il **formalismo** del linguaggio (quali testi sono programmi C corretti, cioè la sua *sintassi*) e la sua **semantica** (che cosa significa ciascun costrutto, cioè che cosa deve fare il programma). Lo scopo è triplice:

- **portabilità**: lo stesso programma si compila ed esegue allo stesso modo con compilatori diversi, su macchine diverse;
- **affidabilità**: il comportamento è definito, non lasciato all'interpretazione di chi scrive il compilatore;
- **manutenibilità**: chiunque conosca lo standard può leggere e modificare il programma.

Dopo il 1990 lo standard è stato aggiornato più volte (C99, C11, C17, C23). Le versioni recenti di gcc seguono per default una di queste, con alcune estensioni proprie; i programmi di questa dispensa funzionano con tutte.

LA PROGRAMMAZIONE STRUTTURATA

Il problema: gli spaghetti

Nel capitolo 2 abbiamo visto che in assembly ripetizioni e scelte si ottengono con i **salti**: "vai all'istruzione all'indirizzo tale". I primi linguaggi di alto livello ereditarono questa abitudine con l'istruzione `GOTO` ("vai a"), che permette di saltare in qualunque punto del programma. In un programma grande, pieno di salti in avanti e all'indietro, seguire l'esecuzione diventa come seguire uno spaghetto nel piatto: è il cosiddetto **spaghetti code** (figura 4.2, a sinistra), di bassa qualità e leggibilità.

Nel 1968–1969 **Edsger W. Dijkstra** criticò duramente l'uso del `GOTO` e propose di regolamentare e standardizzare le metodologie di programmazione: è la **programmazione strutturata**. L'obiettivo è rendere più facile la **lettura**, la **modifica** e la **manutenzione** dei programmi.

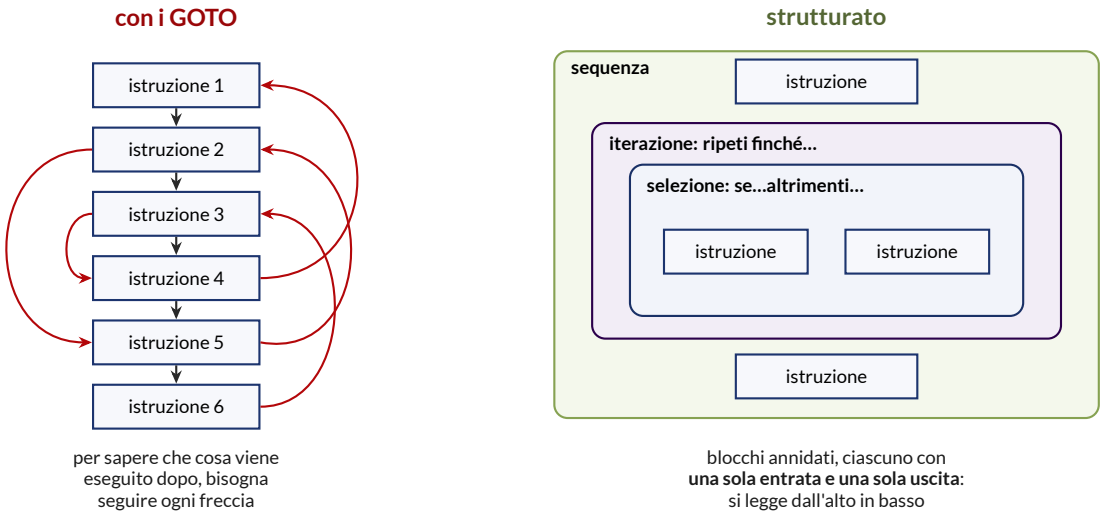


Figura 4.2: *Lo stesso tipo di logica scritto con salti arbitrari (a sinistra) e con le strutture di controllo (a destra).*

Si può davvero fare a meno dei salti arbitrari? La risposta è un teorema, dimostrato nel 1966 da due informatici italiani, **Corrado Böhm** e **Giuseppe Jacopini**.

Teorema di Böhm-Jacopini

Qualsiasi programma scritto usando il `GOTO` può essere riscritto senza, a patto di avere a disposizione tre tipi di **strutture di controllo**: la **sequenza**, la **selezione** e l'**iterazione**.

Le tre strutture bastano, quindi, per *qualunque* algoritmo. Hanno una caratteristica comune decisiva: ciascuna ha **una sola entrata e una sola uscita**. Per questo si possono incastrare l'una dentro l'altra come scatole (figura 4.2, a destra), e un programma si legge dall'alto in basso, capendo ogni blocco senza dover guardare il resto.

Che fine ha fatto il GOTO?

Il C ha ancora un'istruzione `goto`, per ragioni storiche e per pochi casi molto particolari. In questo corso **non si usa**: tutto ciò che serve si scrive con le strutture di controllo.

LE TRE STRUTTURE DI CONTROLLO

Le strutture di controllo del flusso sono dunque tre (figura 4.3). Vediamole nel dettaglio, ciascuna con un problema del mondo reale e un problema di informatica.

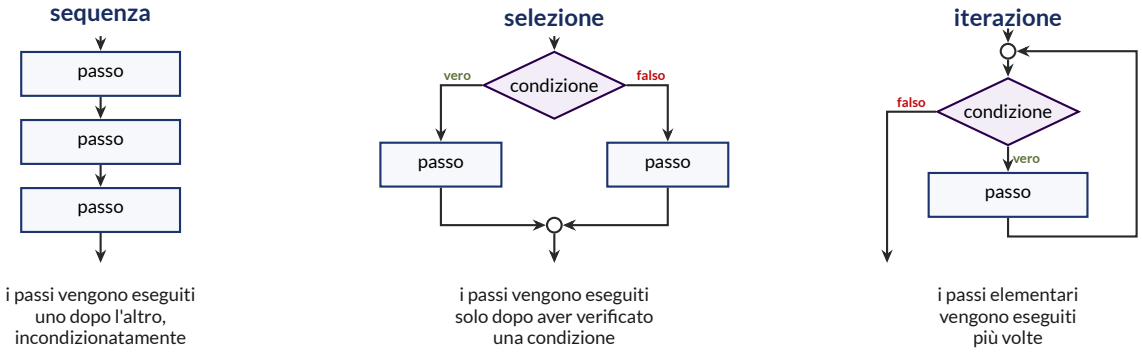


Figura 4.3: *Le tre strutture di controllo del flusso.*

La sequenza

La **sequenza** è la struttura più banale: i passi vengono eseguiti **uno dopo l'altro**, incondizionatamente, nell'ordine in cui sono scritti. Ma l'ordine conta: per alzarsi dal letto bisogna prima mettersi a sedere, e per stampare un risultato bisogna prima averlo calcolato (figura 4.4).

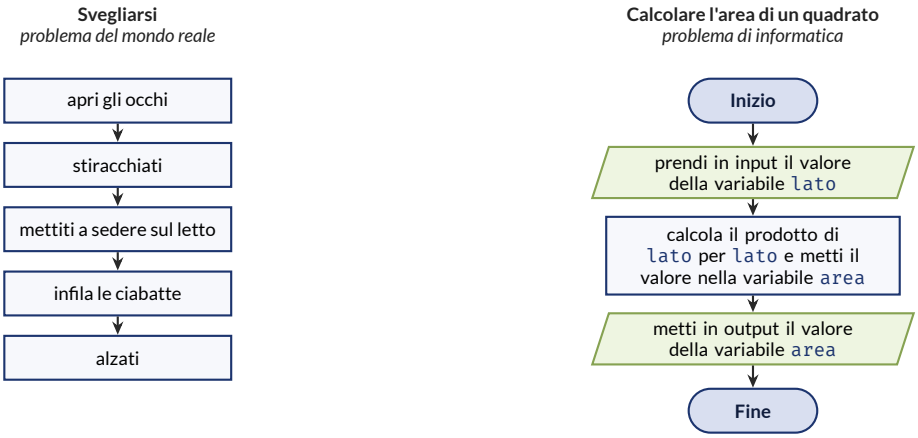


Figura 4.4: *Due sequenze.*

Nel quadrato compare per la prima volta la parola **variabile**: un contenitore con un nome (`lato`, `area`) in cui si mette un valore. È una cella di memoria (o un gruppo di celle) a cui diamo un nome, così da non doverci occupare del suo indirizzo.

La selezione a una via

La **selezione** esegue dei passi **solo dopo aver verificato una condizione**. Ne esistono tre tipi; il più semplice è la **selezione a una via**: se la condizione è vera si eseguono dei passi, se è falsa non si fa niente di particolare e si prosegue (figura 4.5).

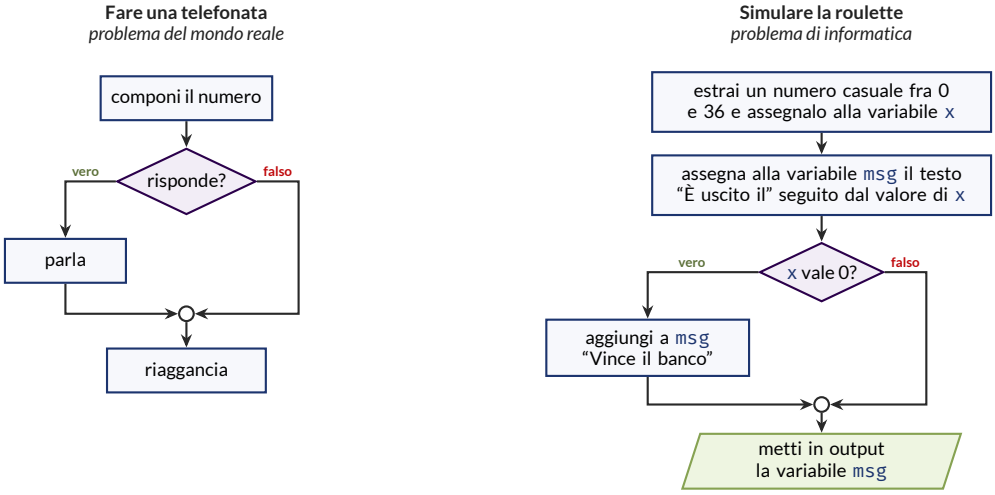


Figura 4.5: *Due selezioni a una via: il ramo "falso" non contiene passi.*

Il pallino dove le strade si ricongiungono è importante: qualunque cosa succeda alla domanda, dopo si riaggancia. La selezione ha una sola entrata (la domanda) e una sola uscita (il pallino), proprio come una singola istruzione.

La selezione a due vie

La **selezione a due vie** prevede delle operazioni da svolgere sia nel caso in cui la condizione risulti **vera**, sia nel caso in cui risulti **falsa**: viene eseguito sempre *uno* dei due rami, mai entrambi, mai nessuno (figura 4.6).

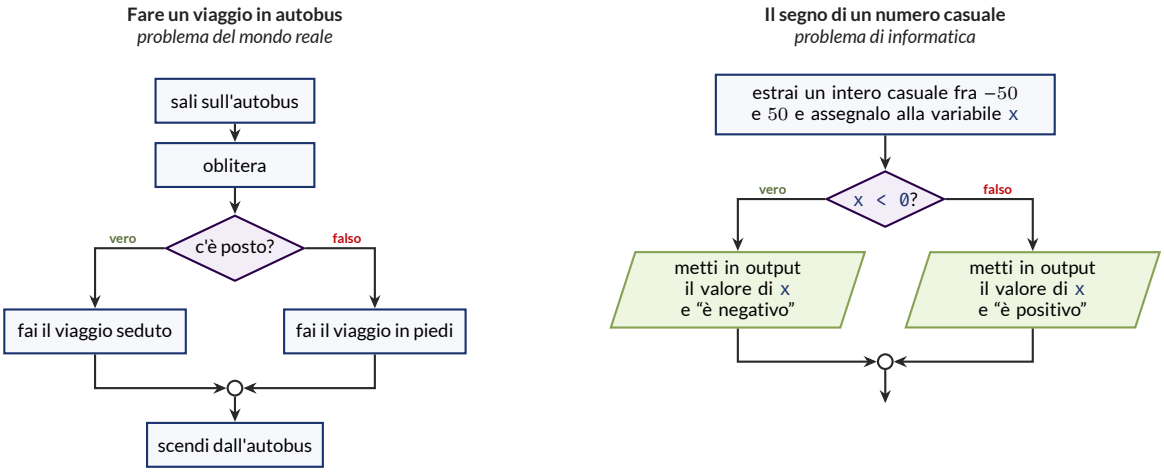


Figura 4.6: *Due selezioni a due vie.*

E se esce lo zero?

L'algoritmo del segno, così com'è, ha un caso particolare: se il numero estratto è 0, la condizione `x < 0` è falsa e viene stampato "0 è positivo". Lo zero però non è né positivo né negativo. Per trattarlo correttamente servono **tre** strade, come nell'esempio 3 del capitolo 3. È un buon esempio di come si verifica un algoritmo: si prova ogni ramo, e si cercano i **casi limite** (qui, il confine fra le due strade).

La selezione a n vie

La **selezione a n vie** prevede operazioni diverse in **diversi casi**, a seconda del valore di qualcosa: invece di una domanda con risposta vero o falso, c'è una domanda con molte risposte possibili, ciascuna con la sua strada (figura 4.7).

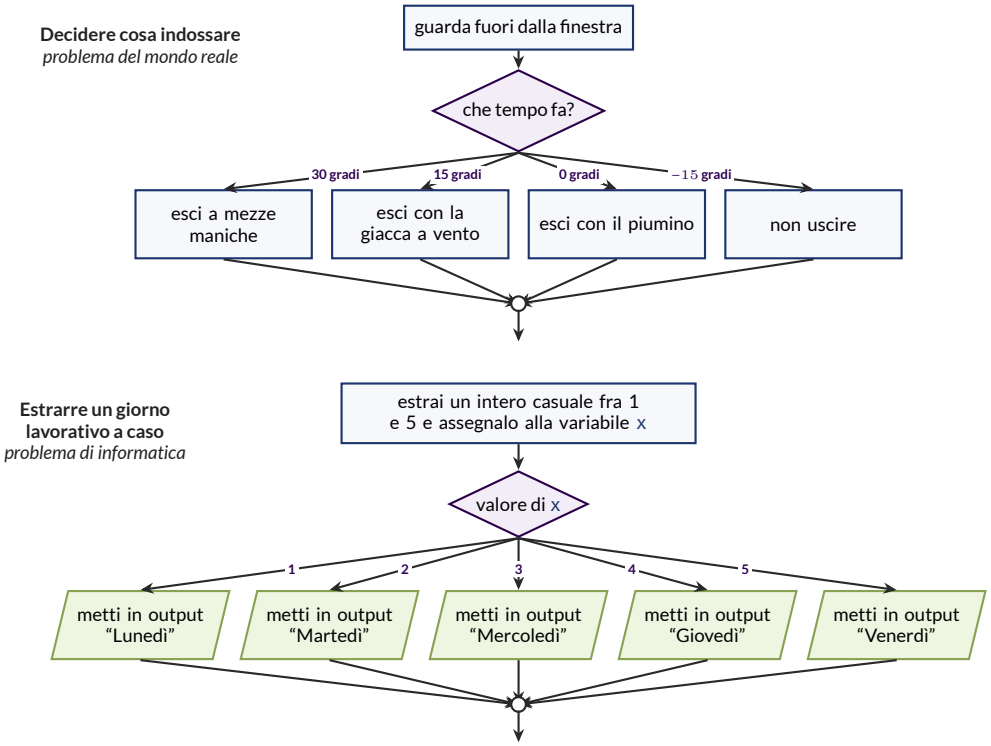


Figura 4.7: *Due selezioni a n vie: le etichette sulle frecce sono i valori possibili.*

Si potrebbe ottenere lo stesso risultato con una catena di selezioni a due vie ("x vale 1? altrimenti, vale 2? altrimenti…"), ma la selezione a n vie dice in modo più diretto che **una sola** domanda decide fra molte strade. In C esistono entrambe le forme (capitolo **??**).

Il concetto di **ciclo** (o *iterazione*) è stato introdotto per alleggerire situazioni come questa: dipingere le statuette dei sette nani. Scritto come sequenza, l'algoritmo ripete sette volte lo stesso passo (figura 4.8, a sinistra). Una volta isolata l'istruzione da eseguire **più volte**, si può scrivere un algoritmo che la esegue semplicemente quante volte serve (a destra).

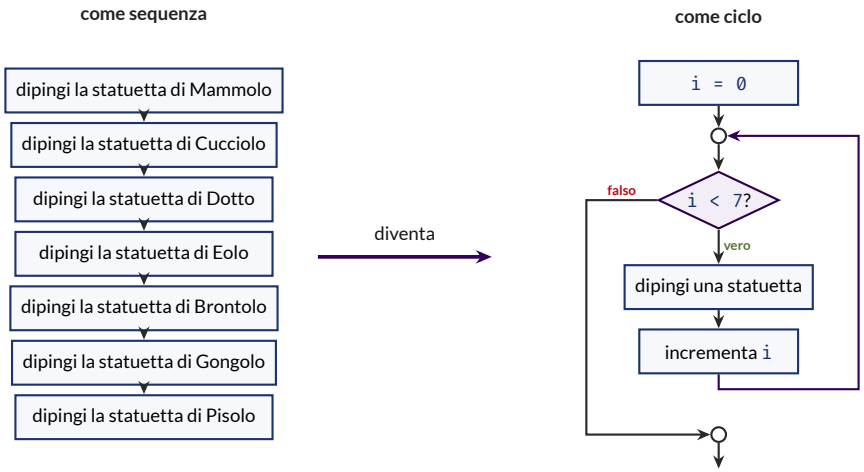


Figura 4.8: *Dipingere le statuette dei sette nani: la ripetizione viene isolata in un ciclo.*

Per sapere quando fermarsi, il ciclo usa una variabile, `i`, che **conta** quante volte il corpo del ciclo è già stato eseguito. Una variabile di questo tipo si chiama **contatore** e per tradizione si chiama `i`. Esistono però due grandi famiglie di cicli, e la difficoltà sta nel capire in quale circostanza conviene usare l'una o l'altra:

- il **ciclo su contatore**, quando si sa *quante volte* ripetere;
- il **ciclo su condizione**, quando non lo si sa, ma si sa *quando* continuare e quando smettere.

Il ciclo su contatore

Nel ciclo su contatore la variabile contatore viene **inizializzata** prima di fare il ciclo, **testata** prima di ripetere il ciclo e **modificata** ogni volta che il ciclo è stato eseguito (figura 4.9, a sinistra). Dimenticare uno dei tre passi è un errore classico: senza inizializzazione il conto parte da un valore a caso, senza modifica il test resta sempre vero e il ciclo non finisce mai.

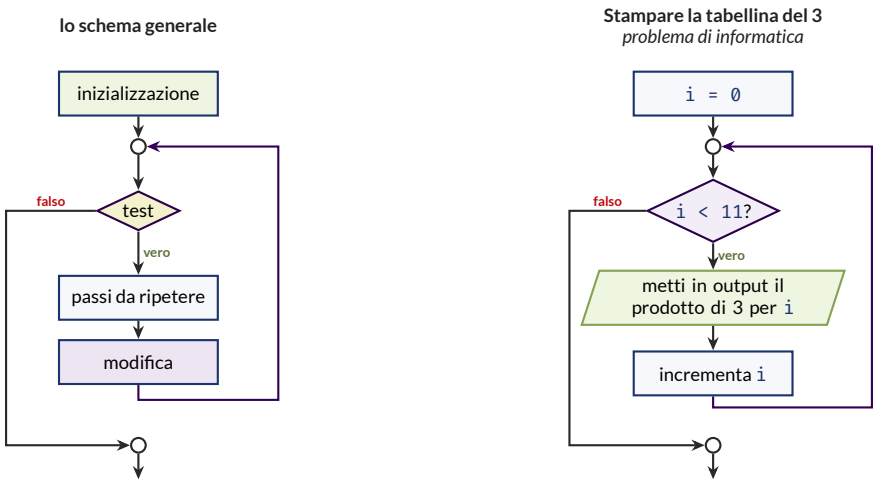


Figura 4.9: *Il ciclo su contatore: lo schema e un esempio.*

Seguiamo l'esecuzione della tabellina, scrivendo a ogni giro il valore di `i`, l'esito del test e l'output. È la tecnica più utile per capire un ciclo (e per trovarne gli errori): **simularlo a mano**, una riga per giro.

valore di `i`	0	1	2	3	4	5	6	7	8	9	10	11
`i < 11`?	vero	vero	vero	vero	vero	vero	vero	vero	vero	vero	vero	**falso**
output	0	3	6	9	12	15	18	21	24	27	30	—

Il corpo viene eseguito 11 volte (per `i` da 0 a 10), e il test viene valutato 12 volte: l'ultima, con `i` che vale 11, è quella che fa uscire dal ciclo.

Il ciclo su condizione

Se non sappiamo a priori **quante** volte dobbiamo eseguire il ciclo, sicuramente sappiamo **quando** eseguirlo e quando no: serve allora un **ciclo su condizione**, che si ripete finché una condizione è vera. Anche qui ci sono due casi, a seconda di quando si fa il test:

- prima eseguiamo il ciclo e poi facciamo il test (**condizione in coda**);
- oppure prima facciamo il test e poi eseguiamo il ciclo (**condizione in testa**).

La differenza sembra piccola, ma non lo è: con la condizione in coda il corpo del ciclo viene eseguito **almeno una volta**; con la condizione in testa può essere eseguito **zero volte**.

Condizione in coda

Si pensi a mangiare un piatto di spaghetti: si mangia una forchettata, poi ci si chiede se ce n'è ancora; se sì, si mangia un'altra forchettata, e così via. Chiedersi "ce n'è ancora?" ha senso solo **dopo** aver cominciato (figura 4.10, a sinistra).

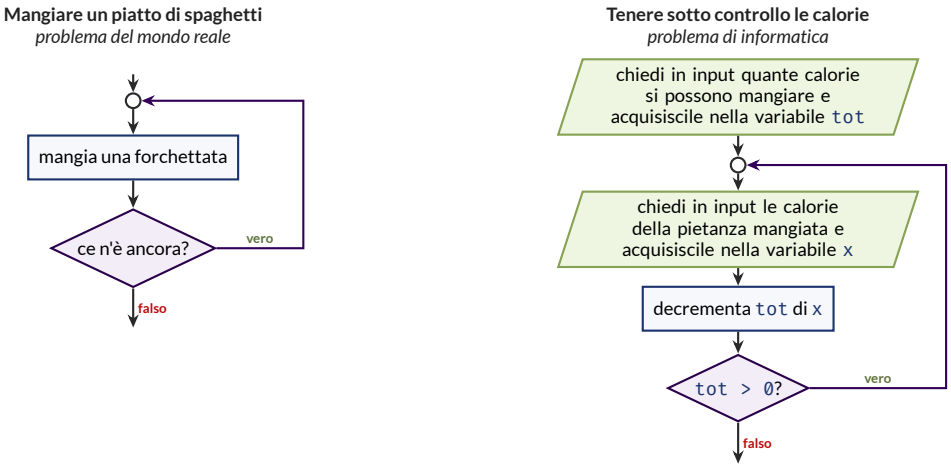


Figura 4.10: *Cicli con la condizione in coda: il corpo viene eseguito almeno una volta.*

Il ciclo su condizione con verifica in coda si presta a due casi:

- quando ha senso porsi la domanda **solo dopo** aver già eseguito una prima volta il ciclo, come per gli spaghetti e le calorie;
- quando c'è un obiettivo da raggiungere mediante uno o più **tentativi**: si tenta, e si ripete finché il tentativo non riesce (figura 4.11).

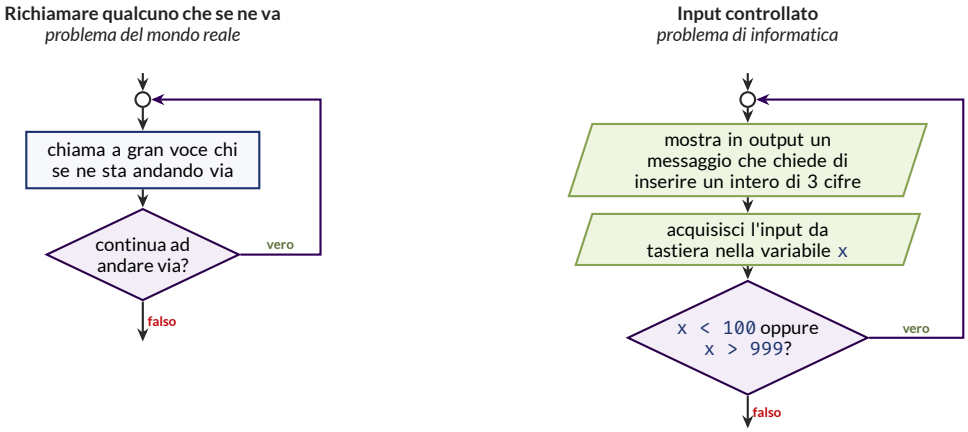


Figura 4.11: *Cicli con la condizione in coda per raggiungere un obiettivo con più tentativi.*

L'input controllato è un caso che ritroveremo spesso: un programma non può fidarsi di ciò che scrive l'utente. Chiede un numero di tre cifre e, finché il numero ricevuto è fuori dall'intervallo da 100 a 999, lo richiede.

Quando invece ha senso porsi la domanda **prima** di eseguire la prima volta il ciclo, allora si pone la verifica della condizione in testa: è il **ciclo su condizione con verifica in testa**. Per alzare tutte le tapparelle di casa ci si chiede prima se ce ne sono ancora da alzare: potrebbero essere già tutte aperte, e allora non si fa nulla (figura 4.12). Si noti che il ciclo su contatore visto prima è un caso particolare di questo schema, con il test in testa.

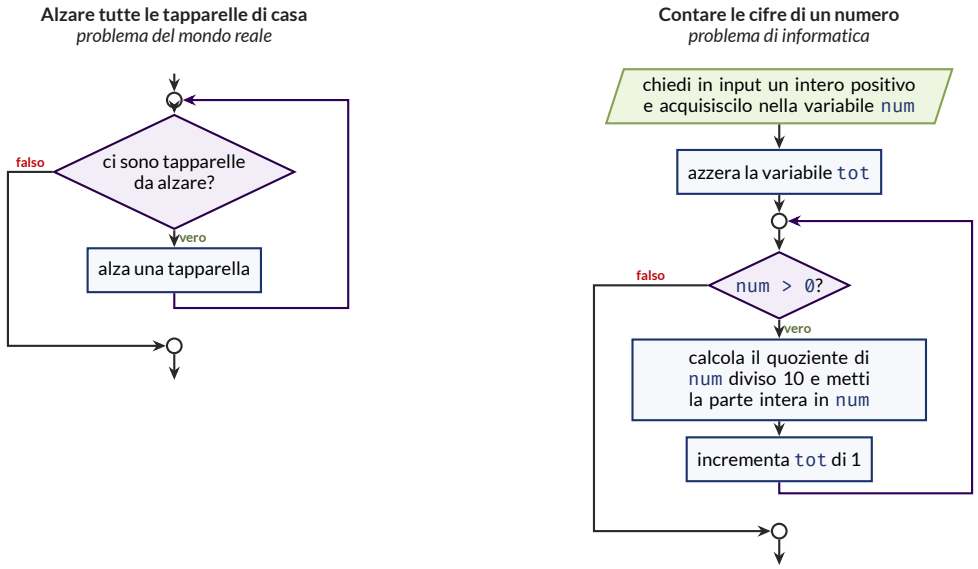


Figura 4.12: *Cicli con la condizione in testa: il corpo può non essere mai eseguito.*

Perché il secondo algoritmo conta le cifre? Dividere per 10 un numero intero, tenendo solo la parte intera, **toglie l'ultima cifra**: da 4072 si passa a 407. Quindi le cifre sono tante quante le divisioni che servono per arrivare a 0. Simuliamo con `num` che vale 4072:

giro	`num` prima	`num > 0`?	dopo il giro
1	4072	vero	`num` = 407, `tot` = 1
2	407	vero	`num` = 40, `tot` = 2
3	40	vero	`num` = 4, `tot` = 3
4	4	vero	`num` = 0, `tot` = 4
5	0	**falso**	si esce: le cifre sono **4**

L'algoritmo chiede un intero **positivo**, ed è importante: con `num` uguale a 0 il test sarebbe subito falso e `tot` resterebbe 0, mentre il numero 0 ha una cifra. Ancora una volta, i casi limite.

Combinare le strutture

Le strutture di controllo possono essere **combinate** tra loro: ovunque un diagramma contiene un passo, quel passo può essere sostituito da un'intera struttura, perché ogni struttura ha una sola entrata e una sola uscita, esattamente come un passo.

Riprendiamo la telefonata. In realtà, il passo "componi il numero" non è esattamente un passo elementare, in quanto è scomponibile: si digita una cifra, e si continua finché ci sono ancora cifre (un ciclo con la condizione in coda). Sostituendo il passo con il suo ciclo si ottiene il diagramma della figura 4.13.

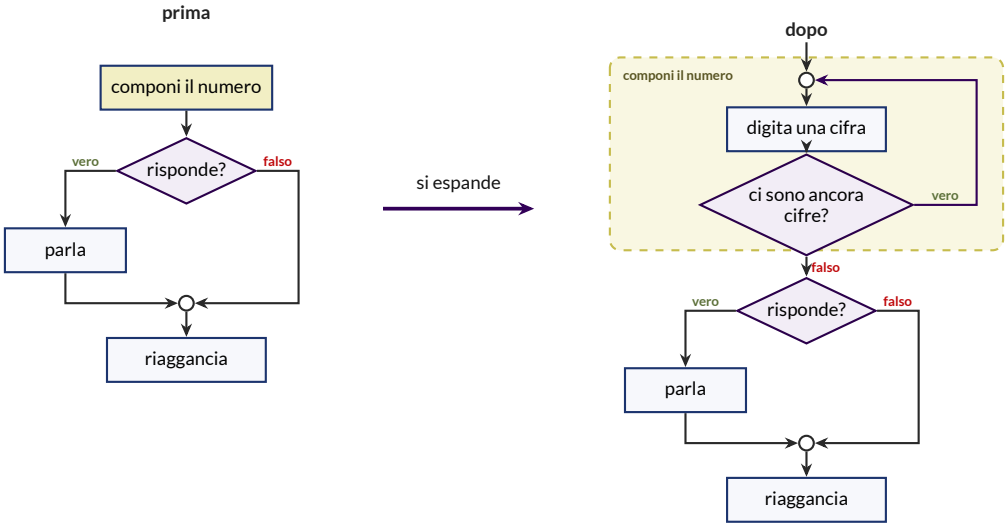


Figura 4.13: *Il passo "componi il numero" sostituito dal ciclo che lo realizza: una selezione che segue un ciclo.*

Questo principio, sostituire un passo con la struttura che lo realizza, è alla base dell'approccio **top-down**.

Lo sviluppo top-down

Dividere il problema

Per **sviluppo top-down** ("dall'alto verso il basso") si intende un approccio alla soluzione di un problema che prevede la sua **scomposizione in sotto-problemi**, più piccoli e quindi più semplici da risolvere. Ogni sotto-problema, a sua volta, si può scomporre ancora, finché non si arriva a passi abbastanza semplici da scrivere direttamente.

Ogni sotto-problema sarà risolto con la realizzazione di un **sotto-programma** (in C, una *funzione*), e uno dei passi elementari del nostro algoritmo potrebbe essere proprio l'esecuzione di uno dei sotto-programmi. Lo svolgimento di un compito ben preciso (anche piccolo) può avvenire in autonomia, ma spesso il sotto-programma ha bisogno di scambiare dei dati con chi lo usa. Per esempio, "componi il numero" richiede che gli si fornisca il numero da comporre.

- I dati che un sotto-programma riceve da chi lo invoca si chiamano **parametri**.
- Il dato che un sotto-programma fornisce in output a chi lo ha invocato si chiama **valore restituito**.

Un'analogia: in un libro di cucina, la ricetta delle lasagne dice "prepara un litro di besciamella (vedi pagina 112)". La ricetta della besciamella è un sotto-programma; "un litro" è il parametro, e la besciamella pronta è il valore restituito. La ricetta delle lasagne non deve sapere come si fa la besciamella: le basta sapere che cosa darle e che cosa riceverà.

Come si invoca un sotto-programma

Supponiamo di avere un programma (che questo sia il programma principale o un sotto-programma non ha alcuna importanza) e un sotto-programma qualsiasi. Quando progettiamo il sotto-programma dobbiamo, fra l'altro, indicare i parametri di cui necessita: sono i **parametri formali**, i nomi con cui il sotto-programma si riferisce ai dati ricevuti. Supponiamo che siano `a`, `b`, `c` e `d`.

L'**invocazione** (da parte del programma chiamante) avviene indicando il nome del sotto-programma, fornendo i parametri con cui lavorare (i **parametri attuali**) e indicando che cosa fare del valore restituito. La figura 4.14 segue l'invocazione passo per passo.

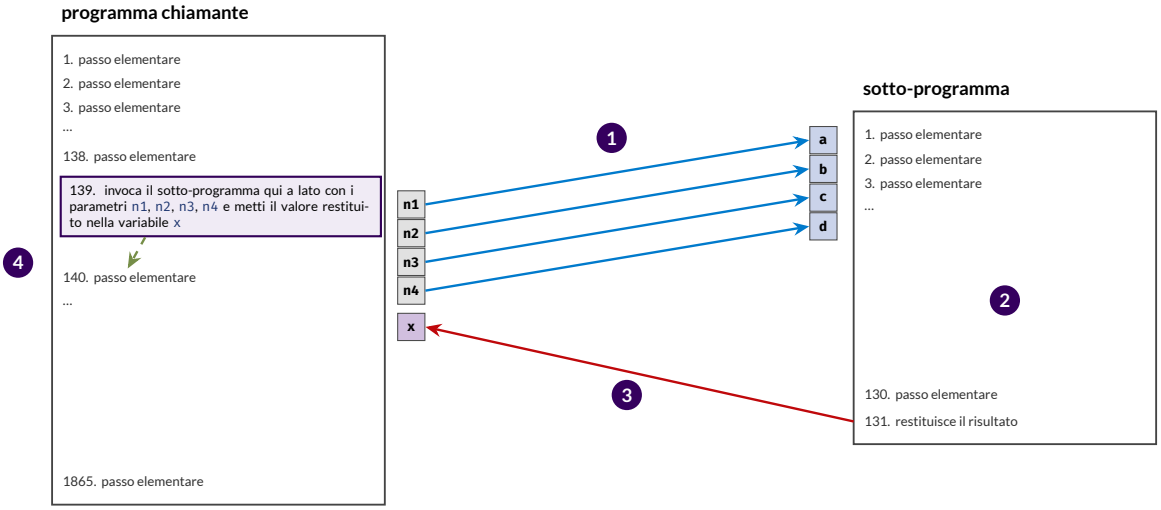


Figura 4.14: *L'invocazione di un sotto-programma con quattro parametri.*

1. All'invocazione, il controllo del flusso passa al sotto-programma, e ogni parametro formale riceve una **copia** del valore del parametro attuale corrispondente: in `a` viene copiato il valore di `n1`, in `b` quello di `n2`, e così via.
2. Il sotto-programma esegue i suoi passi, lavorando con `a`, `b`, `c` e `d`.
3. Quando ha eseguito la sua ultima istruzione, fornisce al programma chiamante il suo **valore di restituzione**, che finisce in `x`…
4. …e l'esecuzione riprende dall'istruzione successiva a quella dell'invocazione (la 140) del programma chiamante.

Due osservazioni importanti. I nomi dei parametri attuali e formali **possono anche non coincidere** (qui `n1` e `a`): il sotto-programma è scritto una volta sola e deve funzionare con qualunque dato gli venga passato, quindi usa nomi suoi. Invece devono **assolutamente coincidere** il numero, il **tipo** di ogni parametro e l'**ordine**: il primo parametro attuale va nel primo formale, il secondo nel secondo. Scambiare l'ordine non produce un errore visibile, produce un risultato sbagliato.

I TIPI DI ISTRUZIONE

Riassumendo, ogni passo elementare di un algoritmo sarà un'istruzione di uno di questi tipi: **input**, **output**, **assegnazione** (mettere un valore in una variabile), **selezione**, **iterazione**, **invocazione di funzione**. Selezione e iterazione si dividono a loro volta come nella figura 4.15.

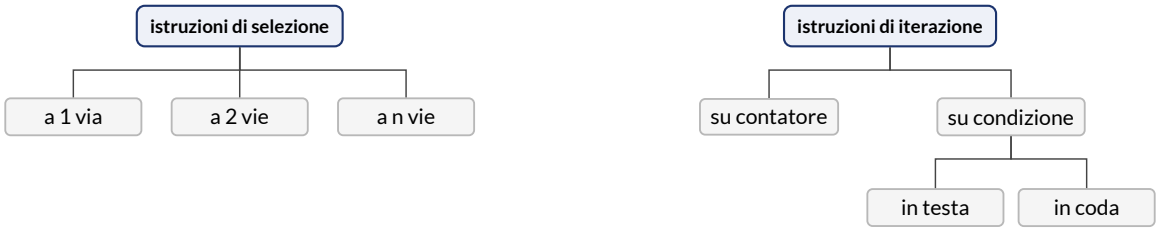


Figura 4.15: *I tipi di istruzioni di selezione e di iterazione.*

Ognuno di questi concetti diventerà un costrutto preciso del C. La tabella seguente anticipa la corrispondenza, per avere fin d'ora una mappa del percorso: non serve ricordarla, la ritroveremo pezzo per pezzo.

istruzione		**in C**	**capitolo**
input		`scanf, getchar`	5, ??
output		`printf, putchar`	5, ??
assegnazione		`x = valore;`	5
selezione	a 1 via	`if`	5, ??
	a 2 vie	`if … else`	5, ??
	a n vie	`switch`	??
iterazione	su contatore	`for`	5, ??
	su condizione, in testa	`while`	5, ??
	su condizione, in coda	`do … while`	??
invocazione di funzione		`nome(parametri)`	??

In sintesi

Il C nasce nel 1972 per scrivere Unix, come evoluzione tipizzata del B; lo standard (ANSI e ISO, dal 1989–90) ne garantisce portabilità, affidabilità e manutenibilità. La programmazione strutturata rinuncia ai salti arbitrari: per il teorema di Böhm-Jacopini bastano **sequenza**, **selezione** (a una, due o n vie) e **iterazione** (su contatore, o su condizione con il test in testa o in coda). Ogni struttura ha una sola entrata e una sola uscita, quindi le strutture si annidano come scatole; sostituire un passo con la struttura che lo realizza è l'idea dello sviluppo **top-down**, in cui i sotto-problemi diventano sotto-programmi che ricevono **parametri** (copiati, nell'ordine) e restituiscono un **valore**.

Riferimenti

- **Kernighan & Ritchie**, *Il linguaggio C*: prefazione e introduzione (storia del linguaggio).
- **E. W. Dijkstra**, *Go To Statement Considered Harmful*, Communications of the ACM, 1968.
- **C. Böhm, G. Jacopini**, *Flow diagrams, Turing machines and languages with only two formation rules*, Communications of the ACM, 1966.

Primi programmi in C

Questo capitolo è una panoramica del C attraverso una serie di programmi di esempio, ciascuno spiegato elemento per elemento. Si introducono brevemente i costrutti principali (l'output, le variabili, l'input, le scelte, i cicli, le funzioni) per avere subito un'idea di come è fatto un programma vero; ogni argomento verrà poi approfondito nei capitoli successivi. Il consiglio è uno solo: **scrivere ed eseguire ogni esempio**, poi modificarlo e osservare che cosa cambia.

Il primo programma, riga per riga

Scrivere, compilare, eseguire

Un programma deve comunicare per essere utile: produce sempre un output di qualche tipo. Il primo esempio stampa una frase sullo schermo. Lo scriviamo nel file `sea.c`, lo compiliamo e lo eseguiamo, come abbiamo imparato nel capitolo 1:

sea.c

```c
#include <stdio.h>

int main(void)
{
    printf("from sea to shining C\n");
    return 0;
}
```

Terminale

```
$ gcc sea.c
$ ./a.out
from sea to shining C
```

Se non ci sono errori, gcc crea il file eseguibile `a.out`.

Sono sette righe, ma ognuna ha un ruolo preciso. La figura 5.1 le numera, e l'elenco che la segue le spiega.

```c
#include <stdio.h>                      1
int main(void)                          2
{                                       3
    printf("from sea to shining C\n");   4
    return 0;                           5
}                                       3
```

Figura 5.1: *Anatomia del primo programma: i numeri rimandano all'elenco seguente.*

1. **Direttiva per il preprocessore**: copia qui il contenuto del file di intestazione `stdio.h`, che descrive `printf`.
2. **Testa della funzione** `main`, da cui comincia l'esecuzione: restituisce un `int` e non riceve parametri (`void`).
3. Le **graffe** delimitano il **corpo** della funzione.
4. Un'**istruzione**: chiama `printf` con un argomento, una **stringa costante**; `\n` va a capo; il `;` chiude l'istruzione.
5. Restituisce 0 al sistema operativo: "tutto bene".

Il preprocessore e `#include`

Prima della compilazione vera e propria viene eseguito, automaticamente, il **preprocessore** (capitolo 3). Le righe che cominciano con `#` sono **direttive** per il preprocessore. La direttiva `#include <stdio.h>` chiede di includere in quel punto una copia del file di intestazione (*header file*) `stdio.h`, che contiene le informazioni necessarie per usare le funzioni di input e output, come `printf`.

`#include` non include la libreria

A lezione il docente è categorico: guai a dire che `#include` "include la libreria". Include un **file di testo**, l'header, che **descrive** le funzioni (come si chiamano, che parametri vogliono, che cosa restituiscono). Il **codice** delle funzioni sta nella libreria, che viene collegata dal **linker**, dopo. Sono due cose diverse, che intervengono in due fasi diverse: lo vedremo in modo molto concreto con la funzione `pow` più avanti in questo capitolo.

La funzione `main`

`int main(void)` è la prima riga della definizione di una **funzione** chiamata `main`. Le parentesi dopo il nome dicono che `main` è una funzione; `int` e `void` sono parole speciali che danno informazioni al compilatore: `void` fra le parentesi significa che la funzione **non ha parametri**, `int` davanti al nome significa che **restituisce un intero**.

Ogni programma C ha **una** funzione `main`, e l'esecuzione comincia da lei. Il corpo della funzione è racchiuso fra le graffe `{` e `}`, che in C servono in generale a raggruppare istruzioni.

`printf`, le stringhe e il punto e virgola

`printf` è una funzione che stampa sullo schermo (più precisamente sullo *standard output*, come vedremo). Non fa parte del linguaggio: appartiene alla **libreria standard**, un insieme di librerie che forniscono al programmatore funzioni pronte; gli header file permettono di usarle, e `stdio.h` fornisce le informazioni per `printf`.

Il testo fra virgolette, `"from sea to shining C\n"`, è una **stringa costante**: è l'**argomento** di `printf` e stabilisce che cosa viene stampato. Al suo interno, `\n` (una barra rovesciata seguita da `n`) è un **singolo carattere**, il *newline*: fa avanzare il cursore all'inizio della riga successiva. L'intera riga è un'**istruzione**, e in C le istruzioni **terminano con un punto e virgola**.

`return` e il valore restituito al sistema

`return 0;` termina la funzione `main` e restituisce il valore 0 al sistema operativo. Per convenzione 0 significa "tutto è andato bene", un valore diverso da 0 segnala un problema. Il valore non si vede nell'output, ma la shell lo conserva, e lo si può leggere con il comando `echo $?` subito dopo l'esecuzione:

Terminale

```
$ ./a.out
from sea to shining C
$ echo $?
0
```

`return` **termina** la funzione: ciò che viene dopo non è eseguito. Il file `sea.c` distribuito con le lezioni ne dà una prova, perché dopo il `return` contiene un'altra `printf`:

sea.c

```c
#include <stdio.h>

int main()
{
    printf("from sea to shining C\n");

    return 0;
    printf("Pippo\n");
}
```

Terminale

```
$ gcc -o sea sea.c
$ ./sea
from sea to shining C
```

"Pippo" non viene mai stampato: l'esecuzione esce da `main` al `return`.

(Qui `main` è scritto con le parentesi vuote, `main()`: per una definizione di funzione è equivalente a `main(void)`; in questa dispensa useremo la forma con `void`, più esplicita.)

L'OUTPUT CON `printf`

Il cursore

`printf` scrive sullo schermo da sinistra a destra e dall'alto verso il basso, come si scrive su un quaderno; va a capo **solo** quando incontra il carattere `\n`. Ogni chiamata di `printf` comincia da dove la precedente ha lasciato il cursore. Per questo il programma seguente, diverso dal primo, stampa **lo stesso** output:

```c
printf("from sea to ");
printf("shining ");
printf("C\n");
```

Terminale

```
from sea to shining C
```

Se vogliamo scrivere su tre righe dobbiamo inserire dei `\n`; e se ne possono mettere più di uno nella stessa stringa:

```c
printf("from sea to\n");
printf("shining\nC\n");
```

Terminale

```
from sea to
shining
C
```

Disegnare con i caratteri

Per essere leggibile, l'output va spaziato opportunamente sullo schermo. Il programma seguente stampa la stessa frase in una cornice di asterischi, preceduta e seguita da dieci righe vuote (ciascun `\n` iniziale produce una riga vuota):

sea_box.c

```c
#include <stdio.h>

int main(void)
{
    printf("\n\n\n\n\n\n\n\n\n\n");
    printf("      ************************\n");
    printf("      *  from sea            *\n");
    printf("      *  to shining C        *\n");
    printf("      ************************\n");
    printf("\n\n\n\n\n\n\n\n\n\n");
    return 0;
}
```

Terminale

```
$ ./sea_box


      ************************
      *  from sea            *
      *  to shining C        *
      ************************


$
```

Qui sono mostrate solo due delle dieci righe vuote sopra e sotto.

VARIABILI, ESPRESSIONI E ASSEGNAZIONE

Il problema della maratona

Il secondo esempio converte una distanza: la lunghezza di una maratona è di 26 miglia e 385 iarde, e vogliamo sapere quanti chilometri sono. Servono due regole: in un miglio ci sono 1760 iarde, e un miglio è lungo 1,609 km.

marathon.c

```c
/* The distance of a marathon in kilometers. */

#include <stdio.h>

int main(void)
{
    int     miles, yards;
    float   kilometers;

    miles = 26;
    yards = 385;
    kilometers = 1.609 * (miles + yards / 1760.0);
    printf("\nA marathon is %f kilometers.\n\n", kilometers);
    return 0;
}
```

Terminale

```
$ gcc -o marathon marathon.c
$ ./marathon

A marathon is 42.185970 kilometers.
```

L'opzione `-o` sceglie il nome del file eseguibile.

I commenti

Tutto ciò che è compreso fra `/*` e `*/` è un **commento**: viene ignorato dal compilatore (anzi, eliminato già dal preprocessore) e serve solo a chi legge. Si possono anche scrivere commenti su una sola riga, che cominciano con `//` e arrivano fino alla fine della riga. Un commento all'inizio del file spiega lo scopo del programma; vi si possono aggiungere informazioni sugli autori e sulle versioni.

Le dichiarazioni

```
int     miles, yards;
float   kilometers;
```

Queste righe sono **dichiarazioni**. `int` e `float` sono **parole chiave** del C che indicano due **tipi di dato fondamentali**: la prima riga dice al compilatore che `miles` e `yards` sono **variabili** di tipo intero, che assumono valori interi; la seconda che `kilometers` è una variabile di tipo `float`, che assume valori reali. Anche le dichiarazioni terminano con un punto e virgola.

Dichiarare una variabile significa chiedere al compilatore di riservarle dello spazio in memoria e di ricordarne il tipo. In C **ogni variabile va dichiarata prima di essere usata**.

L'assegnazione

`miles = 26;` è un'**istruzione di assegnazione**: il simbolo `=` è l'**operatore di assegnazione**, `26` è una costante intera, e l'effetto è che il valore 26 viene scritto nella variabile `miles` (figura 5.2). Allo stesso modo 385 viene assegnato a `yards`.

= non è l'uguale della matematica

In matematica $x = x + 1$ è un'equazione impossibile. In C `x = x + 1;` è perfettamente sensato e significa: **calcola** il valore dell'espressione a destra (il valore attuale di `x` più 1) e **scrivilo** nella variabile a sinistra. L'assegnazione va letta da destra a sinistra: "`x` riceve…".

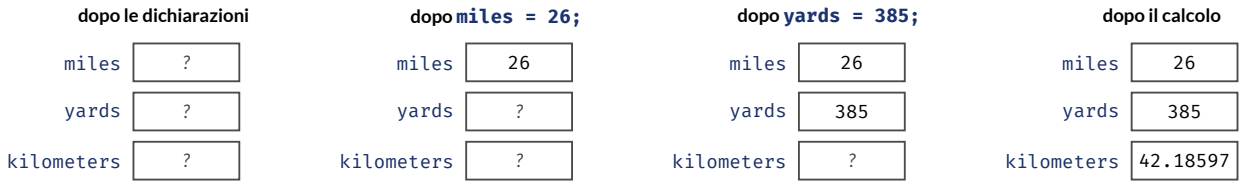


Figura 5.2: *Le variabili della maratona, istruzione dopo istruzione. Appena dichiarata, una variabile contiene un valore a caso ("?"): finché non le si assegna qualcosa, non va usata.*

Come si calcola un'espressione

`kilometers = 1.609 * (miles + yards / 1760.0);` assegna un valore calcolato da un'**espressione**. Le regole sono quelle dell'aritmetica: si calcola prima ciò che sta fra parentesi, e dentro le parentesi la divisione ha la precedenza sulla somma. Quindi prima `yards / 1760.0`, poi il risultato viene sommato a `miles`, e infine moltiplicato per `1.609` (figura 5.3).

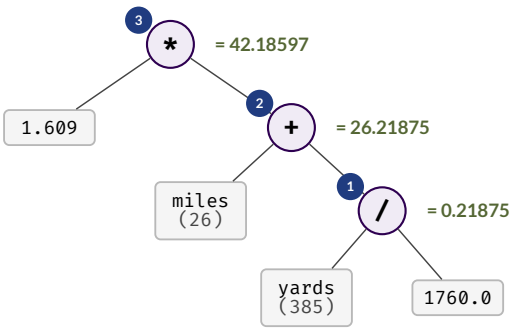


Figura 5.3: *L'albero dell'espressione: si calcola dal basso verso l'alto, nell'ordine indicato dai numeri.*

`1.609` e `1760.0` sono **costanti reali** (in C, di tipo `double`): per questo `yards / 1760.0` è una divisione fra reali, con risultato reale. Infine `printf("…%f…", kilometers)` stampa il valore: la stringa di controllo contiene una **specifica di conversione**, `%f`, che dice "qui va stampato un numero reale"; la variabile corrispondente, il secondo argomento, deve essere di quel tipo.

La trappola della divisione intera

Basta togliere il `.0` alla costante per cambiare completamente il risultato:

```
kilometers = 1.609 * (miles + yards / 1760);
```

```
Terminale
A marathon is 41.834000 kilometers.
```

Ora `1760` è una costante **intera**, e `yards / 1760` è una divisione fra interi: in C il risultato di una divisione fra interi è un intero, e il resto si perde. 385 diviso 1760 fa 0 con resto 385, quindi `yards / 1760` vale **0** e i 385 metri e rotti… spariscono: $1{,}609 \cdot 26 = 41{,}834$. Il compilatore non segnala niente, perché non c'è nessun errore di linguaggio: c'è un errore di ragionamento.

Parole chiave e nomi



Parole come `int` e `float` sono **parole chiave** (*keywords*): sono **riservate** e non possono essere usate come nomi di variabili. Una variabile non può chiamarsi `int`, né `return`. Per lo stesso motivo di chiarezza, non si usano come nomi di variabili i nomi delle funzioni: niente variabili chiamate `printf` o `main`. L'elenco completo delle parole chiave e le regole per i nomi sono nel capitolo 6.

Il preprocessore: `#define` e `#include`



Le costanti simboliche

Il preprocessore scandisce tutto il file e individua le direttive, cioè le righe che cominciano con `#`. Due delle più usate sono `#define` e `#include`. Consideriamo le righe

```
#define LIMIT 100
#define PI    3.14159
```

Il preprocessore sostituirà nel file sorgente tutte le occorrenze di `LIMIT` con `100` e tutte quelle di `PI` con `3.14159`, tranne quelle che compaiono fra virgolette (nelle stringhe) e nei commenti. Un `#define` può stare ovunque nel file e ha effetto solo sulle righe successive; di solito si mette all'inizio, e per convenzione il nome si scrive in maiuscolo.

Le **costanti simboliche** rendono il programma più **leggibile** (`LIMIT` dice che cosa rappresenta, `100` no) e più facile e sicuro da **modificare**. Se la velocità della luce, circa 299 792,458 km/s, compare in cento punti di un programma, scrivendo

```
#define C 299792.458   /* velocità della luce in km/s */
```

e usando `C` ovunque, una stima più accurata si inserisce cambiando una sola riga, senza rischiare di dimenticare qualche occorrenza.

I file di intestazione

Una riga come

```
#include "my_file.h"
```

copia il contenuto di `my_file.h` in quel punto del sorgente. Anche `#include` può stare ovunque, ma tipicamente si mette all'inizio. I file che si includono sono gli **header file** e per convenzione hanno estensione `.h`. Il sistema C ne fornisce alcuni, come `stdio.h`, `stdlib.h`, `string.h`, `math.h`. La differenza fra le due forme:

- `#include <stdio.h>`, con le parentesi angolari, cerca il file fra gli header del sistema;
- `#include "my_file.h"`, con le virgolette, lo cerca prima nella cartella del sorgente: si usa per i nostri header.

Ogni volta che si usa una funzione di input/output come `printf` si deve includere `stdio.h`, che ne contiene il **prototipo**: la descrizione che serve al compilatore per controllare che la chiamata sia corretta. L'header serve alla **compilazione**; il codice della funzione arriva dalla libreria, durante il **linking**.

Header e libreria: la prova

La funzione `pow(x, y)` calcola x^y; il suo prototipo sta in `math.h`, il suo codice nella **libreria matematica**, che su Linux è separata dalla libreria standard e va chiesta esplicitamente al linker con l'opzione `-lm` (*library m*):

potenza.c

```c
#include <stdio.h>
#include <math.h>

int main(void)
{
    double base, esponente;

    printf("Base ed esponente: ");
    scanf("%lf%lf", &base, &esponente);
    printf("%g elevato a %g fa %g\n",
           base, esponente, pow(base, esponente));
    return 0;
}
```

Terminale

```
$ gcc -Wall -o potenza potenza.c
/usr/bin/ld: /tmp/ccVCbC9C.o: in function `main':
potenza.c:(.text+0x61): undefined reference to `pow'
collect2: error: ld returned 1 exit status
$ gcc -Wall -o potenza potenza.c -lm
$ ./potenza
Base ed esponente: 2 10
2 elevato a 10 fa 1024
```

Il primo tentativo fallisce, ma si guardi **chi** protesta: `ld`, il linker, non il compilatore. Grazie a `math.h` il compilatore sapeva tutto ciò che gli serviva e ha prodotto il file oggetto; è il linker che non ha trovato il **codice** di `pow` (*undefined reference*). Con `-lm` gli si indica la libreria, e tutto funziona. (Su macOS la libreria matematica è già compresa in quella di sistema e `-lm` non serve, ma non fa danni.)

Un esempio con le costanti: Pacific Sea

Il programma

Pacific Sea è il nome dato alla parte di oceano Pacifico visibile dal campus di Santa Cruz, sulla baia di Monterey. Il programma converte la sua area da chilometri quadrati in acri, miglia quadrate, piedi quadrati e pollici quadrati. Le costanti stanno in un header scritto da noi:

pacific_sea.h

```h
#include <stdio.h>

#define   AREA                        2337
#define   SQ_MILES_PER_SQ_KILOMETER   0.3861021585424458
#define   SQ_FEET_PER_SQ_MILE         (5280 * 5280)
#define   SQ_INCHES_PER_SQ_FOOT       144
#define   ACRES_PER_SQ_MILE           640
```

pacific_sea.c

```c
/* Measuring the Pacific Sea. */

#include "pacific_sea.h"

int main(void)
{
    const int   pacific_sea = AREA;     /* in sq kilometers */
    double      acres, sq_miles, sq_feet, sq_inches;

    printf("\nThe Pacific Sea covers an area");
    printf(" of %d square kilometers.\n", pacific_sea);
    sq_miles = SQ_MILES_PER_SQ_KILOMETER * pacific_sea;
    sq_feet = SQ_FEET_PER_SQ_MILE * sq_miles;
    sq_inches = SQ_INCHES_PER_SQ_FOOT * sq_feet;
    acres = ACRES_PER_SQ_MILE * sq_miles;
    printf("In other units of measure this is:\n\n");
    printf("%22.7e acres\n", acres);
    printf("%22.7e square miles\n", sq_miles);
    printf("%22.7e square feet\n", sq_feet);
    printf("%22.7e square inches\n\n", sq_inches);
    return 0;
}
```

Terminale

```
$ ./pacific_sea

The Pacific Sea covers an area of 2337 square kilometers.
In other units of measure this is:

         5.7748528e+05 acres
         9.0232074e+02 square miles
         2.5155259e+10 square feet
         3.6223572e+12 square inches
```

Gli `#include` a catena

La prima riga include una copia di `pacific_sea.h`, e siccome questo contiene a sua volta `#include <stdio.h>`, il preprocessore include anche una copia di `stdio.h`, che serve per `printf` (figura 5.4). Il risultato è un unico testo, che è quello che il compilatore riceve.

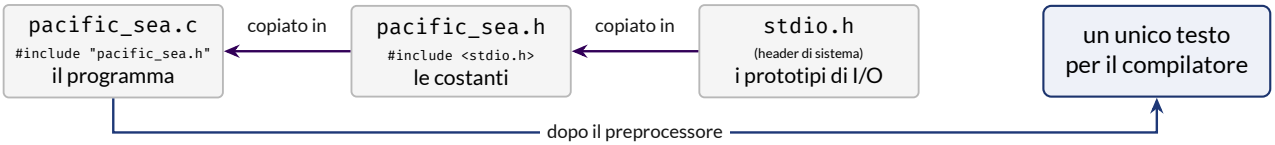


Figura 5.4: *Il preprocessore risolve gli* `#include` *uno dentro l'altro.*

Le costanti dell'header

Le costanti simboliche dell'header (una reale, le altre intere) rendono il codice più chiaro: se ci fosse una nuova stima dell'area basterebbe cambiare `AREA`, e le altre danno un nome significativo ai numeri. Su `SQ_FEET_PER_SQ_MILE` vale la pena fermarsi:

- scrivere `(5280 * 5280)` invece di `27878400` è più chiaro: dice che in un miglio ci sono 5280 piedi, e che un miglio quadrato ne contiene il quadrato. Non si perde efficienza: il compilatore calcola da sé le espressioni fatte solo di costanti;
- le parentesi non sono sintatticamente necessarie, ma sono un'ottima abitudine. Il preprocessore fa una sostituzione di **testo**, senza capire nulla: se fosse `#define SQ_FEET_PER_SQ_MILE 5280 * 5280`, l'espressione `x / SQ_FEET_PER_SQ_MILE` diventerebbe `x / 5280 * 5280`, cioè (divisione e moltiplicazione si eseguono da sinistra a destra) $(x/5280) \cdot 5280$, che vale di nuovo circa x. Con le parentesi diventa `x / (5280 * 5280)`, come si voleva.

`const`

`const int pacific_sea = AREA;` è la dichiarazione di una variabile `pacific_sea` di tipo `int`, **inizializzata** (cioè con un valore assegnato già nella dichiarazione) a 2337: prima della compilazione il preprocessore ha sostituito `AREA` con il suo valore. La parola chiave `const` qualifica il tipo: la variabile riceve il suo valore all'inizializzazione, ma **non può essere modificata**. Il compilatore lo fa rispettare:

costante.c

```c
#include <stdio.h>

int main(void)
{
    const int pacific_sea = 2337;

    pacific_sea = 3000;
    printf("%d\n", pacific_sea);
    return 0;
}
```

Terminale

```
$ gcc -Wall -o costante costante.c
costante.c: In function ‘main’:
costante.c:7:17: error: assignment of read-only
    variable ‘pacific_sea’
    7 |     pacific_sea = 3000;
      |                 ^
```

I tipi reali e la notazione scientifica

`acres`, `sq_miles` e gli altri sono dichiarati `double`. In C i tipi reali (detti *floating point*, "a virgola mobile") sono tre: `float` (tipicamente 6 cifre significative), `double` (tipicamente 15) e `long double` (almeno quanto `double`; sui PC di solito 18). I valori dipendono dall'implementazione; il capitolo **??** li approfondisce.

L'istruzione `printf("%22.7e acres\n", acres);` stampa la riga `5.7748528e+05 acres`. Il numero è in **notazione scientifica**: `e+05` significa "per 10^5", quindi il valore è $5{,}7748528 \cdot 10^5 = 577\,485{,}28$ acri. La conversione `%e` chiede proprio questo formato (detto *e-format*), e la forma `%`m`.`n`e`, con m e n interi positivi, indica che il numero occupa in tutto m caratteri (allineato a destra) con n cifre dopo il punto: qui 22 caratteri e 7 cifre decimali. Per questo nell'output i numeri sono preceduti da spazi e allineati in colonna.

printf E scanf

Funzioni di input e output formattato

`printf` si usa per l'output, `scanf` per l'input; la `f` finale sta per *formatted* (formattato). Non fanno parte del linguaggio, ma della libreria standard, che viene collegata implicitamente alla fine della compilazione. Il programmatore deve comunque dichiarare al compilatore le funzioni che usa: il meccanismo dell'ANSI C è il **prototipo**, e i prototipi delle funzioni della libreria standard sono negli header standard (per `printf` e `scanf`, `stdio.h`).

Entrambe accettano una lista di argomenti:

- la **stringa di controllo**, una stringa che contiene le **specifiche di conversione**: ciascuna è il carattere `%` seguito da un **carattere di conversione** (per esempio `%d` per un intero);
- gli altri argomenti, **uno per ogni conversione** della stringa di controllo, nello stesso ordine.

Le conversioni di `printf`

Per stampare le prime tre lettere dell'alfabeto ci sono molti modi: senza conversioni, con tre conversioni per caratteri, con una conversione per stringhe. Si può anche decidere quanto spazio dare a ciascuna conversione: `%3c` significa "un carattere, in un campo largo 3", allineato a destra.

```c
printf("abc\n");
printf("%c%c%c\n", 'a', 'b', 'c');
printf("%s\n", "abc");
printf("%c%3c%5c\n", 'A', 'B', 'C');
```

Terminale

```
abc
abc
abc
A  B    C
```

Nel codice, `'a'` fra apici singoli è una **costante carattere** (un solo carattere), mentre `"abc"` fra virgolette è una **stringa**. I caratteri di conversione principali di `printf` sono:

conversione	stampa	esempio
`%c`	un carattere	`printf("%c", 'A')` → `A`
`%d`	un intero, in decimale	`printf("%d", 42)` → `42`
`%e`	un reale, in notazione scientifica	`printf("%e", 314159.0)` → `3.141590e+05`
`%f`	un reale	`printf("%f", 3.14159)` → `3.141590`
`%g`	un reale, nel più corto fra `%e` e `%f`	`printf("%g", 314159.0)` → `314159`
`%s`	una stringa	`printf("%s", "abc")` → `abc`

Fra `%` e il carattere di conversione si possono indicare l'ampiezza del campo e, per i reali, il numero di cifre decimali (la *precisione*); un segno meno allinea a sinistra. Le parentesi quadre nell'esempio servono solo a mostrare dove comincia e finisce il campo:

```c
printf("[%d] [%5d] [%-5d]\n", 42, 42, 42);
printf("[%f] [%.2f] [%10.3f]\n", 3.14159, 3.14159, 3.14159);
printf("[%e] [%g] [%g]\n", 314159.0, 314159.0, 0.0000314);
```

Terminale

```
[42] [   42] [42   ]
[3.141590] [3.14] [     3.142]
[3.141590e+05] [314159] [3.14e-05]
```

Si noti che `%f` stampa sempre 6 cifre decimali, se non si indica altro, e che la precisione **arrotonda**: 3.14159 con 3 decimali diventa 3.142.

scanf e l'operatore &

scanf è l'analoga di printf per l'input. Il primo parametro è la stringa di controllo; l'input è sempre una sequenza di caratteri (ciò che si digita sulla tastiera), e le conversioni specificano come i caratteri devono essere interpretati. I parametri successivi, però, non sono variabili ma **indirizzi**: gli indirizzi delle variabili in cui scrivere i valori letti.

```
1 scanf("%d", &x);
```

& è l'**operatore di indirizzo**: &x è l'indirizzo della variabile x, il "numero civico" della sua cella di memoria (figura 5.5). Perché serve? Perché scanf deve **scrivere** nella variabile, e per farlo deve sapere dove si trova. Scrivendo scanf("%d", x) le si passerebbe il *valore* di x (un numero qualunque, ancora indefinito), e scanf non saprebbe dove mettere il dato letto; il motivo profondo lo vedremo alla fine del capitolo, con il passaggio dei parametri per valore.



Figura 5.5: scanf *riceve l'indirizzo di* x *e scrive lì il valore letto (l'indirizzo 1000 è solo un esempio).*

Le conversioni principali di scanf:

conversione	legge e scrive in una variabile di tipo
%c	char (un carattere)
%d	int (un intero decimale)
%f	float
%lf	double
%s	una stringa (capitolo ??)

%f o %lf?

Con printf, %f va bene sia per i float sia per i double. Con scanf no: per un double serve %lf. Il motivo: scanf riceve solo un indirizzo, e dalla conversione deve capire quanti byte scrivere a quell'indirizzo (4 per un float, 8 per un double). Con la conversione sbagliata scrive un numero di byte sbagliato, e il valore letto è senza senso.

Un esempio: echo

Il programma seguente legge tre caratteri, un intero, un `float` e un `double`, e li ristampa (fa da "eco"):

```c
#include <stdio.h>

int main(void)
{
    char     c1, c2, c3;
    int      i;
    float    x;
    double   y;

    printf("\n%s\n%s", "Input three characters ", "an int, a float, and a double \n");
    scanf("%c%c%c%d%f%lf", &c1, &c2, &c3, &i, &x, &y);
    printf("\nHere is the data that you typed in:\n");
    printf("%3c%3c%3c\n%17d%17f%17f\n\n", c1, c2, c3, i, x, y);
    return 0;
}
```

Proviamolo due volte, con input quasi uguali: nel secondo c'è solo uno spazio fra `B` e `C`.

Terminale

```
$ ./echo

Input three characters
an int, a float, and a double
ABC 3 55 77.7

Here is the data that you typed in:
  A  B  C
              3        55.000000        77.700000
```

Terminale

```
$ ./echo

Input three characters
an int, a float, and a double
AB C 3 55 77.7

Here is the data that you typed in:
  A  B
      1386527472         0.000000         0.000000
```

Il secondo risultato è disastroso, e il motivo è una regola di `scanf`: quando legge **numeri** salta gli spazi che li precedono, ma quando legge **caratteri** no, perché lo spazio *è* un carattere. La figura 5.6 mostra quali caratteri "mangia" ciascuna conversione.

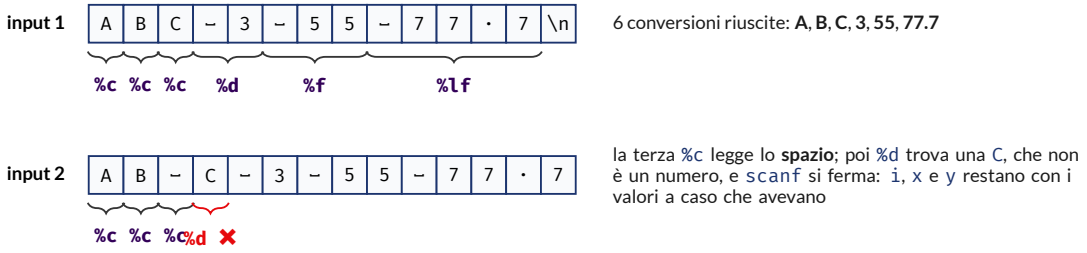


Figura 5.6: *I caratteri dell'input (␣ indica uno spazio) e la conversione che li consuma.*

Nel secondo caso `i`, `x` e `y` non ricevono nulla, e `printf` stampa ciò che contenevano: valori casuali, diversi da un computer all'altro (e da un'esecuzione all'altra). Come accorgersene dal programma? `scanf` **restituisce il numero di conversioni riuscite**. Salvandolo in una variabile e stampandolo (è il programma `echo_conversions.c` delle lezioni) si ottiene 6 nel primo caso e 3 nel secondo:

```c
int conversions;
...
conversions = scanf("%c%c%c%d%f%lf", &c1, &c2, &c3, &i, &x, &y)
    ;
...
printf("Correct conversions. %d\n", conversions);
```

Terminale

```
Correct conversions. 6
...
Correct conversions. 3
```

Controllare il valore restituito da `scanf` è il modo corretto di verificare che l'input sia stato letto: lo useremo fra poco.

Il controllo del flusso

Vero e falso in C

Le istruzioni sono normalmente eseguite in sequenza, ma la maggior parte degli algoritmi richiede di alterare il flusso sequenziale (capitolo 4). Il C offre `if` e `if-else` per scegliere fra azioni alternative, `while` e `for` per ripetere. Tutti richiedono di valutare un'espressione **logica**, cioè vera o falsa, e in C la regola è semplicissima:

- il **falso** è rappresentato dal valore **0**;
- **ogni valore diverso da zero** è interpretato come **vero**.

`if`

La forma generale è

```c
if (espressione) {
    istruzioni_if;
}
istruzione_successiva;
```

Se `espressione` è diversa da zero (vera) si eseguono `istruzioni_if` e poi `istruzione_successiva`; se vale 0 (falsa) si salta direttamente a `istruzione_successiva`. È la selezione a una via del capitolo precedente. Un esempio:

```c
a = 1;
if (b == 3) {
    a = 5;
}
printf("%d", a);
```

Il simbolo `==` è l'operatore di **uguaglianza**: verifica se `b` è uguale a 3. Se è vero, si assegna 5 ad `a` e viene stampato **5**; se è falso si esegue subito la stampa, e viene stampato **1**.

`==` e `=` sono due cose diverse

`==` **confronta**, `=` **assegna**. Scrivere `if (b = 3)` non è un errore per il compilatore: assegna 3 a `b`, e siccome il valore dell'assegnazione è 3 (diverso da zero, cioè vero), il ramo viene eseguito **sempre**. È uno degli errori più frequenti in C; il capitolo **??** ci torna in dettaglio. Compilando con `-Wall`, gcc avvisa.

Fra le graffe può esserci una sequenza di istruzioni; con una sola istruzione le graffe si possono omettere, ma conviene **metterle sempre**: se in seguito si aggiunge un'istruzione al blocco, senza graffe resterebbe fuori dall'`if`, con un errore difficile da vedere.

if-else

```c
if (espressione) {
    istruzioni_if;
} else {
    istruzioni_else;
}
```

Se `espressione` è diversa da zero si esegue il primo blocco, se è zero il secondo: è la selezione a due vie. Uno dei due blocchi viene sempre eseguito, mai entrambi.

Nell'esempio seguente, se `cnt` vale 0 il programma stampa $2 + 3 + 5 = 10$, altrimenti $-1 - 2 - 3 = -6$. A destra la variante con `if (cnt)`: non c'è nessun confronto, la decisione si basa solo sul valore di `cnt` (0 è falso, il resto vero), e i risultati si invertono.

```c
if (cnt == 0) {
    a = 2;
    b = 3;
    c = 5;
} else {
    a = -1;
    b = -2;
    c = -3;
}
printf("%d", a + b + c);
```

```c
if (cnt) {
    a = 2;
    b = 3;
    c = 5;
} else {
    a = -1;
    b = -2;
    c = -3;
}
printf("%d", a + b + c);
```

	con `cnt == 0`	con `cnt`
se `cnt` vale 0	stampa 10	stampa -6
se `cnt` vale, per esempio, 4	stampa -6	stampa 10

sum.c

```c
#include <stdio.h>

int main(void)
{
    int   i = 1, sum = 0;

    while (i <= 5) {
        sum = sum + i;
        i = i + 1;
    }
    printf("sum = %d\n", sum);
    return 0;
}
```

Terminale

```
$ ./sum
sum = 15
```

Il programma somma i numeri da 1 a 5. Nella dichiarazione, `int i = 1, sum = 0;` le due variabili vengono anche **inizializzate**. `<=` significa "minore o uguale".

Il `while` verifica se `i <= 5`: se la condizione è vera esegue le istruzioni del blocco fino alla `}`, poi il controllo **torna all'inizio** del ciclo per ripetere il test. L'iterazione continua finché il test non fallisce. È il ciclo con la condizione in testa del capitolo 4. Simuliamolo:

giro	`i <= 5`?	`sum` dopo il giro	`i` dopo il giro
1	$1 \le 5$, vero	$0 + 1 = 1$	2
2	$2 \le 5$, vero	$1 + 2 = 3$	3
3	$3 \le 5$, vero	$3 + 3 = 6$	4
4	$4 \le 5$, vero	$6 + 4 = 10$	5
5	$5 \le 5$, vero	$10 + 5 = 15$	6
6	$6 \le 5$, **falso**	si esce: `sum` vale 15	

La forma generale è

```
while (espressione) {
    istruzioni_while;
}
istruzioni;
```

Se `espressione` è diversa da zero si eseguono `istruzioni_while` e poi si verifica di nuovo `espressione`; il processo continua finché `espressione` vale 0. Perché il ciclo finisca, è **necessaria** un'istruzione dentro il ciclo che possa cambiare il valore dell'espressione (qui `i = i + 1`): senza, il test resterebbe vero per sempre.

Assegnazioni abbreviate e incremento

Istruzioni come `sum = sum + i;` sono così frequenti che il C ha una forma abbreviata: `sum += i;`. In generale, un'istruzione della forma `variabile op= espressione;` (con `op` uno fra `+`, `-`, `*`, `/`) equivale a `variabile = variabile op espressione;`.

Per aggiungere o togliere 1 ci sono gli **operatori di incremento e decremento**:

forma	come istruzione a sé, equivale a
`++i` e `i++`	`i = i + 1;`
`--i` e `i--`	`i = i - 1;`

Il ciclo di `sum.c` si scrive quindi anche così:

```
while (i <= 5) {
    sum += i;
    ++i;
}
```

++ dentro le espressioni

Come istruzioni isolate, `++i` e `i++` fanno la stessa cosa. Usati **dentro** un'espressione più grande (per esempio `a = i++;`) si comportano in modo diverso, e il loro funzionamento è più complicato: lo vedremo con calma nel capitolo 6. Per ora li useremo solo da soli.

`for`

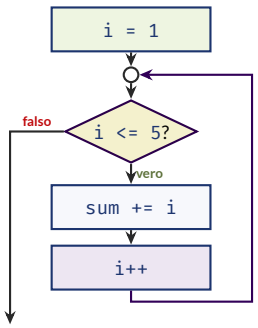
Il ciclo `for` ha la forma

```
for (expr1; expr2; expr3) {
    istruzioni_for;
}
```

```
expr1;
while (expr2) {
    istruzioni_for;
    expr3;
}
```

ed è equivalente al `while` a destra. Tipicamente `expr1` fa un'assegnazione iniziale, `expr2` un test, `expr3` incrementa una variabile: sono i tre passi del ciclo su contatore del capitolo 4 (inizializzazione, test, modifica), riuniti in un'unica riga. La somma da 1 a 5 diventa:

```
for (i = 1; i <= 5; i++) {
    sum += i;
}
```



UN ESEMPIO COMPLETO: `running_sum`

Il programma

Il programma seguente legge una sequenza di numeri e, per ciascuno, stampa una riga di tabella con il conteggio, il numero letto, il minimo, il massimo, la somma e la media dei numeri letti fino a quel momento.

running_sum.c

```c
/* Compute the minimum, maximum, sum, and average. */

#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int      i;
    double   x, min, max, sum, avg;

    if (scanf("%lf", &x) != 1) {
        printf("No data found - bye!\n");
        exit(1);
    }
    min = max = sum = avg = x;
    printf("%5s%9s%9s%9s%12s%12s\n%5s%9s%9s%9s%12s%12s\n\n",
           "Count", "Item", "Min", "Max", "Sum", "Average",
           "-----", "----", "---", "---", "---", "-------");
    printf("%5d%9.1f%9.1f%9.1f%12.3f%12.3f\n",
           1, x, min, max, sum, avg);
    for (i = 2; scanf("%lf", &x) == 1; ++i) {
        if (x < min)
            min = x;
        else if (x > max)
            max = x;
        sum += x;
        avg = sum / i;
        printf("%5d%9.1f%9.1f%9.1f%12.3f%12.3f\n",
               i, x, min, max, sum, avg);
    }
    return 0;
}
```

Riga per riga

- `if (scanf("%lf", &x) != 1)`: `scanf` restituisce il numero di conversioni riuscite; qui ne è richiesta una sola, quindi un valore diverso da 1 (`!=` significa "diverso") vuol dire che non è stato letto niente. In quel caso il programma stampa un messaggio ed esce con `exit(1)`: `exit`, il cui prototipo sta in `stdlib.h`, termina il programma restituendo al sistema operativo il valore indicato (qui 1, "qualcosa è andato storto").
- `min = max = sum = avg = x;`: un'assegnazione in C è anche un'espressione, che **vale** il valore assegnato. Le assegnazioni a catena si eseguono da destra a sinistra: `avg = x`, poi `sum = (avg = x)`, e così via. Tutte e quattro le variabili ricevono il primo numero letto.
- Le due `printf` con `%5s`, `%9s`… stampano l'intestazione della tabella: stringhe costanti in campi di larghezza fissa, così le colonne restano allineate.
- `for (i = 2; scanf("%lf", &x) == 1; ++i)`: un ciclo `for` in cui il **test è la lettura stessa**. Il ciclo continua finché `scanf` riesce a leggere un numero, cioè finché c'è input; non si sa in anticipo quante volte girerà. `i` conta i numeri letti e parte da 2 perché il primo è già stato letto prima del ciclo.
- `if (x < min) min = x; else if (x > max) max = x;`: aggiorna, se necessario, il minimo o il massimo. L'`else` contiene a sua volta un `if`. (Si poteva anche scrivere con due `if` indipendenti.)
- `avg = sum / i;`: qui la divisione è fra un `double` e un `int`, quindi è reale: nessuna trappola.

Tre modi di dare l'input

Il programma legge i numeri dallo **standard input**, il "canale" da cui un programma riceve i dati. Normalmente è la tastiera, e allora i numeri si possono scrivere uno alla volta, premendo Invio dopo ciascuno, oppure tutti su una riga; ma come fa il programma a sapere che l'input è finito? Bisogna segnalarlo con Ctrl+D all'inizio di una riga: `scanf` non riesce più a leggere, restituisce un valore diverso da 1, e il ciclo finisce.

Terminale

```
$ ./running_sum
3
Count     Item      Min       Max         Sum     Average
-----     ----      ---       ---         ---     -------

    1      3.0      3.0       3.0       3.000       3.000
-5 7
    2     -5.0     -5.0       3.0      -2.000      -1.000
    3      7.0     -5.0       7.0       5.000       1.667
Ctrl+D
$
```

(Ctrl+D in realtà non compare sullo schermo: qui è indicato per chiarezza.) Il terzo modo è il più comodo per provare un programma: scrivere i dati in un file e usarlo **al posto** della tastiera, con la **redirezione** `<` (figura 5.7). Il file `running_sum.in` delle lezioni contiene i dieci numeri `3 -5 7 -9 11 -13 15 -17 19 -21`.

Terminale

```
$ ./running_sum < running_sum.in
Count     Item      Min       Max         Sum     Average
-----     ----      ---       ---         ---     -------

    1      3.0      3.0       3.0       3.000       3.000
    2     -5.0     -5.0       3.0      -2.000      -1.000
    3      7.0     -5.0       7.0       5.000       1.667
    4     -9.0     -9.0       7.0      -4.000      -1.000
    5     11.0     -9.0      11.0       7.000       1.400
    6    -13.0    -13.0      11.0      -6.000      -1.000
    7     15.0    -13.0      15.0       9.000       1.286
    8    -17.0    -17.0      15.0      -8.000      -1.000
    9     19.0    -17.0      19.0      11.000       1.222
   10    -21.0    -21.0      19.0     -10.000      -1.000
```

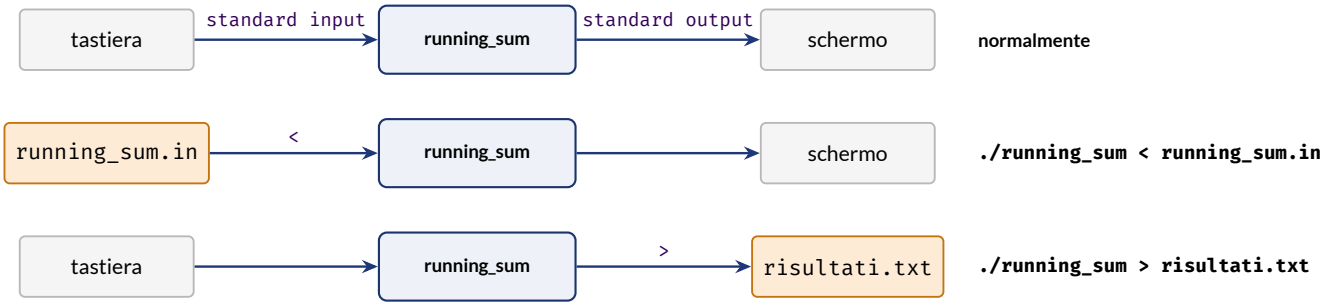


Figura 5.7: *La redirezione: con `<` un file prende il posto della tastiera, con `>` un file prende il posto dello schermo.*

È la shell a collegare il file al programma: il programma non se ne accorge nemmeno, continua a leggere dallo standard input. Simmetricamente, con `>` si può mandare l'output in un file invece che sullo schermo, e i due si possono combinare: `./running_sum < running_sum.in > risultati.txt`. Infine, se non c'è nessun dato, il programma esce con il suo messaggio, e il valore restituito al sistema è 1:

Terminale

```
$ ./running_sum < /dev/null
No data found - bye!
$ echo $?
1
```

(`/dev/null` è un file speciale di Linux, sempre vuoto.)

Una versione senza la lettura iniziale

Nel programma si fa una prima lettura **prima** del ciclo, poi le altre **nel** ciclo: lo stesso codice di lettura compare due volte. Quando è possibile, è più elegante mettere tutto dentro il ciclo. Ecco la versione delle lezioni con un `while`, in cui `cont` conta i numeri letti e il caso "nessun dato" si controlla alla fine:

running_sum2.c

```c
/* Compute the minimum, maximum, sum, and average. */

#include <stdio.h>

int main(void)
{
    int      cont = 0;
    double   x, min, max, sum, avg;

    while (scanf("%lf", &x) == 1) {
        cont = cont + 1;
        if (cont == 1) {
            sum = avg = min = max = x;
            printf("%5s%9s%9s%9s%12s%12s\n%5s%9s%9s%9s%12s%12s\n\n",
                   "Count", "Item", "Min", "Max", "Sum", "Average",
                   "-----", "----", "---", "---", "---", "-------");
        } else {
            if (x < min) min = x;
            else if (x > max) max = x;
            sum += x;
            avg = sum / cont;
        }
        printf("%5d%9.1f%9.1f%9.1f%12.3f%12.3f\n",
               cont, x, min, max, sum, avg);
    }
    if (cont == 0) {
        printf("No data found - bye!\n");
    }
    return 0;
}
```

L'output è identico. Di solito una variabile come `i` è la variabile di un ciclo su contatore; qui `cont` conta gli elementi letti, e il primo elemento è trattato a parte dentro il ciclo (`cont == 1`). Non serve più `stdlib.h`, perché non si usa `exit`.

Quando la compilazione fallisce

I messaggi del compilatore sono importanti e vanno letti; a volte però possono creare confusione. A lezione si compila il file `running_sum2Errore.c`, identico a quello appena visto tranne che per un dettaglio: nella prima riga manca la barra iniziale del commento.

```c
* Compute the minimum, maximum, sum, and average. */

#include <stdio.h>
...
```

Il risultato è una lunga, deprimente lista di **51 errori**, molti dei quali addirittura negli header file di sistema:

```
$ gcc -Wall -o running_sum2 running_sum2Errore.c
running_sum2Errore.c:1:11: error: expected '=', ',', ';', 'asm' or '__attribute__' before 'the'
    1 | * Compute the minimum, maximum, sum, and average. */
      |           ^~~
running_sum2Errore.c:1:11: error: unknown type name 'the'
In file included from /usr/include/stdio.h:44,
                 from running_sum2Errore.c:3:
/usr/include/x86_64-linux-gnu/bits/types/struct_FILE.h:95:3: error: unknown type name 'size_t'
   95 |   size_t __pad5;
      |   ^~~~~~
...
```

Che cosa è successo? Senza `/*` la prima riga non è più un commento: il compilatore la legge come codice C e cerca di darle un senso (`* Compute the…`), fallendo. Ma non sa dove finisce l'errore, e continua a leggere il resto del file "fuori fase": anche il contenuto di `stdio.h`, perfettamente corretto, viene letto nel contesto sbagliato e produce errori a cascata. La regola del capitolo 1 vale più che mai: si guarda **il primo** errore, che qui indica correttamente la riga 1, e si ignorano gli altri finché non lo si è corretto.

Una curiosità

Il file `running_sum2.c` distribuito con le lezioni ha lo stesso difetto di `running_sum2Errore.c` (i due file sono identici): per compilarlo bisogna aggiungere la `/` mancante all'inizio.

LE FUNZIONI

Dichiarare prima di usare

Una **funzione** è un pezzo di codice che rappresenta un blocco del processo di soluzione del problema: è il sotto-programma del capitolo 4. Un programma C è costituito da una o più funzioni, in uno o più file sorgente. Una di esse è necessariamente `main`, da cui comincia l'esecuzione; le altre sono richiamate da `main` o da altre funzioni.

In C le funzioni devono essere **dichiarate prima di usarle**. Supponiamo di voler usare la funzione `pow` della libreria matematica: la chiamata `pow(x, y)` restituisce x^y. La sua dichiarazione è

```
1 double pow(double x, double y);
```

Questo tipo di dichiarazione si chiama **prototipo** della funzione: dice il tipo del valore restituito (`double`), il nome e il tipo dei parametri. I nomi dei parametri, nel prototipo, sono ignorati dal compilatore: `double pow(double, double);` è equivalente, perché al compilatore interessano solo i tipi. È comunque buona norma metterli, soprattutto se sono significativi.

La forma generale di un prototipo è `tipo nome_funzione(lista dei tipi dei parametri);`. La parola chiave `void` si usa quando la funzione non ha parametri (`int f(void);`) o non restituisce niente (`void f(int);`). Se necessario, al momento della chiamata gli argomenti vengono convertiti al tipo indicato nel prototipo: chiamando `pow(2, 10)` con due interi, questi diventano `2.0` e `10.0`.

L'esempio maxmin

Il programma seguente stampa delle informazioni su se stesso, legge un intero `n`, legge `n` numeri reali e ne trova il massimo e il minimo.

maxmin.c

```c
#include <stdio.h>

float   maximum(float x, float y);
float   minimum(float x, float y);
void    prn_info(void);

int main(void)
{
    int     i, n;
    float   max, min, x;

    prn_info();
    printf("Input n:  ");
    scanf("%d", &n);
    printf("\nInput %d real numbers:  ", n);
    scanf("%f", &x);
    max = min = x;
    for (i = 2; i <= n; ++i) {
        scanf("%f", &x);
        max = maximum(max, x);
        min = minimum(min, x);
    }
    printf("\n%s%11.3f\n%s%11.3f\n\n",
           "Maximum value:", max,
           "Minimum value:", min);
    return 0;
}

float maximum(float x, float y)
{
    if (x > y)
        return x;
    else
        return y;
}

float minimum(float x, float y)
{
    if (x < y)
        return x;
    else
        return y;
}

void prn_info(void)
{
    printf("\n%s\n%s\n\n",
           "This program reads an integer value for n, and then",
           "processes n real numbers to find max and min values.");
}
```

Terminale

```
$ ./maxmin

This program reads an integer value for n, and then
processes n real numbers to find max and min values.

Input n:  5

Input 5 real numbers:  737.7799  -11.2e+3  -777  0.001  3.14159

Maximum value:    737.780
Minimum value: -11200.000
```

Si noti che `scanf` con `%f` accetta anche la notazione scientifica: `-11.2e+3` è $-11{,}2 \cdot 10^3 = -11\,200$.

Come funziona

- In cima al file ci sono i **prototipi** di `maximum`, `minimum` e `prn_info`. I primi due dicono che le funzioni hanno due parametri di tipo `float` e restituiscono un `float`; il terzo che `prn_info` non ha parametri e non restituisce alcun valore.
- `prn_info();` chiama la funzione, che a sua volta chiama `printf` per stampare le informazioni sul programma e sul suo uso.
- Il programma chiede e legge `n` (`%d`: un intero decimale), poi legge il primo numero reale in `x`, e `max = min = x;` inizializza massimo e minimo con quel valore.
- A ogni giro del ciclo si legge un nuovo valore di `x`; `max = maximum(max, x);` calcola il massimo fra `max` e `x` e lo assegna a `max`, e lo stesso per il minimo.

La **definizione** di `maximum` specifica che cosa la funzione fa quando viene chiamata. È fatta di una **testa**, la parte prima della prima graffa (`float maximum(float x, float y)`: il primo `float` è il tipo del valore restituito, gli altri due i tipi dei parametri), e di un **corpo**, la parte fra le graffe.

L'istruzione `return` **interrompe** la funzione e restituisce al chiamante il valore della sua espressione; la forma generale è `return espressione;`. In una funzione `void` si può scrivere `return;` senza espressione, o lasciare che la funzione arrivi da sola alla graffa finale, come fa `prn_info`.

Due x diverse

Le funzioni `maximum` e `minimum` hanno entrambe i parametri `x` e `y`, e una variabile `x` c'è anche in `main`. Non c'è nessuna relazione fra loro: sono variabili **logicamente e fisicamente diverse**, in celle di memoria diverse. Usare lo stesso nome informa solo chi legge che il loro uso è simile. La figura 5.8 mostra la memoria durante la chiamata `maximum(max, x)` al secondo giro.

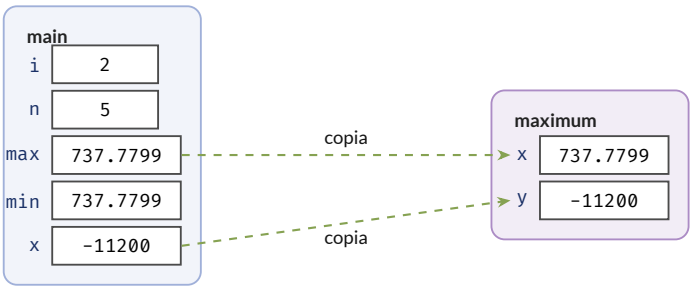


Figura 5.8: *La* `x` *di* `main` *e la* `x` *di* `maximum` *sono due variabili distinte:* `maximum` *riceve una* ***copia*** *dei valori.*

Che cosa succede senza prototipi

Le slide mostrano anche una versione di `maxmin` senza i prototipi in cima al file. Con gcc il risultato è questo:

Terminale

```
$ gcc -Wall -o maxmin maxmin.c
maxmin.c:9:4: warning: implicit declaration of function ‘prn_info’
maxmin.c:17:13: warning: implicit declaration of function ‘maximum’
maxmin.c:18:13: warning: implicit declaration of function ‘minimum’
maxmin.c:26:7: error: conflicting types for ‘maximum’; have ‘float(float,  float)’
maxmin.c:34:7: error: conflicting types for ‘minimum’; have ‘float(float,  float)’
```

Il compilatore legge il file dall'alto in basso. Quando incontra `maximum(max, x)` non sa ancora nulla di `maximum`: per una vecchia regola del C, "immagina" che sia una funzione che restituisce un `int` (è la *dichiarazione implicita* di cui avvisa il warning). Più avanti trova la definizione vera, che restituisce un `float`, e la contraddizione diventa un errore. Le versioni più recenti di gcc (dalla 14) non accettano più le dichiarazioni implicite nemmeno come avviso: la chiamata di una funzione non dichiarata è direttamente un errore. La lezione è semplice: **ogni funzione va dichiarata prima dell'uso**.

Un modo alternativo, che va bene per i file piccoli, è mettere le **definizioni** delle funzioni **prima** di `main`: quando il compilatore arriva alla chiamata ha già letto la definizione, e la testa della funzione fa da prototipo.

Un punto e virgola di troppo

Nelle prime slide su `maxmin` la funzione `maximum` contiene un errore di battitura istruttivo:

```c
float maximum(float x, float y)
{
    if (x > y) ;{
        return x;
    }
    else {
        return y;
    }
}
```

Terminale

```
$ gcc -Wall -o maxmin maxmin.c
maxmin.c: In function ‘maximum’:
maxmin.c:31:4: warning: this ‘if’ clause does not guard... [-
    Wmisleading-indentation]
   31 |    if (x > y) ;{
      |    ^~
maxmin.c:34:4: error: ‘else’ without a previous ‘if’
   34 |    else {
      |    ^~~~
```

Il `;` dopo la condizione è un'**istruzione vuota**: l'`if` controlla quella, cioè nulla. Il blocco `{ return x; }` che segue non fa più parte dell'`if`: è un blocco qualunque, eseguito sempre. E quando arriva l'`else`, l'`if` a cui dovrebbe appartenere è già concluso: "`else` senza un `if` precedente". gcc avvisa anche che l'indentazione è *ingannevole*: il codice sembra dire una cosa e ne fa un'altra.

Il passaggio dei parametri per valore

Esistono due modi di passare i parametri a una funzione:

- il **passaggio per valore**: viene creata una **copia** della variabile passata come parametro;
- il **passaggio per riferimento**: viene passata la variabile stessa.

In C i parametri sono passati solo per valore. Il programma seguente lo dimostra: la funzione prova a cambiare il valore di `a`, ma `main` non se ne accorge.

try_to.c

```c
#include <stdio.h>

int main(void)
{
    int    a = 1;
    void   try_to_change_it(int);

    printf("%d\n", a);      /* 1 is printed */
    try_to_change_it(a);
    printf("%d\n", a);      /* 1 is printed again! */
    return 0;
}

void try_to_change_it(int a)
{
    a = 777;
}
```

Terminale

```
$ ./try_to
1
1
```

Il prototipo è scritto dentro `main`: è lecito, e vale solo lì. Di solito lo si mette in cima al file.

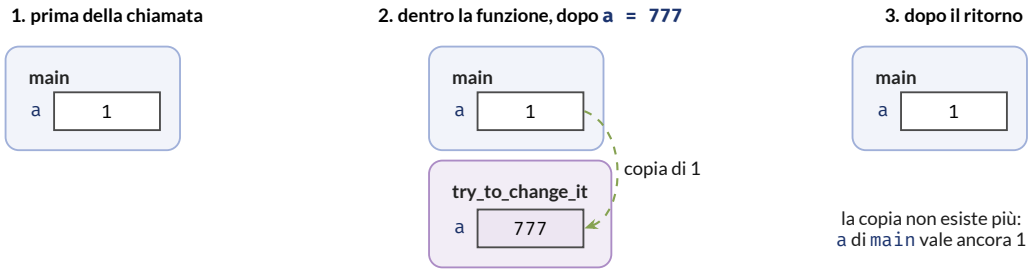


Figura 5.9: *Il passaggio per valore: la funzione lavora su una copia, che scompare quando la funzione termina.*

Alla chiamata, il parametro `a` di `try_to_change_it` (una variabile diversa da quella di `main`, anche se ha lo stesso nome) riceve una **copia** del valore 1. L'assegnazione `a = 777` modifica la copia; quando la funzione termina la copia scompare, e `main` ritrova la sua `a` intatta (figura 5.9).

Ecco perché `scanf` ha bisogno di `&x`: se ricevesse `x`, riceverebbe solo una copia del suo valore e non potrebbe modificare la variabile. Ricevendo l'**indirizzo**, sa dove andare a scrivere. Come una funzione possa usare un indirizzo per modificare una variabile del chiamante (i *puntatori*) è l'argomento del capitolo **??**.

In sintesi

Un programma C è fatto di funzioni, e l'esecuzione comincia da `main`. Le righe che cominciano con `#` sono per il preprocessore: `#include` copia un header (che **descrive** le funzioni; il loro codice arriva dalla libreria al momento del linking), `#define` definisce costanti simboliche. Le variabili si dichiarano con un tipo prima di usarle; `=` assegna (si legge da destra a sinistra); la divisione fra interi tronca. `printf` e `scanf` usano una stringa di controllo con le conversioni (`%d`, `%f`, `%lf`…); `scanf` vuole gli **indirizzi** delle variabili e restituisce il numero di conversioni riuscite. In C 0 è falso e ogni altro valore è vero; `if`, `while` e `for` realizzano selezione e iterazione. Le funzioni vanno dichiarate con un prototipo prima dell'uso, e ricevono sempre una **copia** dei parametri.

Riferimenti

- **A. Kelley, I. Pohl**, *A Book on C*, Addison-Wesley: capitolo 1, da cui provengono gli esempi di questo capitolo.
- **Deitel & Deitel**, *Il linguaggio C*: capitoli 2–5.
- **printf** e **scanf** — cppreference.com.
  `https://en.cppreference.com/w/c/io/fprintf`
  `https://en.cppreference.com/w/c/io/fscanf`

Elementi lessicali e operatori

Il C è un linguaggio, e come ogni linguaggio ha un **alfabeto**, delle regole per formare le **parole** e delle regole per comporre le parole in **frasi** corrette, la **sintassi**. Il compilatore, per prima cosa, verifica che il programma sia sintatticamente corretto: la correttezza sintattica è necessaria per la traduzione (un programma con un errore di sintassi non viene tradotto). Non è però sufficiente: un programma sintatticamente perfetto può comunque fare la cosa sbagliata, come la divisione intera della maratona nel capitolo 5.

Questo capitolo guarda il C "al microscopio": come il compilatore legge il testo, quali parole sono riservate, come si scelgono i nomi, in che ordine si calcolano le espressioni.

Dai caratteri ai token

I token

Un programma è una sequenza di caratteri leciti (lettere, cifre, simboli, spazi). Il compilatore, per prima cosa, raggruppa i caratteri in unità dotate di significato, i **token**, come noi raggruppiamo le lettere in parole. Prendiamo un programma che legge due interi e ne stampa la somma:

somma.c

```c
/* Read in two integers and print their sum. */

#include <stdio.h>

int main(void)
{
    int   a, b, sum;

    printf("Input two integers:  ");
    scanf("%d%d", &a, &b);
    sum = a + b;
    printf("%d + %d = %d\n", a, b, sum);
    return 0;
}
```

Terminale

```
$ ./somma
Input two integers:  7 35
7 + 35 = 42
```

I token sono di alcune categorie (figura 6.1):

- gli **identificatori**, i nomi: `main`, `a`, `b`, `sum`, `printf`, `scanf`;
- le **parole chiave**: `int`, `void`, `return`;
- gli **operatori**: `=`, `+`, `&`, e le parentesi `( )` dopo `main`, che dicono che `main` è una funzione;
- la **punteggiatura**: `{ }`, `;` e `,`;
- le **costanti**: i numeri come `0` e le stringhe come `"Input two integers: "`, che il compilatore tratta come un **unico** token, spazi compresi.

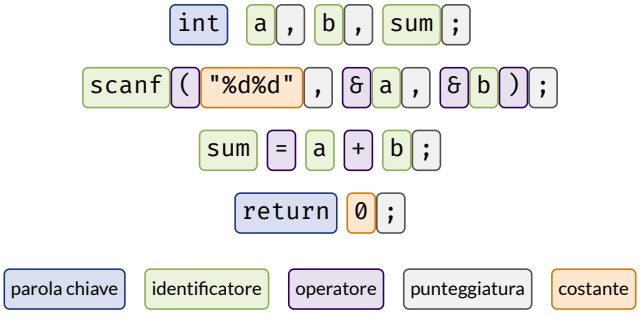


Figura 6.1: *Alcune righe di* `somma.c` *scomposte in token, colorati per categoria.*

Gli spazi

Il compilatore usa gli spazi bianchi (spazi, tabulazioni, a capo) per distinguere i token. Dove due token si "toccherebbero" formando una parola diversa, lo spazio è **necessario**: non si può scrivere `inta, b, sum;`, perché il compilatore non riesce a staccare `int` da `a` e vede un unico identificatore, `inta`, mai dichiarato:

Terminale

```
$ gcc -Wall -c somma.c
somma.c: In function ‘main’:
somma.c:7:5: error: ‘inta’ undeclared (first use in this function); did you mean ‘int’?
    7 |     inta, b, sum;
      |     ^~~~
      |     int
```

Non tutti gli spazi però sono necessari: `int a,b,sum;` va benissimo, perché le virgole separano già i token. E dove servono, servono davvero: `int absum;` dichiara un'unica variabile chiamata `absum`.

Al contrario, gli spazi **in più** sono ignorati: `sum=a+b;` e `sum = a + b ;` sono la stessa istruzione. Ma uno spazio **dentro** un token lo spezza: `s u m = a + b;` non è lecito, perché il compilatore tratta `s`, `u` e `m` come tre token diversi.

Terminale

```
$ gcc -Wall -c somma.c
somma.c:11:5: error: unknown type name ‘s’
   11 |     s u m = a + b;
      |     ^
```

Lo stesso vale per l'operatore di indirizzo: in `&a` il carattere `&` è un token a sé, separato dalla variabile, quindi `& a` o `& b` sono scritture lecite (anche se insolite). Però i due argomenti devono comunque essere separati dalla virgola: `& a & b` non è lecito, perché viene letto come `&a & b`, cioè l'operatore `&` (che fra due operandi ha un altro significato) applicato all'indirizzo di `a` e a `b`, e il compilatore protesta:

Terminale

```
$ gcc -Wall -c somma.c
somma.c:10:23: error: invalid operands to binary & (have ‘int *’ and ‘int’)
   10 |     scanf("%d%d", & a & b);
      |                   ~~~ ^
```

Identificatori della libreria

`printf` e `scanf` sono identificatori come gli altri; le parentesi che li seguono indicano che sono funzioni. Il compilatore non ne conosce il codice: dopo la compilazione, il linker crea l'eseguibile prendendo la loro definizione dalla libreria standard. Non sono parole chiave, ma andrebbero considerati **riservati**: usarli per altri scopi è una pessima idea, come mostra l'ultimo esempio di questo capitolo.

I COMMENTI

I **commenti** sono sequenze arbitrarie di testo fra i delimitatori `/*` e `*/`, oppure da `//` alla fine della riga. I commenti **non sono token**: il compilatore trasforma ogni commento in uno spazio. Lo si può verificare con un esempio un po' estremo, in cui un commento separa `int` da `x`:

commenti.c

```c
#include <stdio.h>

int main(void)
{
    int/* un commento vale come uno spazio */x = 5;

    printf("%d\n", x);   // un commento su una riga
    return 0;
}
```

Terminale

```
$ ./commenti
5
```

I commenti `/* */` **non si annidano**: il primo `*/` chiude il commento, qualunque cosa ci sia prima. Con `/* esterno /* interno */ ancora esterno? */` il commento finisce dopo `interno`, e "ancora esterno?" viene letto come codice (gcc avvisa: *"/*" within comment*).

I commenti sono la documentazione del codice. Andrebbero scritti **insieme** al codice: una volta che il programma funziona si tende a scriverne di brevissimi, o a non scriverne affatto. Un buon commento spiega il *perché*, non ripete il *che cosa*: `i = i + 1; /* incrementa i */` non aiuta nessuno, `/* il primo elemento è già stato letto */` sì.

LE PAROLE CHIAVE

Le **parole chiave** sono parole riservate, che non possono essere usate in altri contesti (per esempio come nomi di variabili). Il C ne ha, nel complesso, poche; l'ANSI C ne definisce 32:

`auto`	`break`	`case`	`char`	`const`	`continue`	`default`	`do`
`double`	`else`	`enum`	`extern`	`float`	`for`	`goto`	`if`
`int`	`long`	`register`	`return`	`short`	`signed`	`sizeof`	`static`
`struct`	`switch`	`typedef`	`union`	`unsigned`	`void`	`volatile`	`while`

Le parole chiave sono minuscole

Nella tabella delle slide compaiono `Volatile` e `Unsigned` con l'iniziale maiuscola: è un effetto della correzione automatica. Le parole chiave sono tutte **minuscole**, e siccome il C distingue maiuscole e minuscole, `Unsigned` non sarebbe una parola chiave ma un identificatore qualunque.

Alcune implementazioni hanno parole chiave aggiuntive (gcc, per esempio, riconosce `asm` e `typeof`), e le versioni successive dello standard ne hanno aggiunte poche altre (fra cui `inline`, e nel C23 `bool`, `true` e `false`). Non serve impararle a memoria: VS Code le colora in blu o in viola, e l'errore di usarne una come nome si vede subito.

GLI IDENTIFICATORI

La regola

Un **identificatore** (un nome di variabile, di funzione, di tipo…) è una sequenza di caratteri composta da **lettere**, **cifre** e dal carattere speciale `_` (*underscore*), che **non comincia con una cifra**. Formalmente, nella notazione delle slide:

$$\texttt{identifier ::= \{letter | underscore\}}_1\ \texttt{\{letter | underscore | digit\}}_{0+}$$

dove `|` significa "oppure", il pedice $_1$ "esattamente una volta" e il pedice $_{0+}$ "zero o più volte". La stessa regola si può disegnare come un percorso da seguire, un *diagramma sintattico* (figura 6.2): un nome è lecito se lo si può ottenere percorrendo le frecce dall'inizio alla fine e raccogliendo un carattere a ogni casella attraversata.

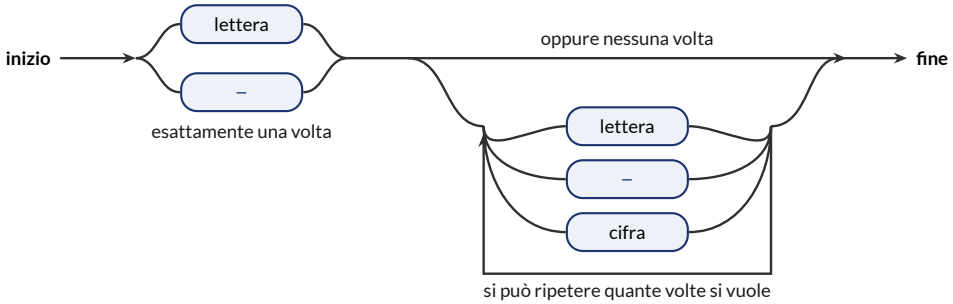


Figura 6.2: *Il diagramma sintattico di un identificatore.*

identificatori leciti		**identificatori non leciti**	
`K`	una lettera sola	`not#me`	`#` non è ammesso
`_id`	comincia con `_`	`101_south`	comincia con una cifra
`iamanidentifier2`	cifre dopo l'inizio	`-plus`	`-` è un operatore
`so_am_i`		`int`	è una parola chiave

Maiuscole, lunghezza, scelta dei nomi

Il C è *case sensitive*: le lettere maiuscole e minuscole sono considerate **diverse**, quindi `sum`, `Sum` e `SUM` sono tre identificatori distinti.

Sulla lunghezza: nei sistemi più vecchi contavano solo i primi 8 caratteri, per cui `i_am_an_identifier` e `i_am_an_elephant` sarebbero stati lo stesso nome (i primi 8 caratteri sono `i_am_an_` in entrambi). L'ANSI C garantisce che siano considerati almeno i primi 31 caratteri, e i compilatori moderni ne considerano molti di più: in pratica, la lunghezza non è un problema.

Gli identificatori vanno scelti in modo che contribuiscano alla **leggibilità** e alla **documentazione** del programma. Per calcolare l'IVA di un prodotto,

```
1 float tax_iva, price, tax_iva_rate;
2
3 tax_iva = price * tax_iva_rate;
```

si capisce da sé, mentre `t = p * r;` richiede un commento per ogni variabile. Infine, per convenzione gli identificatori di sistema (della libreria, del compilatore) cominciano con uno o due underscore: per evitare conflitti, conviene non creare identificatori che cominciano con `_`.

Precedenza e associatività

Gli operatori hanno regole di **precedenza** e di **associatività** che determinano come vengono calcolate le espressioni. Le espressioni fra parentesi sono calcolate per prime; le parentesi si possono usare sia per **chiarire** l'ordine delle operazioni, sia per **modificarlo**.

- In `1 + 2 * 3` l'operatore `*` ha una precedenza più alta di `+`: il risultato è 7, come in `1 + (2 * 3)`.
- In `(1 + 2) * 3` le parentesi hanno la priorità: il risultato è 9.
- In `1 + 2 - 3 + 4 - 5` gli operatori binari `+` e `-` hanno la **stessa** precedenza: l'ordine lo stabilisce la regola di associatività, che per loro è **da sinistra a destra**. L'espressione equivale a `(((1 + 2) - 3) + 4) - 5`, cioè −1.

La figura 6.3 mostra gli alberi delle tre espressioni: precedenza e associatività servono proprio a stabilire, senza ambiguità, quale albero corrisponde a una sequenza di token.

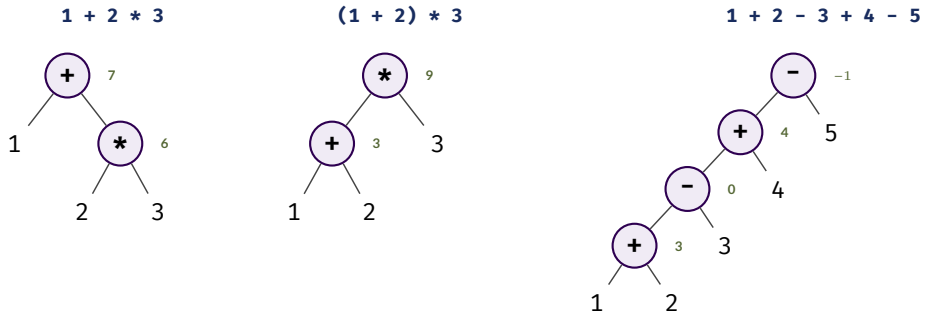


Figura 6.3: *Gli alberi delle tre espressioni, con il valore di ogni operazione: si calcola dal basso verso l'alto.*

La tabella degli operatori

La tabella seguente riassume precedenza e associatività degli operatori visti finora, dalla precedenza più alta (in alto) alla più bassa. Gli operatori sulla stessa riga hanno la stessa precedenza.

operatori	significato	associatività
`()  ++  --`	parentesi; incremento e decremento **postfissi**	da sinistra a destra
`+  -  ++  --`	più e meno **unari**; incremento e decremento **prefissi**	da destra a sinistra
`*  /  %`	moltiplicazione, divisione, resto	da sinistra a destra
`+  -`	somma e sottrazione	da sinistra a destra
`=  +=  -=  *=  /=`	assegnazioni	da destra a sinistra

Qualche commento:

- gli operatori **unari** hanno un solo operando: in `-x` il meno è unario (cambia segno), in `a - x` è binario (sottrae). Gli unari hanno precedenza più alta dei binari: `-a * b` è `(-a) * b`;
- `%` è il **resto** della divisione fra interi (detto anche *modulo*): `7 / 2` vale 3 e `7 % 2` vale 1, perché $7 = 3 \cdot 2 + 1$. Si usa spesso per sapere se un numero è pari (`n % 2 == 0`) o divisibile per un altro;
- le assegnazioni hanno la precedenza più bassa (si calcola prima tutto il lato destro) e associano da **destra** a sinistra: è il motivo per cui `min = max = sum = avg = x;` del capitolo 5 funziona, perché equivale a `min = (max = (sum = (avg = x)));`.

```c
printf("%d %d %d\n", 1 + 2 * 3, (1 + 2) * 3, 1 + 2 - 3 + 4 - 5);
printf("%d %d %d\n", 7 / 2, 7 % 2, -7 / 2);
```

Terminale

```
7 9 -1
3 1 -3
```

Si noti l'ultimo valore: la divisione fra interi **tronca verso lo zero**, quindi $-7/2 = -3{,}5$ diventa -3 (non -4).

Nel dubbio, parentesi

Nessuno ricorda a memoria tutta la tabella delle precedenze del C (che conta una quindicina di livelli). Quando l'ordine non è ovvio, le parentesi lo rendono esplicito per il compilatore e, soprattutto, per chi legge: `(a * b) + c` non costa nulla e non lascia dubbi.

Incremento e decremento

Prefisso e postfisso

Gli operatori di incremento `++` e decremento `--` sono **unari** e si possono usare in due modi: come operatore **prefisso** (`++i`, prima della variabile) o **postfisso** (`i++`, dopo). Si applicano a **variabili**, non a costanti o a espressioni: `++i` e `cnt--` sono usi leciti, `777++` e `++(a * b - 1)` no, perché non c'è una variabile da modificare. Il compilatore lo dice così (*lvalue*, "valore a sinistra", è qualcosa che può stare a sinistra di un'assegnazione, cioè una variabile):

Terminale

```
$ gcc -Wall -c costanti.c
costanti.c:4:8: error: lvalue required as increment operand
costanti.c:5:5: error: lvalue required as increment operand
```

Le due forme non hanno lo stesso comportamento, e per capire la differenza bisogna ricordare che in C **ogni espressione ha un valore**:

- `++i`: la variabile `i` viene incrementata di 1, e l'espressione vale il **nuovo** valore di `i` ("prima incrementa, poi usa");
- `i++`: l'espressione vale il **vecchio** valore di `i`, e poi la variabile viene incrementata ("prima usa, poi incrementa").

Allo stesso modo `--i` decrementa e vale il nuovo valore, `i--` vale il vecchio valore e poi decrementa. Come istruzioni a sé (`i++;`) le due forme sono equivalenti, perché il valore dell'espressione non viene usato da nessuno.

Un esempio da simulare

incr.c

```c
#include <stdio.h>

int main(void)
{
    int a, b, c = 0;

    a = ++c;
    b = c++;
    printf("%d %d %d\n", a, b, ++c);
    return 0;
}
```

Terminale

```
$ ./incr
1 1 3
```

Nelle slide la dichiarazione è scritta `int a, b, c==0;`: è un refuso, `==` è il confronto; serve `=`.

Simuliamo l'esecuzione, istruzione per istruzione:

istruzione	che cosa succede	a	b	c
`int a, b, c = 0;`	dichiarazioni, solo `c` è inizializzata	?	?	0
`a = ++c;`	`c` diventa 1; `++c` vale 1, che va in `a`	1	?	1
`b = c++;`	`c++` vale 1 (il vecchio valore), che va in `b`; poi `c` diventa 2	1	1	2
`printf(…, ++c);`	`c` diventa 3; `++c` vale 3 e viene stampato	1	1	3

Quando ++ diventa pericoloso

`++` e `--` **modificano** la variabile su cui operano. Usati da soli non danno problemi; in espressioni più complesse bisogna prestare molta attenzione. Le slide mostrano questo programma, con l'intento di calcolare $a \cdot (a + 1)$ in tre modi:

ub.c

```c
#include <stdio.h>

int main(void)
{
    int a = 1;

    printf("a*(a+1) = %d\n", a * (a + 1));
    printf("a*(a+1) = %d\n", a * a++);
    a = 1;
    printf("a*(a+1) = %d\n", a * ++a);
    return 0;
}
```

Ecco che cosa succede compilandolo con gcc (su Linux) e con clang (il compilatore di macOS):

gcc

```
$ gcc -Wall -o ub ub.c
ub.c:8:35: warning: operation on ‘a’ may be undefined
     [-Wsequence-point]
ub.c:10:34: warning: operation on ‘a’ may be
    undefined [-Wsequence-point]
$ ./ub
a*(a+1) = 2
a*(a+1) = 2
a*(a+1) = 4
```

clang

```
$ clang -Wall -o ub ub.c
ub.c:8:35: warning: unsequenced modification and
    access to 'a' [-Wunsequenced]
ub.c:10:34: warning: unsequenced modification and
    access to 'a' [-Wunsequenced]
$ ./ub
a*(a+1) = 2
a*(a+1) = 1
a*(a+1) = 2
```

Lo stesso programma dà risultati diversi con compilatori diversi, e nessuno dei due ha torto. In `a * a++` il C non stabilisce se il primo `a` vada letto prima o dopo l'incremento: il programma legge e modifica la stessa variabile nella stessa espressione, senza un ordine definito, e lo standard dice che in questo caso il comportamento è **indefinito** (*undefined behavior*): il compilatore può fare qualunque cosa. Le slide mostrano i risultati di gcc (2, 2, 4) come se fossero "quelli giusti"; in realtà non esistono risultati giusti, e infatti entrambi i compilatori avvisano.

Una variabile modificata con `++` non va usata altrove nella stessa espressione

Espressioni come `a * a++`, `i = i++` o `printf("%d %d", n, n++)` hanno comportamento indefinito. La regola pratica delle slide è la migliore: **meglio evitarle** e scrivere esplicitamente ciò che si intende, per esempio `a * (a + 1)`. E i warning di `-Wall` vanno sempre letti.

La libreria standard e i nomi riservati

Che cosa succede se si **ridefinisce** una funzione della libreria standard? Le slide lo provano con `scanf`, scrivendo una funzione con lo stesso nome che stampa la stringa ricevuta. Per non avere conflitti con il prototipo vero, il file **non** include `stdio.h`:

main.c

```c
void scanf(const char *s)
{
    printf(s);
}

int main()
{
    scanf("The printing scanf\n");
    return 0;
}
```

Terminale

```
$ gcc -o ridef main.c
main.c:1:6: warning: conflicting types for built-in function ‘scanf’; expected ‘int(const char *, ...)’ [-
    Wbuiltin-declaration-mismatch]
main.c:3:5: warning: implicit declaration of function ‘printf’ [-Wimplicit-function-declaration]
main.c:3:5: warning: incompatible implicit declaration of built-in function ‘printf’
main.c:3:5: warning: format not a string literal and no format arguments [-Wformat-security]
$ ./ridef
The printing scanf
```

Il risultato è istruttivo:

- anche senza `stdio.h`, il compilatore si accorge che si sta usando il nome di una funzione “predefinita” (*built-in*) con un tipo diverso da quello che conosce;
- dà però solo dei **warning**, non degli errori: l’eseguibile viene creato;
- e viene usata la **nuova** definizione: la “`scanf`” stampa invece di leggere.

Le slide mostrano anche la variante con la funzione in un file separato (`scanf.c`) e il solo prototipo in `main.c`: il risultato è lo stesso, perché il linker, trovando `scanf` già definita nei nostri file oggetto, non va a cercarla nella libreria. (Si notino anche i warning su `printf`: senza `stdio.h` il compilatore non ne conosce il prototipo. Le versioni di gcc dalla 14 in poi considerano questa dichiarazione implicita un errore, e il programma non si compila più.)

La morale: i nomi delle funzioni della libreria standard **non sono parole chiave**, quindi il compilatore non ne impedisce l’uso, ma vanno trattati come se lo fossero. Un programma che ridefinisce `scanf` fa qualcosa di diverso da ciò che chiunque si aspetta leggendolo.

In sintesi

Il compilatore legge il programma come una sequenza di **token**: identificatori, parole chiave, operatori, punteggiatura, costanti; gli spazi separano i token quando serve e altrimenti sono ignorati, i commenti valgono come spazi. Le 32 parole chiave dell’ANSI C sono riservate e minuscole; un identificatore è fatto di lettere, cifre e `_` e non comincia con una cifra, e maiuscole e minuscole contano. **Precedenza** e **associatività** stabiliscono l’albero di un’espressione (le parentesi lo rendono esplicito). `++i` vale il nuovo valore, `i++` il vecchio; leggere e modificare la stessa variabile nella stessa espressione dà un comportamento **indefinito**. I nomi della libreria standard vanno trattati come riservati.

Riferimenti

- **A. Kelley, I. Pohl**, *A Book on C*: capitolo 2 (elementi lessicali, operatori e il sistema C).
- **C Operator Precedence** — cppreference.com.
  `https://en.cppreference.com/w/c/language/operator_precedence`
- **Order of evaluation** (comportamento indefinito di `i++ + i++` e simili) — cppreference.com.
  `https://en.cppreference.com/w/c/language/eval_order`

Programmi su più file e make

Finora ogni programma stava in un solo file. I programmi veri sono fatti di molti file, a volte migliaia, e compilarli tutti a ogni modifica sarebbe lentissimo. Questo capitolo mostra come si divide un programma in più file, come si compilano separatamente e si collegano (riprendendo il capitolo 3), e come il programma `make` automatizza il lavoro, ricompilando solo ciò che serve.

Perché più file

Dividere un programma in più file sorgente ha diversi vantaggi:

- **organizzazione**: ogni file raccoglie le funzioni che riguardano un aspetto del problema (l'input, i calcoli, l'output), come i capitoli di un libro;
- **riuso**: un file con funzioni utili (per esempio di geometria) si può usare in più programmi;
- **lavoro di gruppo**: persone diverse lavorano su file diversi;
- **compilazione più rapida**: se si modifica un solo file, si ricompila solo quello e si rifà il collegamento.

Un piccolo progetto

Prendiamo un programma che calcola l'area di un cerchio e di un quadrato, e dividiamolo in tre file: le funzioni di calcolo in `aree.c`, il programma principale in `main.c`, e un **file di intestazione** `aree.h` con ciò che serve a entrambi.

aree.h
```h
/* aree.h: costanti e prototipi delle funzioni sulle aree */

#define PI 3.14159

double area_cerchio(double raggio);
double area_quadrato(double lato);
```

aree.c
```c
/* aree.c: definizioni delle funzioni sulle aree */

#include "aree.h"

double area_cerchio(double raggio)
{
    return PI * raggio * raggio;
}

double area_quadrato(double lato)
{
    return lato * lato;
}
```

main.c
```c
#include <stdio.h>
#include "aree.h"

int main(void)
{
    double x;

    printf("Inserisci una misura: ");
    scanf("%lf", &x);
    printf("cerchio di raggio %g: area %g\n",
           x, area_cerchio(x));
    printf("quadrato di lato %g: area %g\n",
           x, area_quadrato(x));
    return 0;
}
```

Il ruolo di ciascun file:

- `aree.h` contiene le **dichiarazioni**: la costante e i prototipi. Chi lo include sa *che* le funzioni esistono e come si chiamano;
- `aree.c` contiene le **definizioni**: il codice delle funzioni, scritto una volta sola. Include anche lui `aree.h`, così il compilatore controlla che le definizioni corrispondano ai prototipi;
- `main.c` include `aree.h` e può quindi chiamare le funzioni, anche se il loro codice sta in un altro file.

Compilare separatamente, collegare insieme

Ogni file `.c` si compila per conto suo con l'opzione `-c`, che produce il file oggetto; poi il linker unisce i file oggetto nell'eseguibile:

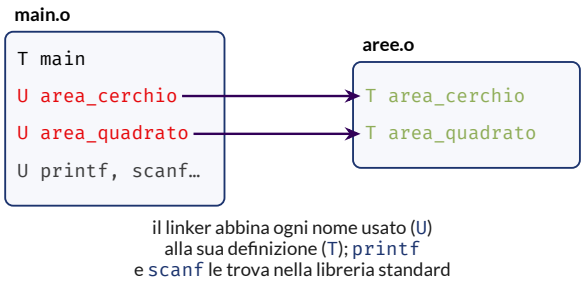
Terminale

```
$ gcc -Wall -c main.c
$ gcc -Wall -c aree.c
$ gcc -o aree main.o aree.o
$ ./aree
Inserisci una misura: 2
cerchio di raggio 2: area 12.5664
quadrato di lato 2: area 4
```

Che cosa contengono i due file oggetto, e come fa il linker a metterli insieme? Lo mostra `nm` (capitolo 3): in `main.o` le due funzioni sono *undefined* (`U`), cioè usate ma senza codice; in `aree.o` sono presenti (`T`). Il linker abbina ogni nome mancante con la sua definizione (schema a destra).

Terminale

```
$ nm main.o
                 U __isoc99_scanf
                 U __stack_chk_fail
                 U area_cerchio
                 U area_quadrato
0000000000000000 T main
                 U printf
$ nm aree.o
0000000000000000 T area_cerchio
0000000000000025 T area_quadrato
```



il linker abbina ogni nome usato (`U`) alla sua definizione (`T`); `printf` e `scanf` le trova nella libreria standard

Che cosa succede se si dimentica `aree.o` al momento del linking? La compilazione di `main.c` era già riuscita (grazie a `aree.h` il compilatore sapeva tutto ciò che serviva), ma il linker non trova il codice delle due funzioni:

Terminale

```
$ gcc -o aree main.o
/usr/bin/ld: main.o: in function `main':
main.c:(.text+0x54): undefined reference to `area_cerchio'
/usr/bin/ld: main.c:(.text+0x89): undefined reference to `area_quadrato'
collect2: error: ld returned 1 exit status
```

È lo stesso errore di `pow` senza `-lm` del capitolo 5: *undefined reference* significa sempre "il linker non trova il codice di questa funzione".

IL PROGETTO DELLE LEZIONI

I file

A lezione si usa un progetto su tre file, che gestisce un elenco di esami sostenuti da alcuni studenti (letti da un file di testo): `main.c`, `funzioni.c` e `studente.h` (figura 7.1). Il codice usa strumenti che vedremo più avanti (strutture, liste, file), e verrà spiegato per intero nel capitolo **??**; per capire `make` basta sapere chi dipende da chi.

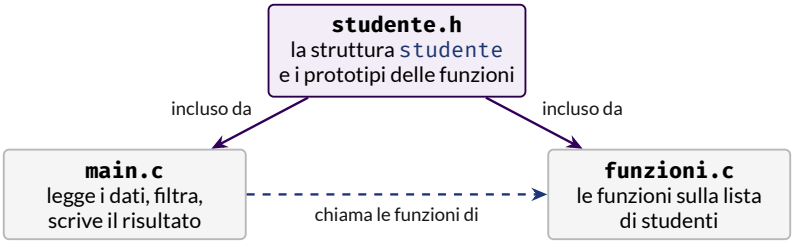


Figura 7.1: *Il progetto delle lezioni: due sorgenti e un header condiviso.*

L'eseguibile si crea come abbiamo visto: la compilazione (`gcc -c main.c funzioni.c`, che si può fare anche con un solo comando per più file) produce `main.o` e `funzioni.o`; il linking (`gcc -o esercizio main.o funzioni.o`) produce l'eseguibile `esercizio`.

Il problema

Con molti file la compilazione può essere lunga, e non sempre è necessario ricompilare tutti i sorgenti: se si modifica solo `funzioni.c`, `main.o` è ancora valido. Vorremmo rendere le operazioni più semplici e automatiche.

Un primo passo è uno **script**: un file di testo che contiene i comandi, da eseguire tutti in fila.

compila.sh

```
#!/bin/bash
gcc -c main.c
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

Terminale

```
$ chmod +x compila.sh
$ ./compila.sh
```

La prima riga (`#!/bin/bash`, detta *shebang*) dice al sistema quale programma deve eseguire lo script: la shell bash. `chmod +x` rende il file eseguibile, una volta per tutte; da lì in poi `./compila.sh` esegue i tre comandi. Il procedimento è automatico, ma i sorgenti vengono comunque compilati **tutti**, ogni volta. Vorremmo un meccanismo che compila solo i file che sono stati modificati.

L'idea: dipendenze e date

Il comando `make` esegue la compilazione in modo automatico seguendo, fondamentalmente, due regole:

- la **dipendenza** fra i file: `esercizio` si costruisce da `main.o` e `funzioni.o`; `main.o` si costruisce da `main.c` (e da `studente.h`, che `main.c` include), e così via (figura 7.2);
- l'**ordine temporale** in cui sono stati creati o modificati i file sorgente, oggetto ed eseguibile: ogni file ha una data e ora di ultima modifica, che `ls -l` mostra.

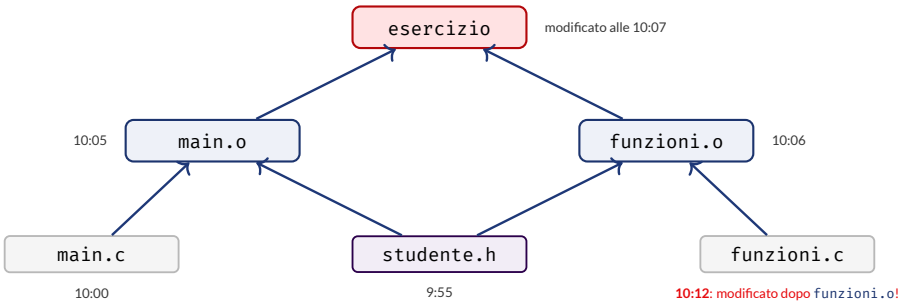


Figura 7.2: *Il grafo delle dipendenze del progetto: le frecce vanno da ciò che serve a ciò che si costruisce.* `funzioni.c` *è più recente di* `funzioni.o`*, quindi* `funzioni.o` *va ricostruito, e di conseguenza anche* `esercizio`*.*

Il ragionamento di `make` è quello che faremmo noi: per costruire un file, prima si assicura che siano aggiornati i file da cui dipende (e così via, all'indietro); poi, se il file **non esiste** oppure è **più vecchio** di almeno uno dei file da cui dipende, lo ricostruisce eseguendo i comandi indicati. Altrimenti non fa niente. Nella situazione della figura 7.2, `make` ricompila solo `funzioni.c` e rifà il linking; `main.c` non viene toccato.

Il makefile

`make` esegue le operazioni descritte in un file, chiamato di solito `makefile` o `Makefile`, che contiene la descrizione delle dipendenze, i comandi da eseguire e, eventualmente, delle definizioni di *macro*. Per usarlo basta scrivere `make` nella cartella che lo contiene; se il file ha un altro nome, per esempio `makeEsercizio`, lo si indica con l'opzione `-f`:

Terminale

```
$ make
$ make -f makeEsercizio
```

Ecco il primo makefile delle lezioni. Nei listati di questo capitolo il carattere di tabulazione (TAB) è disegnato come una freccia ⟶, come fa VS Code quando si chiede di mostrare gli spazi bianchi: il motivo sarà chiaro fra poco.

makefile1

```
1  # makefile di esempio
2  # versione 1
3  #
4
5  eseguibile: main.o funzioni.o
6  ⟶ gcc -o esercizio main.o funzioni.o
7
8  main.o: main.c
9  ⟶ gcc -c main.c
10
11 funzioni.o: funzioni.c
12 ⟶ gcc -c funzioni.c
```

I tipi di riga

In un makefile si distinguono quattro tipi di righe:

- le righe di **dipendenza**: definiscono una relazione fra un **target** (l'obiettivo, ciò che si costruisce) e uno o più file da cui dipende, separati dal carattere `:`. Per esempio `main.o: main.c`;
- le righe di **comando**: definiscono l'operazione da eseguire per ottenere il target, e seguono la sua riga di dipendenza. **Devono cominciare con un TAB**;
- le righe di **commento**: cominciano con `#`;
- le righe di definizione di **macro**, che vedremo fra poco.

Il TAB non è uno spazio

Le righe di comando devono cominciare con un carattere di **tabulazione**, non con degli spazi, anche se a occhio sono identici. Il file `makefile4` delle lezioni contiene proprio questo errore (quattro spazi al posto del TAB), e `make` si rifiuta di proseguire con un messaggio poco chiaro: *makefile4:9: *** missing separator. Stop.* VS Code inserisce davvero un TAB solo se riconosce il file come makefile, cioè se il file si chiama `makefile` o `Makefile`: in basso a destra, nella barra di stato, compare allora *Tab Size* invece di *Spaces*.

Che cosa fa `make`, passo per passo

Proviamo il primo makefile. La prima volta mancano tutti i file oggetto e l'eseguibile, quindi `make` esegue tutto, cominciando dai file da cui dipende il primo target:

Terminale

```
$ make -f makefile1
gcc -c main.c
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

`make` stampa ogni comando prima di eseguirlo. Si noti l'ordine: per costruire `eseguibile` servono `main.o` e `funzioni.o`, quindi prima vengono costruiti loro. Se ora modifichiamo soltanto `funzioni.c` (il comando `touch` cambia la data di un file, come se lo avessimo modificato e salvato) e rilanciamo `make`, viene ricompilato solo quello:

Terminale

```
$ touch funzioni.c
$ make -f makefile1
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

Le dipendenze dall'header

Che cosa succede se modifichiamo `studente.h`? Con il primo makefile, niente di buono:

Terminale

```
$ touch studente.h
$ make -f makefile1
gcc -o esercizio main.o funzioni.o
```

Nessun file viene ricompilato, perché nel makefile i file oggetto dipendono solo dai rispettivi `.c`. Ma entrambi i `.c` **includono** `studente.h`: se nell'header cambia, per esempio, la struttura `studente`, i due file oggetto sono stati compilati con la versione vecchia e il programma risultante è incoerente, con errori imprevedibili. La soluzione è dichiarare la dipendenza:

makefile2

```
1  # makefile di esempio
2  # versione 2
3
4  eseguibile: main.o funzioni.o
5  ⟶ gcc -o esercizio main.o funzioni.o
6
7  main.o: main.c studente.h
8  ⟶ gcc -c main.c
9
10 funzioni.o: funzioni.c studente.h
11 ⟶ gcc -c funzioni.c
```

Terminale

```
$ make -f makefile2
gcc -c main.c
gcc -c funzioni.c
gcc -o esercizio main.o funzioni.o
```

Ora, dopo una modifica all'header, vengono ricompilati entrambi i sorgenti. La regola generale: **un file oggetto dipende dal suo `.c` e da tutti gli header (nostri) che questo include**.

Il nome del target

Un'altra stranezza: con i primi due makefile il linking viene eseguito **ogni volta**, anche se nulla è cambiato. Possiamo evitarlo? Il motivo è il nome del target: `eseguibile`. `make` controlla se esiste un file con quel nome e se è aggiornato; ma il comando produce un file chiamato `esercizio`, quindi un file `eseguibile` non esiste mai, e `make` rifà il linking per crearlo. Basta che il target abbia lo stesso nome del file prodotto:

makefile3

```
# makefile di esempio
# versione 3

esercizio: main.o funzioni.o
	gcc -o esercizio main.o funzioni.o

main.o: main.c studente.h
	gcc -c main.c

funzioni.o: funzioni.c studente.h
	gcc -c funzioni.c
```

Terminale

```
$ make -f makefile3
make: 'esercizio' is up to date.
```

Tutto è aggiornato, e `make` lo dice senza eseguire nulla.

Le macro

Una riga di definizione di **macro** definisce un nome simbolico per un insieme di oggetti, con la sintassi `NOME = valore`; il nome si usa poi scrivendo `$(NOME)`. È l'equivalente del `#define` del C: si scrive l'elenco una volta sola, e se cambia si modifica in un solo punto.

makefile4

```
# makefile di esempio
# versione 4

OBJ = funzioni.o main.o

esercizio: $(OBJ)
	gcc -o esercizio $(OBJ)

main.o: main.c studente.h
	gcc -c main.c

funzioni.o: funzioni.c studente.h
	gcc -c funzioni.c
```

(Questa è la versione corretta, con il TAB; nel file distribuito con le lezioni la riga del linking comincia con quattro spazi e produce l'errore *missing separator*.)

UN MAKEFILE COMPLETO

Nella pratica i makefile usano quasi sempre qualche macro in più e un target di servizio per fare pulizia. Ecco un makefile per il progetto `aree` di inizio capitolo:

Makefile

```
CC = gcc
CFLAGS = -Wall
OBJ = main.o aree.o

aree: $(OBJ)
	$(CC) $(CFLAGS) -o aree $(OBJ)

main.o: main.c aree.h
	$(CC) $(CFLAGS) -c main.c

aree.o: aree.c aree.h
	$(CC) $(CFLAGS) -c aree.c

clean:
	rm -f aree $(OBJ)
```

Terminale

```
$ make
gcc -Wall -c main.c
gcc -Wall -c aree.c
gcc -Wall -o aree main.o aree.o
$ make
make: 'aree' is up to date.
$ touch main.c
$ make
gcc -Wall -c main.c
gcc -Wall -o aree main.o aree.o
$ make clean
rm -f aree main.o aree.o
```

- `CC` e `CFLAGS` sono i nomi tradizionali delle macro per il compilatore e le sue opzioni: per aggiungere un'opzione a tutte le compilazioni (per esempio `-g`, che serve al debugger) basta cambiare una riga;
- senza argomenti, `make` costruisce il **primo** target del file (qui `aree`); con `make main.o` si costruisce solo quello;
- `clean` non corrisponde a nessun file e non ha dipendenze: è un target "di servizio", che con `make clean` cancella l'eseguibile e i file oggetto, per ripartire da zero.

make non è solo per il C

`make` non sa nulla di C: sa solo che certi file si costruiscono da altri file eseguendo dei comandi, se sono più vecchi. Per questo si usa anche per compilare documenti, generare siti, elaborare dati: ovunque ci sia una catena di file da ricostruire quando qualcosa cambia.

In sintesi

Un programma si divide in più file `.c`, ciascuno compilato separatamente in un file oggetto (`gcc -c`), e in header `.h` con le dichiarazioni condivise; il linker unisce gli oggetti, abbinando ogni funzione usata alla sua definizione (*undefined reference* vuol dire che una definizione manca). `make` legge un **makefile** fatto di righe di dipendenza (`target: file`), righe di comando (che cominciano con un **TAB**), commenti e macro, e ricostruisce un target solo se non esiste o è più vecchio di qualcosa da cui dipende. Gli header vanno elencati fra le dipendenze, e il nome del target deve essere quello del file prodotto.

Riferimenti

- **GNU make — An Introduction to Makefiles**, il manuale ufficiale.
  https://www.gnu.org/software/make/manual/html_node/Introduction.html
- **Deitel & Deitel**, *Il linguaggio C*: appendice sulla compilazione di programmi su più file.